

Power Side-Channel Evaluation and Hardening of PQC Algorithms on OpenTitan

Master Thesis**Author(s):**

Hakim, Filali

Publication date:

2025-03

Permanent link:

<https://doi.org/https://doi.org/10.3929/ethz-c-000782676>

Rights / license:

In Copyright - Non-Commercial Use Permitted

DEPARTMENT OF INFORMATION TECHNOLOGY AND
ELECTRICAL ENGINEERING

Autumn Semester 2024

Power Side-Channel Evaluation and Hardening of PQC Algorithms on OpenTitan

Master Project

Hakim Filali
hfilali@student.ethz.ch

March 2025

Supervisors: Dr. Pirmin Vogel, vogelpi@lowrisc.org
Dr. Pascal Nasahl, nasahlpa@lowrisc.org
Christopher Reinwardt, creinwar@iis.ee.ethz.ch

Professor: Prof. Dr. L. Benini, lbenini@ethz.ch

Abstract

The rise of quantum computing threatens the security of many existing cryptographic algorithms, driving the need for post-quantum cryptography (PQC). One such algorithm, the Module Lattice Digital Signature Algorithm (ML-DSA), derived from CRYSTALS-Dilithium, was standardized by the National Institute of Standards and Technology (NIST) in 2024. ML-DSA enables secure digital signature generation and verification in the face of the rise of quantum computing. However, given its recent standardization, ongoing research explores its side-channel security and efficiency.

This work focuses on the power side-channel-resistant implementation, analysis, and acceleration of the security-sensitive components of the ML-DSA signing algorithm on the OpenTitan Big Number (OTBN) core. It identifies vulnerable sub-functions, develops hardened implementations, and evaluates their resistance through side-channel evaluation on an FPGA. Additionally, this thesis conducts a comprehensive performance analysis of the full algorithm and its sub-functions. Based on these performance numbers, this thesis designs and implements a hardware accelerator to enhance performance. The results demonstrate a 30% reduction in execution time for the ML-DSA sign algorithm with only a 1.5% increase in OTBN area.

Acknowledgements

I am deeply grateful to my advisors for their unwavering support and invaluable insights throughout my master's thesis. Their guidance has been instrumental in shaping this work. I especially appreciate my supervisors for their commitment, consistently providing direction during our weekly meetings.

I would like to extend my sincere thanks to Pascal Nasahl for his insightful research pointers and detailed explanations whenever needed. His introduction to the power side-channel measurement setup at lowRISC was particularly invaluable. I am also grateful to Christopher Reinwardt for handling administrative tasks and offering valuable expertise in RTL design, FPGAs, and many other technical aspects. Additionally, I thank Pirmin Vogel and Pascal Nasahl for helping me identify a deeply engaging research topic for my thesis.

I am thankful to lowRISC for providing me with the opportunity to conduct this research using their infrastructure. Their setup and code repositories have been a pleasure to work with.

I would also like to express my appreciation to Professor Benini and the IIS for allowing me to undertake my semester thesis in collaboration with the IIS, making this an enriching learning experience.

Finally, I am immensely grateful to my friends and family for their unwavering support throughout my studies. Their encouragement allowed me to stay focused and dedicated to my work.

Declaration of Originality

I hereby confirm that I am the sole author of the written work here enclosed and that I have compiled it in my own words. Parts excepted are corrections of form and content by the supervisor. For a detailed version of the declaration of originality, please refer to Appendix B

Hakim Filali,
Zurich, March 2025

Contents

1. Introduction	1
2. Background	4
2.1. ML-DSA	4
2.1.1. Learning with Errors	4
2.1.2. Short Integer Solution Problem	5
2.1.3. The ML-DSA Signature Scheme	5
2.2. Number Theoretic Transform	11
2.2.1. The Cooley-Tukey Technique for Fast NTT	13
2.2.2. The Gentleman-Sande Technique for Fast INTT	13
2.3. Power Side-Channel Analysis	14
2.3.1. Masking as a Countermeasure	15
2.3.2. Masked Gadgets	16
2.3.3. Test Vector Leakage Assessment (TVLA)	19
2.4. OpenTitan	20
2.4.1. OpenTitan Big Number Accelerator (OTBN)	20
3. Related Work	21
3.1. Post Quantum Cryptography Implementations	21
3.2. Sensitivity Analysis of Post Quantum Cryptography Algorithms	22
3.3. Boolean to Arithmetic Mask Conversion	22
4. Hardening ML-DSA Sign on OTBN	23
4.1. ML-DSA Sign Sensitivity Analysis	23
4.2. Secure NTT	25
4.2.1. The Security of <i>secNTT</i>	26
4.3. Secure INTT	26
4.3.1. The Security of <i>secINTT</i>	27
4.4. Secure Vector Addition	28
4.4.1. The Security of <i>secVecAdd</i>	28

Contents

4.5.	Secure Vector Subtraction	29
4.5.1.	The Security of <i>secVecSub</i>	29
4.6.	Secure Coefficient-Wise Vector Multiplication	29
4.6.1.	The Security of <i>secVecMul</i>	30
4.7.	Secure Vector Coefficient-Wise Multiply and Accumulate	30
4.7.1.	The Security of <i>secVecMac</i>	31
4.8.	Secure And	31
4.8.1.	The Security of <i>secAnd</i>	32
4.9.	Secure Add	32
4.9.1.	The Security of <i>secAdd</i>	33
4.10.	Secure Add Modulo q	34
4.10.1.	The Security of <i>secAddm</i>	34
4.11.	Secure Arithmetic to Boolean Mask Conversion	35
4.11.1.	The Security of <i>SecA2BMod_k^d</i>	35
4.12.	Secure Boolean to Arithmetic Mask Conversion	36
4.12.1.	The Security of <i>SecB2AMod_k^d</i>	38
4.13.	Secure Bound Check	38
4.13.1.	The Security of <i>SecBoundCheck_{q,λ₀,λ₁}^d</i>	39
4.14.	Secure Decompose	39
4.14.1.	The Security of <i>SecDecompose</i>	40
5.	Analysis	41
5.1.	Security Analysis	41
5.1.1.	Importance of FPGA Side-Channel Evaluation	41
5.1.2.	The Measurement Setup	42
5.1.3.	Challenges While Hardening the Sub-Functions	43
5.1.4.	Power Side-Channel Leakage Analysis	44
5.2.	Performance Analysis	58
5.2.1.	Memory Utilization Analysis	58
5.2.2.	Cycle Analysis	60
6.	Hardware Accelerator for Boolean to Arithmetic Mask Conversion	63
6.1.	Design	64
6.1.1.	Boolean to Arithmetic Mask Conversion Modulo q	64
6.1.2.	Boolean to Arithmetic Conversion Modulo 2^k	66
6.1.3.	Borrow Bit Generator	68
6.2.	Functional Verification	74
6.3.	Results	75
7.	Conclusion and Future Work	78
7.1.	Summary	78
7.2.	Key Findings	78
7.3.	Limitations	79
7.4.	Future Work	79

Contents

7.5. Final Words	79
A. Task Description	81
B. Declaration of Originality	91
C. File Structure	93

List of Figures

2.1. Simulatability examples. All outputs (bold, red) are probed. Inputs needed for simulation are circled in blue [1].	17
2.2. Implementation of $(x + y)(x + z)$ masked with $d = 2$ shares, with input sharings (x_0, x_1) , (y_0, y_1) and (z_0, z_1) . The circuit is made of a 1-SNI multiplication with linear circuits at its inputs (two trivial implementations of the addition). The circuits illustrate (a) the limitation of SNI input composability and (b) a simple fix. The arrows indicate the adversarial probes (there is thus one internal probe in the multiplication) and the red snake wires are the propagated probes. The R box is a 1-SNI refresh gadget [1].	18
4.1. A graphical representation of the signature generation process is shown. The input consists of the secret key (sk) and message (M), while the output is the signature $\sigma = (\tilde{c}, z, h)$. Curved arrows indicate rejection checks. Red elements represent sensitive operations and variables and blue elements represent non-sensitive operations and variables [2].	24
5.1. The measurement setup including the target FPGA board, the CW Husky scope and the ot-sca side-channel evaluation framework.	43
5.2. The unmasked TVLA results for the NTT sub-function.	45
5.3. The masked TVLA results for the NTT sub-function.	46
5.4. The unmasked TVLA results for the INTT sub-function.	47
5.5. The masked TVLA results for the INTT sub-function.	47
5.6. The unmasked TVLA results for the Vector Add sub-function.	48
5.7. The masked TVLA results for the Vector Add sub-function.	49
5.8. The unmasked TVLA results for the Vector Subtract sub-function.	49
5.9. The masked TVLA results for the Vector Subtract sub-function.	50
5.10. The unmasked TVLA results for the Vector Multiply sub-function.	51
5.11. The masked TVLA results for the Vector Multiply sub-function.	51

List of Figures

5.12. The unmasked TVLA results for the Vector Multiply and Accumulate sub-function.	52
5.13. The masked TVLA results for the Vector Multiply and Accumulate sub-function.	53
5.14. The unmasked TVLA results for the Bound Check sub-function.	54
5.15. The masked TVLA results for the Bound Check sub-function.	55
5.16. The first part of the unmasked TVLA results for the Decompose sub-function.	56
5.17. The second part of the unmasked TVLA results for the Decompose sub-function.	56
5.18. The first part of the masked TVLA results for the Decompose sub-function.	57
5.19. The second part of the masked TVLA results for the Decompose sub-function.	58
6.1. The top level block for the Boolean to Arithmetic mask conversion modulo q <i>B2Amodq</i>	64
6.2. Boolean-to-Arithmetic conversion block (B2A_2k).	66
6.3. The Ripple Carry Adder used in the Boolean to Arithmetic mask conversion modulo 2^k <i>B2A2kRippleCarry</i>	67
6.4. This block only contains an <i>AND</i> and an <i>XOR</i> gate and is there to simplify the Ripple Carry Adder block diagram.	68
6.5. The <i>DOMf</i> block, which can be used as <i>secAnd</i> , <i>secAndRefresh</i> , <i>secAndXor</i> or <i>secAndXorRefresh</i>	69
6.6. This figure explains how the indices should be combined in each stage.	70
6.7. Stage one of the secure Kogge-Stone Adder implementation <i>borrowBitGenStage1</i>	71
6.8. This block implements a protected version of the carry generate bit calculation while also adding $u = 2^k - q$	72
6.9. Stages two to four of the secure Kogge-Stone Adder implementation <i>borrowBitGenStage2-4</i>	73
6.10. Stage five of the secure Kogge-Stone Adder implementation <i>borrowBitGenStage5</i>	74
6.11. Functional verification setup.	75

List of Tables

2.1. ML-DSA parameter sets [3]	6
5.1. Data memory usage broken down into the individual variables.	59
5.2. Instruction memory usage broken down into the individual secure sub- functions and their unmasked counterparts.	60
5.3. Cycle Analysis for the ML-DSA Subfunctions.	61
5.4. Total execution time in cycles split into the sub-functions mentioned in this work. The third column is the percentage compared to the estimated total execution of the hardened ML-DSA sign algorithm. The last column contains the number of additional cycles due to hardening.	62
6.1. Cycle Analysis for the accelerated ML-DSA Subfunctions.	76
6.2. Total execution time in cycles split into the sub-functions mentioned in this work using the proposed hardware accelerator. The third column is the percentage compared to the estimated total execution of the hardened ML- DSA sign algorithm. The last column contains the number of additional cycles due to hardening.	76
6.3. Resource utilization of the B2A mask conversion after synthesis and im- plementation, split up into sub-modules and compared to OTBN.	77
6.4. Resource utilization of the B2A mask conversion after synthesis and im- plementation, split up into sub-modules and compared to OTBN.	77

Chapter 1

Introduction

Most cryptographic algorithms in use today rely on the hardness of problems such as the discrete logarithm and integer factorization problems. However, with the advancement of quantum computing, these problems are expected to be solvable in polynomial time once quantum computers with a certain amount of sufficiently stable qubits are available. This threatens existing cryptographic algorithms, as messages encrypted using them could be decrypted once such quantum computers emerge.

To address this risk, the National Institute of Standards and Technology (NIST) initiated a standardization process to identify quantum-resistant cryptographic algorithms. One prominent outcome of this effort is CRYSTALS-Dilithium, a digital signature scheme. These algorithms are based on lattice-based problems, such as the Short Integer Solution (SIS) and Learning with Errors (LWE) problems, which are believed to be hard to solve, even for quantum computers. In August 2023, NIST published a draft of the Module-Lattice-Based Digital Signature Standard (ML-DSA), derived from CRYSTALS-Dilithium, with the final version released in August 2024.

While cryptographic algorithms may be theoretically secure—ensuring secrets cannot be derived from public information—real world implementations remain vulnerable to side-channel attacks. Adversaries can exploit timing and power side-channels by analyzing computation time or power consumption patterns to extract sensitive data. These risks underscore the need for side-channel-resistant implementations to safeguard post-quantum cryptographic schemes, as power side-channels can be leveraged in attacks.

Contributions

This work aims to lay the groundwork for a power side-channel-resistant implementation of the ML-DSA signing algorithm. It begins by reviewing existing research on proposed

1. Introduction

hardened implementations and adapting these techniques in assembly on the OpenTitan Big Number (OTBN) core, a coprocessor for asymmetric cryptographic operations. The implemented design is then analyzed for side-channel leakage on an FPGA, with improvements applied where necessary.

Next, the final implementation is evaluated in terms of instruction and data memory usage, as well as execution time. To further optimize performance, this work identifies sub-routines that could benefit from hardware acceleration, explores methods to enhance their efficiency, and implements a hardware accelerator. Finally, the potential speed-up achieved by the accelerator is assessed, alongside an analysis of its resource utilization in an FPGA implementation.

Outline

The following chapters provide a comprehensive overview of this work, beginning with a foundational introduction in Chapter 2. This chapter covers the mathematical principles, tools, and techniques used throughout the work. It explains the ML-DSA algorithms and practical implementation methods, such as the Number Theoretic Transform. Additionally, it explores power side-channel attacks and strategies for hardening algorithms against them. Finally, it introduces the OpenTitan project, the platform used for implementing the hardened ML-DSA.

Chapter 3 reviews related work in the field, providing context for this study. It offers a concise overview of previous research on ML-DSA implementations and general hardening techniques used to mitigate power side-channel attacks.

Chapter 4 focuses on hardening the ML-DSA signing algorithm. It begins with a sensitivity analysis to identify security-critical variables and sub-functions. The chapter then details the hardening techniques applied to these sub-functions, explaining their effectiveness in mitigating side-channel attacks. Additionally, it introduces supplementary security gadgets essential for protecting the ML-DSA signing process, including software implementations of adders, bitwise AND operations, and mask conversion algorithms.

Chapter 5 presents a comprehensive analysis of the hardened implementations for security-sensitive sub-functions. It begins with a security evaluation, assessing power side-channel leakage in the hardened sub-functions. Next, it examines data memory and instruction memory utilization, along with the execution time of individual sub-functions. The results show a 30% reduction in execution time for the initial hardened ML-DSA sign implementation, with only a 1.5% increase in Look Up Table (LUT) resource utilization compared to OTBN. Finally, the chapter estimates the overall execution time of the fully hardened ML-DSA signing algorithm.

Chapter 6 introduces a hardware accelerator designed to enhance the performance of the hardened ML-DSA signing algorithm. Specifically, it accelerates the Boolean-to-Arithmetic mask conversion, a procedure needed for the sub-function with the longest

1. Introduction

execution time. The chapter provides a detailed explanation of the overall algorithm and the individual hardware blocks used in the design. It concludes with an analysis of the hardware implementation, including resource utilization on an FPGA. Additionally, the chapter presents an estimation of the potential speed-up achieved in the hardened ML-DSA signing algorithm using the hardware accelerator.

Chapter 2

Background

This chapter provides the foundational knowledge and key concepts that underpin the research presented in this work. The topics covered include a brief explanation of the cryptographic signing algorithm ML-DSA, power side-channel analysis, masking techniques, and mathematical concepts used in this thesis. This background section sets the stage for the more advanced discussions and analyses that follow in the subsequent chapters of the thesis.

2.1. ML-DSA

The Module-Lattice-Based Digital Signature Algorithm (ML-DSA) [3], derived from CRYSTALS-Dilithium, is a digital signature scheme chosen by NIST as part of its Post-Quantum Cryptography (PQC) standardization process. It was selected as a quantum-resistant algorithm for secure public-key cryptography in a post-quantum world.

The security of the signature scheme relies on the computational hardness of two well-established problems: a nonstandard variant of the Module Short Integer Solution (MSIS) problem [4] and the Module Learning With Errors (MLWE) problem [5]. Both are believed to be resistant to attacks even by quantum computers, forming the foundation of the scheme's robustness.

2.1.1. Learning with Errors

To better understand the Learning With Errors (LWE) problem, it is helpful to first examine the simpler case of Learning Without Errors. In this scenario, we have a secret vector $\mathbf{x} \in \mathbb{Z}_q^n$, where $\mathbb{Z}_q^n$ represents a polynomial ring of degree n with modulus q . We generate a system of linear equations $A\mathbf{x} = \mathbf{c}$, where $A \in \mathbb{Z}_q^{m \times n}$ is a public matrix and $\mathbf{c} \in$

2. Background

$\mathbb{Z}_q^m$ is a public vector. In this case, solving for the secret vector $\mathbf{x}$ is straightforward, for example, by using Gaussian elimination, especially when the system is over-determined (i.e., when $m > n$).

In the Learning With Errors (LWE) problem, we introduce a secret error vector $\mathbf{e}$ to the system, modifying the linear equation to $A\mathbf{x} = \mathbf{c} + \mathbf{e}$. With the addition of this small error, finding the secret vectors $\mathbf{x}$ and $\mathbf{e}$ becomes computationally hard in the average case, making the problem difficult to solve efficiently even for powerful adversaries, including quantum computers.

2.1.2. Short Integer Solution Problem

The Short Integer Solution (SIS) problem is dual to the LWE problem, meaning if one of them can be solved in polynomial time, the other can be as well. In the SIS problem, we are given a matrix $A \in \mathbb{Z}_q^{n \times m}$ and a target vector $\mathbf{y} = A\mathbf{x} \in \mathbb{Z}_q^n$, where the vector $\mathbf{x} \in \{0, 1\}^m$ is composed of binary values. The challenge is to find a vector $\mathbf{x} \in \{0, 1\}^m$ that satisfies the equation $A\mathbf{x} = \mathbf{y}$. This problem is considered hard to solve in the average case.

It is relatively straightforward to find an arbitrary integer solution $\mathbf{t}$ to the system of linear equations $A\mathbf{t} = \mathbf{y} \pmod{q}$. However, the solution space forms a lattice of possible solutions, spanned by the particular solution $\mathbf{t}$ and the kernel of the matrix A . The challenge lies in finding a solution $\mathbf{x} = \mathbf{t} + \mathbf{k}$, where $\mathbf{k} \in \{\mathbf{z} \in \mathbb{Z}^m \mid A\mathbf{z} = \mathbf{0}\}$, such that $\mathbf{x} \in \{0, 1\}^m$. This constrained search within the binary space makes the problem computationally hard.

The SIS and LWE problems form the foundational principles behind the relationship between components of the ML-DSA public key, such as $\mathbf{t}$ and $\mathbf{A}$, and parts of the secret key, $\mathbf{s}_1$ and $\mathbf{s}_2$. Specifically, the public key $\mathbf{t}$ is related to the secret keys through the equation $\mathbf{t} = \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2$. The hardness of the LWE problem ensures that deriving the secrets $\mathbf{s}_1$ and $\mathbf{s}_2$ from the public key containing $\mathbf{t}$ and $\mathbf{A}$ is computationally hard. The SIS problem, as incorporated in the signature scheme, centers on the difficulty of finding a non-zero solution $\mathbf{t} \in \mathbb{Z}^n$ to a linear system of the form $A\mathbf{t} = \mathbf{0} \pmod{q}$, where $\|\mathbf{t}\|_\infty$ is small.

2.1.3. The ML-DSA Signature Scheme

As stated in [3] the main idea behind the protocol involves three steps where a prover can demonstrate to a verifier that he possesses the private key:

1. Commitment: The prover generates $\mathbf{y} \in \mathbb{Z}_q^L$ with small coefficients and commits to its value by sending $\mathbf{w}_{approx} = \mathbf{A}\mathbf{y} + \mathbf{y}_2$ to the verifier, where $\mathbf{y}_2 \in \mathbb{Z}_q^K$ is a vector with small coefficients.

2. Background

2. Challenge: The verifier sends a vector $\mathbf{c} \in \mathbb{Z}_q^n$ with small coefficients to the prover.
3. Response: The prover returns $\mathbf{z} = \mathbf{y} + \mathbf{s}_1 \mathbf{c}$, and the verifier checks that $\mathbf{z}$ has small coefficients and that $\mathbf{A}\mathbf{z} - \mathbf{T}\mathbf{c} \approx \mathbf{w}_{approx}$.

This protocol is turned non-interactive by replacing the challenge vector c with a process that pseudo-randomly derives c from a hash of the commitment concatenated with the message to be signed.

As per usual the signature scheme consists of a triple of algorithms. Namely the key generation Algorithm 1, the signature generation Algorithm 2, and the verification Algorithm 3.

Table 2.1.: ML-DSA parameter sets [3]

Parameters	ML-DSA-44	ML-DSA-65	ML-DSA-87
q - modulus	8380417	8380417	8380417
ζ - a 512th root of unity in $\mathbb{Z}_q$	1753	1753	1753
d - # of dropped bits from t	13	13	13
τ - # of ± 1 's in polynomial c	39	49	55
λ - collision strength of $\tilde{c}$	128	192	256
γ_1 - coefficient range of y	2^{17}	2^{19}	2^{19}
γ_2 - low-order rounding range	$(q-1)/88$	$(q-1)/32$	$(q-1)/32$
(k, ℓ) - dimensions of A	(4, 4)	(5, 6)	(6, 8)
η - private key range	2	2	2
$\beta = \tau \cdot \eta$	78	98	110
ω - max # of 1's in the hint h	80	55	75
Challenge entropy $\log_2 \left(\frac{2^{256}}{\tau} \right) + \tau$	192	225	257
Average number of signing iterations	4.25	3.5	3.85
Claimed security strength	Category 2	Category 3	Category 5

As shown in Table 2.1, ML-DSA defines three parameter sets: ML-DSA-44, ML-DSA-65, and ML-DSA-87, ordered by increasing security strength. This work primarily focuses on the ML-DSA-44 variant.

The naming convention for these parameter sets follows the format "ML-DSA- kl ", where (k, l) represent the dimensions of the matrix A .

ML-DSA Key Generation

As described in [3], the Key Generation Algorithm 1 takes a 32-byte random seed as input and produces a public-private key pair encoded as a set of byte strings. The

2. Background

input seed, denoted by ξ , is expanded using an Extendable Output Function (XOF) like SHAKE256 [6], represented by H , to generate additional random values. These values include:

- ρ : A 32-byte public random seed used to pseudo-randomly sample a polynomial matrix $\mathbf{A} \in \mathbb{R}_q^{k \times l}$, where $\mathbb{R}_q^{k \times l}$ is a matrix of k polynomials of degree l , defined over the ring $\mathbb{Z}_m \bmod X^{256} + 1$, with $\mathbb{Z}_m$ representing the integers modulo m .
- ρ' : A 64-byte private random seed used to pseudo-randomly generate the polynomial vectors $\mathbf{s}_1 \in \mathbb{R}_q^l$ and $\mathbf{s}_2 \in \mathbb{R}_q^k$, where each coordinate polynomial has small coefficients bounded within the range $[-\eta, \eta]$.
- K : A 32-byte private random seed used during the signing process.

The primary computation involved in generating the private-public key pair is the calculation of the public value $\mathbf{t} = \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2$. Here, both $\mathbf{A}$ and $\mathbf{t}$ are considered public. For performance optimization, the value $\mathbf{t}$ is compressed into the vector $\mathbf{t}_1$ by removing its lower-order bits. Although these bits are discarded, they can be reconstructed from a small number of signatures, making them effectively public. The vector $\mathbf{t}_0$ stores the d least significant bits of each coefficient of $\mathbf{t}$. Additionally, tr is a hash of the public key, while the private random seed K is reserved for use during the signing process.

Algorithm 1: ML-DSA.KeyGen_internal(ξ)

input : Seed $\xi \in \mathbb{B}^{32}$

output: Public key $pk \in \mathbb{B}^{32+32k(bitlen(q-1)-d)}$
and private key $sk \in \mathbb{B}^{32+32+64+32 \cdot ((l+k) \cdot bitlen(2\eta) + dk)}$

- 1 $(\rho, \rho', K) \in \mathbb{B}^{32} \times \mathbb{B}^{64} \times \mathbb{B}^{32} \leftarrow H(\xi || IntegerToBytes(k, 1) || IntegerToBytes(l, 1), 128)$
▷ expand seed
 - 2 $\hat{\mathbf{A}} \leftarrow ExpandA(\rho)$ ▷ $\mathbf{A}$ is generated and stored in NTT representation as $\hat{\mathbf{A}}$
 - 3 $(s_1, s_2) \leftarrow ExpandS(\rho')$
 - 4 $\mathbf{t} \leftarrow NTT^{-1}(\hat{\mathbf{A}} \circ NTT(s_1)) + s_2$ ▷ compute $\mathbf{t} = \mathbf{A}s_1 + s_2$
 - 5 $(t_1, t_0) \leftarrow PowerTwoRound(\mathbf{t})$ ▷ PowerTwoRound is applied componentwise
 - 6 $pk \leftarrow pkEncode(\rho, t_1)$
 - 7 $tr \leftarrow H(pk, 64)$
 - 8 $sk \leftarrow skEncode(\rho, K, tr, s_1, s_2, t_0)$ ▷ K and tr are for use in signing
 - 9 **return**(pk, sk)
-

ML-DSA Signing

As outlined in [3], the Signing Algorithm 2 takes a private key pk , a formatted message M' and a 32-byte value rnd all encoded as byte strings as inputs and outputs a signature encoded as a byte string.

2. Background

The signer first extracts the following from the private key:

- ρ : a public random seed, which is expanded to the matrix $\mathbf{A}$.
- K : a 32-byte private random seed.
- tr : a 64-byte hash of the public key.
- $\mathbf{s}_1$ and $\mathbf{s}_2$: two secret polynomial vectors.
- $\mathbf{t}_0$: polynomial vector encoding the d least significant bits of each coefficient of the public key polynomial t .

The message M is concatenated with the public key hash tr and then hashed using H to produce a 64-byte message representative, μ , which is then used to produce the signature. Additionally, private randomness is generated by computing $\rho'' \leftarrow H(K || rnd || \mu, 64)$, where K is the private seed. There are two variants of the signing algorithm: in the hedged variant, rnd is a random number, while in the deterministic variant, rnd consists entirely of zeros.

The core of the signature algorithm is a rejection sampling loop, which ensures that the signature does not inadvertently reveal information about the private key. This loop repeats until a valid signature is generated. Within the loop, the signer first computes a commitment $\mathbf{w}_1$, from which a challenge $\mathbf{c}$ is derived, in combination with the message representative μ . Finally, the signer calculates a response $\mathbf{z}$.

In greater detail, the following steps occur within the rejection sampling loop:

- Sampling $\mathbf{y}$: The signer pseudo-randomly samples $\mathbf{y} \in R_q^l$ from a subset of polynomial vectors with short coefficients, specifically in the range $[-\gamma_1 + 1, \gamma_1]$. The sampling is done using the ExpandMask function, which takes the seed ρ'' and a counter κ as inputs.
- Commitment $\mathbf{w}_1$: The signer computes the commitment by first calculating $\mathbf{w} = \mathbf{A}\mathbf{y}$. Then, the value is rounded to the nearest multiple of $2\gamma_2$ using the HighBits function, resulting in the commitment $\mathbf{w}_1$.
- Challenge derivation: The w1Encode function concatenates $\mathbf{w}_1$ and the message representative μ , which are then hashed to produce the commitment hash $\tilde{c}$.
- Sampling the challenge: Using the SampleInBall function, the commitment hash $\tilde{c}$ is used to pseudo-randomly sample a polynomial $c \in R_q$. The polynomial c has coefficients in $-1, 0, 1$ and a fixed Hamming weight τ .
- Response computation: The signer computes the response $\mathbf{z} = \mathbf{y} + c\mathbf{s}_1$. Several validity checks are performed to ensure security, and based on the results, the loop is either repeated or terminated.

2. Background

- Hint computation: Once the loop successfully terminates, the signer calculates a hint polynomial $\mathbf{h}$ using the MakeHint function. This hint helps the verifier later reconstruct $\mathbf{w}_1$.
- Signature creation: Finally, the signer produces the signature, which consists of the byte-encoded form of the commitment hash $\tilde{c}$, the response $\mathbf{z}$, and the hint $\mathbf{h}$.

ML-DSA Verifying

As described in [3], the Verifying Algorithm 3 takes three inputs: the public key pk , the message M , and the signature σ , all encoded as byte strings. The algorithm outputs a boolean value true if the signature is valid and false otherwise.

The verifier returns false if the input lengths differ from what is expected. If the lengths are correct, the verifier proceeds by extracting the public random seed ρ and the compressed polynomial vector $\mathbf{t}_1$ from the public key pk . Then, it extracts the signer's commitment hash $\tilde{c}$, the response $\mathbf{z}$, and the hint $\mathbf{h}$ from the signature σ . If the hint $\mathbf{h}$ is not properly byte-encoded or any other component is malformed, the verifier immediately returns false. This ensures that invalid or tampered signatures are rejected early in the process.

Similar to the signing algorithm, the verifier pseudo-randomly derives the matrix $\mathbf{A}$ from the public seed ρ and generates a message representative μ by hashing the concatenation of the message M and the public key hash. Next, the verifier attempts to reconstruct the signer's commitment $\mathbf{w}_1$ by using the public key pk and the components of the signature σ .

The commitment $\mathbf{w}_1$ is reconstructed as follows:

- Deriving the challenge polynomial: The verifier derives the challenge polynomial c from the signer's commitment hash $\tilde{c}$, following the same process used during the signing phase.
- Computing the approximate commitment: Using the signer's response $\mathbf{z}$, the verifier computes:

$$\mathbf{w}'_{approx} = \mathbf{A}\mathbf{z} - c\mathbf{t}_1 \cdot 2^d$$
 Since both c and $\mathbf{s}_2$ have small coefficients, and $\mathbf{t}_1 \cdot 2^d$ serves as an approximation of $\mathbf{t}$, we can conclude:

$$\mathbf{w} = \mathbf{A}\mathbf{y} = \mathbf{A}\mathbf{z} - c\mathbf{t} + c\mathbf{s}_2 \approx \mathbf{w}'_{approx} = \mathbf{A}\mathbf{z} - c\mathbf{t}_1 \cdot 2^d$$
- Reconstructing $\mathbf{w}'_1$ using the hint: Finally, the verifier uses the signer's hint $\mathbf{h}$ to refine $\mathbf{w}'_{approx}$ and obtain the exact value of $\mathbf{w}'_1$, matching the original commitment used in signing.

2. Background

Algorithm 2: ML-DSA.Sign_internal(sk, M', rnd)

input : Private key $sk \in \mathbb{B}^{32+32+64+32 \cdot ((l+k) \cdot \text{bitlen}(2\eta) + dk)}$, formatted message $M' \in \{0, 1\}^*$, and per message randomness or dummy variable $rnd \in \mathbb{B}^{32}$

output: Signature $\sigma \in \mathbb{B}^{\lambda/4 + l \cdot 32 \cdot (1 + \text{bitlen}(\gamma_1 - 1)) + \omega + k}$

```

1   $(\rho, K, tr, s_1, s_2, t_0) \leftarrow skDecode(sk)$ 
2   $\hat{s}_1 \leftarrow NTT(s_1)$ 
3   $\hat{s}_2 \leftarrow NTT(s_2)$ 
4   $\hat{t}_0 \leftarrow NTT(t_0)$ 
5   $\hat{A} \leftarrow ExpandA(\rho)$   $\triangleright A$  is generated and stored in NTT representation as  $\hat{A}$ 
6   $\mu \leftarrow (H(BytesToBits(tr) || M', 64))$   $\triangleright$  message representative that may optionally be
   computed in a different cryptographic module
7   $\rho'' \leftarrow H(K || rnd || \mu, 64)$   $\triangleright$  compute private random seed
8   $\kappa \leftarrow 0$   $\triangleright$  initialize counter  $\kappa$ 
9   $(z, h) \leftarrow \perp$ 
10 while  $(z, h) = \perp$  do
11  $\triangleright$  rejection sampling loop
12    $y \in R_q^l \leftarrow ExpandMask(\rho'', \kappa)$ 
13    $w \leftarrow NTT^{-1}(\hat{A} \circ NTT(y))$ 
14    $w_1 \leftarrow HighBits(w)$   $\triangleright$  signer's commitment
15    $\tilde{c} \leftarrow H(\mu || w1Encode(w_1), \lambda/4)$   $\triangleright$  commitment hash
16    $c \in R_q \leftarrow SampleInBall(\Delta)$   $\triangleright$  verifier's challenge
17    $\hat{c} \leftarrow NTT(c)$ 
18    $\langle\langle cs_1 \rangle\rangle \leftarrow NTT^{-1}(\hat{c} \circ \hat{s}_1)$ 
19    $\langle\langle cs_2 \rangle\rangle \leftarrow NTT^{-1}(\hat{c} \circ \hat{s}_2)$ 
20    $z \leftarrow y + \langle\langle cs_1 \rangle\rangle$   $\triangleright$  signer's response
21    $r_0 \leftarrow LowBits(w - \langle\langle cs_2 \rangle\rangle)$ 
22   if  $\|z\|_\infty \geq \gamma_1 - \beta$  or  $\|r_0\|_\infty \geq \gamma_2 - \beta$  then
23    $\triangleright$  validity checks
24    $(z, h) \leftarrow \perp$ 
25   else
26    $\langle\langle ct_0 \rangle\rangle \leftarrow NTT^{-1}(\hat{c} \circ \hat{t}_0)$ 
27    $h \leftarrow MakeHint(-\langle\langle ct_0 \rangle\rangle, w - \langle\langle cs_2 \rangle\rangle + \langle\langle ct_0 \rangle\rangle)$   $\triangleright$  signer's hint
28   if  $\|\langle\langle ct_0 \rangle\rangle\|_\infty \geq \gamma_2$  or the number of 1's in  $h$  is greater than
    $\omega$  then  $(z, h) \leftarrow \perp$ 
29   end
30    $\kappa \leftarrow \kappa + l$   $\triangleright$  increment counter
31 end
32  $\sigma \leftarrow sigEncode(\tilde{c}, z \bmod^\pm q, h)$ 
33 return  $\sigma$ 
34
```

2. Background

Finally, the verifier computes a commitment hash, $\tilde{c}'$, by following the same procedure used during signing, and compares it to the signer's original commitment hash, $\tilde{c}$. If $\tilde{c}'$ matches $\tilde{c}$, the verifier proceeds with additional checks to ensure the validity of the signer's hint $\mathbf{h}$ and response $\mathbf{z}$. If all these checks are successful, the verifier returns true, confirming that the signature is valid. Otherwise, if any of the checks fail, the verifier returns false, indicating that the signature is invalid.

Algorithm 3: ML-DSA.Verify_internal(pk, M', σ)

input : Public key $pk \in \mathbb{B}^{32+32k(\text{bitlen}(q-1)-d)}$ and message $M' \in \{0, 1\}^*$
input : Signature $\sigma \in \mathbb{B}^{\lambda/4+l \cdot 32 \cdot (1+\text{bitlen}(\gamma_1-1))+\omega+k}$
output: Boolean

- 1 $(\rho, t_1) \leftarrow pkDecode(pk)$
- 2 $(\tilde{c}, \mathbf{z}, \mathbf{h}) \leftarrow sigDecode(\sigma)$ ▷ signer's commitment hash $\tilde{c}$, response $\mathbf{z}$ and hint $\mathbf{h}$
- 3 **if** $\mathbf{h} = \perp$ **then return** false **end if** ▷ hint was not properly encoded
- 4 $\hat{\mathbf{A}} \leftarrow ExpandA(\rho)$ ▷ $\mathbf{A}$ is generated and stored in NTT representation as $\hat{\mathbf{A}}$
- 5 $tr \leftarrow H(pk, 64)$
- 6 $\mu \leftarrow (H(BytesToBits(tr) || M', 64))$ ▷ message representative that may optionally be computed in a different cryptographic module
- 7 $c \in R_q \leftarrow SampleInBall(\tilde{c})$ ▷ compute verifier's challenge from $\tilde{c}$
- 8 $\mathbf{w}'_{Approx} \leftarrow NTT^{-1}(\hat{\mathbf{A}} \circ NTT(\mathbf{z}) - NTT(c) \circ NTT(\mathbf{t}_1 \cdot 2^d))$ ▷
 $\mathbf{w}'_{Approx} = \mathbf{A}\mathbf{z} - c\mathbf{t}_1 \cdot 2^d$
- 9 $\mathbf{w}'_1 \leftarrow UseHint(\mathbf{h}, \mathbf{w}'_{Approx})$ ▷ reconstruction of signer's commitment
- 10 $\tilde{c}' \leftarrow H(\mu || w1Encode(\mathbf{w}'_1), \lambda/4)$ ▷ hash it; this should match $\tilde{c}$
- 11 **return** $[\|\mathbf{z}\|_\infty < \gamma_1 - \beta]$ **and** $[\tilde{c} = \tilde{c}']$

2.2. Number Theoretic Transform

The Number Theoretic Transform (NTT) is a powerful mathematical tool frequently employed in post-quantum cryptography to perform polynomial multiplications over finite rings efficiently. It reduces the computational complexity of polynomial multiplication from quadratic $\mathcal{O}(n^2)$ to log-linear $\mathcal{O}(n \log n)$. The NTT achieves this by converting a polynomial from its standard coefficient representation to an evaluated form. Conversely, the Inverse NTT (INTT) transforms the evaluated form back to the coefficient representation. Polynomial multiplication over a ring can be expressed as a convolution. When transformed into the NTT domain, this convolution simplifies to coefficient-wise multiplication, significantly reducing computational complexity.

The evaluation points in the NTT are derived from the primitive $2n$ -th roots of unity, denoted as ψ . To be able to calculate negative wrapped convolutions, ψ must satisfy the

2. Background

following two conditions:

$$\psi^2 \equiv \omega \pmod{q} \quad (2.1)$$

and

$$\psi^n \equiv -1 \pmod{q} \quad (2.2)$$

Here, ω represents a primitive n -th root of unity, and q is the modulus of the ring in which the NTT is performed.

Negacyclic convolutions, also called negative wrapped convolutions, are similar to cyclic convolutions, but with a key difference: instead of reducing by $x^n - 1$, we reduce by $x^n + 1$. This change in the reduction polynomial allows us to use the $2n$ -th root of unity ψ for our evaluation points.

As stated in [7], the Negative-Wrapped NTT of a vector of polynomial coefficients $\mathbf{a}$ is defined as $\hat{\mathbf{a}} = NTT^\psi(\mathbf{a})$, where:

$$\hat{\mathbf{a}}_j = \sum_{i=0}^{n-1} \psi^{2ij+i} a_i \pmod{q} \quad (2.3)$$

The Negative-Wrapped INTT of a vector in the evaluated form $\hat{\mathbf{a}}$ is defined as $\mathbf{a} = INTT^{\psi^{-1}}(\hat{\mathbf{a}})$, where:

$$\mathbf{a}_i = n^{-1} \sum_{j=0}^{n-1} \psi^{-(2ij+i)} \hat{a}_j \pmod{q} \quad (2.4)$$

For the INTT we have an additional scaling factor n^{-1} in front of the sum and instead of ψ we use ψ^{-1} .

Equations 2.3 and 2.4 still exhibit a complexity of $\mathcal{O}(n^2)$. However, since the NTT is essentially a Discrete Fourier Transform (DFT) defined over a different ring, well-established Fast Fourier Transform (FFT) optimization techniques can be adapted to the NTT, reducing its complexity to $\mathcal{O}(n \log n)$. Two well-known techniques are proposed by Cooley-Tukey (CT) [8] and Gentleman-Sande (GS) [9]. As stated in [7] these techniques use the periodicity and symmetry property of ψ :

$$periodicity : \psi^{k+2n} = \psi^k \quad (2.5)$$

$$symmetry : \psi^{k+n} = -\psi^k \quad (2.6)$$

Where k is a non-negative integer. Using the divide-and-conquer method and the properties described above, the complexity of the NTT and INTT is reduced, as explained in the following sections.

2. Background

2.2.1. The Cooley-Tukey Technique for Fast NTT

Starting with equation 2.3 we can divide the sum into two sums for even and odd numbers as follows:

$$\hat{\mathbf{a}}_j = \sum_{i=0}^{n-1} \psi^{2ij+i} a_i \mod q \quad (2.7)$$

$$= \sum_{i=0}^{n/2-1} \psi^{4ij+2i} a_{2i} + \sum_{i=0}^{n/2-1} \psi^{4ij+2j+2i+1} a_{2i+1} \mod q \quad (2.8)$$

$$= \sum_{i=0}^{n/2-1} \psi^{4ij+2i} a_{2i} + \psi^{2j+1} \sum_{i=0}^{n/2-1} \psi^{4ij+2i} a_{2i+1} \mod q \quad (2.9)$$

From the symmetry property 2.10 we get:

$$\hat{\mathbf{a}}_{j+n/2} = \sum_{i=0}^{n/2-1} \psi^{4ij+2i} a_{2i} - \psi^{2j+1} \sum_{i=0}^{n/2-1} \psi^{4ij+2i} a_{2i+1} \mod q \quad (2.10)$$

This means that we only have to compute half of the sums and reuse them for the calculation of the other half of the coefficients. Furthermore, by reapplying the steps in 2.7, we can reduce the complexity of the NTT from $\mathcal{O}(n^2)$ to $\mathcal{O}(n \log n)$.

2.2.2. The Gentleman-Sande Technique for Fast INTT

For the Gentleman-Sande (GS) algorithm, we start with equation 2.4. We ignore the n^{-1} term and divide the sum into the upper and lower half of indexes.

$$\mathbf{a}_i = \sum_{j=0}^{n-1} \psi^{-(2ij+i)} \hat{a}_j \mod q \quad (2.11)$$

$$= \left(\sum_{j=0}^{n/2-1} \psi^{-(2ij+i)} \hat{a}_j + \sum_{j=n/2}^{n-1} \psi^{-(2ij+i)} \hat{a}_j \right) \mod q \quad (2.12)$$

$$= \psi^{-i} \left(\sum_{j=0}^{n/2-1} \psi^{-2ij} \hat{a}_j + \sum_{j=0}^{n/2-1} \psi^{-2i(j+n/2)} \hat{a}_{j+n/2} \right) \mod q \quad (2.13)$$

Using the symmetry and periodicity properties of ψ we get an even term:

2. Background

$$\mathbf{a}_{2i} = \psi^{-2i} \left(\sum_{j=0}^{n/2-1} \psi^{-4ij} \hat{a}_j + \sum_{j=0}^{n/2-1} \psi^{-4i(j+n/2)} \hat{a}_{j+n/2} \right) \mod q \quad (2.14)$$

$$= \psi^{-2i} \sum_{j=0}^{n/2-1} (\hat{a}_j + \hat{a}_{j+n/2}) \psi^{-4ij} \mod q \quad (2.15)$$

For the odd term, we get:

$$\mathbf{a}_{2i+1} = \psi^{-2i} \sum_{j=0}^{n/2-1} (\hat{a}_j - \hat{a}_{j+n/2}) \psi^{-4ij} \mod q \quad (2.16)$$

2.3. Power Side-Channel Analysis

As discussed in [10], cryptographic algorithms are typically executed on CMOS devices that consume electrical power. This power consumption can be categorized into two primary components: static and dynamic power consumption. Static power consumption refers to the power drawn by the device regardless of its operational state and is mainly influenced by the static leakage currents of all transistors within the chip. In contrast, dynamic power consumption is linked to the chip's switching activity, which pertains to the rate at which transistor capacitances are charged and discharged. The higher the switching activity, the greater the dynamic power consumption. In addition to the general power consumption, there is also a Gaussian noise distribution, which means that different executions of the same algorithm yield power traces that are not identical. This introduces a statistical correlation between the data being processed and the power consumption of the device. Potential adversaries aim to approximate the actual power trace as closely as possible, minimizing the noise component.

As an example, consider an attack on AES where an attacker aims to extract the encryption key under the following assumptions:

- The attacker possesses a known plaintext-ciphertext pair.
- The attacker can identify the sections of the power trace corresponding to the AES key expansion and characterize the device's power consumption.
- The attacker can extract the Hamming weights of the round-key bytes from the power trace using statistical analysis.

2. Background

The approach described in [11] leverages the dependencies between round-key bytes within the AES key expansion schedule to significantly reduce the number of possible key candidates. Once the key space is sufficiently narrowed, the remaining key candidates can be tested via brute-force using the known plaintext-ciphertext pair.

As more power traces are captured, the influence of the noise term diminishes in statistical analysis, enabling more precise conclusions. To mitigate this correlation and protect sensitive data, suitable masking techniques can be applied in either software or hardware.

2.3.1. Masking as a Countermeasure

The goal of masking is to ensure that the power trace is statistically independent of any security-sensitive intermediate values. This is achieved by randomizing these intermediate values. The process involves splitting the intermediate values into multiple shares and introducing randomness to each share, such that when the shares are recombined, the result equals the original intermediate value. The added randomness is referred to as a "mask" and must differ for each execution of the algorithm. All security-sensitive intermediate values, including those derived from initial sensitive values, must always be masked.

The number of shares required to secure an algorithm depends on the attack model. In this work, we assume an attacker capable of taking a single probe, an attacker is assumed to be able to read a single value for any given execution of the algorithm. An n -th order attacker, on the other hand, can take n probes and read n different values. To thwart an n -th order attack, $n + 1$ shares are necessary [12]. Since we assume a first-order attacker, two shares are needed to mask the secrets.

Increasing the number of shares enhances security but also raises the execution cost, whether in terms of software execution time or hardware size. In this work, two types of masking are employed: Boolean and Arithmetic masking. Both are necessary because Boolean masking is linear with respect to Boolean operations, while Arithmetic masking is linear with respect to Arithmetic operations. Being linear with respect to an operation means that the result of the operation remains unchanged compared to executing the operation without masking.

Boolean Masking

In Boolean masking, the secret is hidden using the XOR operation. The actual secret, denoted as s , is masked by a random value m , resulting in a masked value $s_m = s \oplus m$. This produces a sharing consisting of the two shares s_m and m . Boolean masking is applied to any operation that is linear with respect to the XOR operation and satisfies the property $f(x \oplus y) = f(x) \oplus f(y)$.

2. Background

Refreshing the shares with new masks may be necessary whenever the shares are mixed. In such cases, both shares can be XORed with the same random value. This ensures that the unmasked secret remains unchanged, as XORing twice with the same value will revert the shares to their original state.

Arithmetic Masking

In contrast to Boolean masking, Arithmetic masking conceals the secret using modular addition modulo q . The actual secret, denoted as s , is masked by a random value m . This results in two shares: $s_1 = (s+m) \bmod q$ and $s_2 = (s-m) \bmod q$. The sharing consists of these two shares, s_1 and s_2 . Arithmetic masking is applied to operations that are linear with respect to modular addition and satisfy the property $f(x+y) = f(x) + f(y)$.

If refreshing the sharing is necessary, we can add a new mask to the first share and subtract it from the second share. The unmasked intermediate value will remain unchanged, as adding and then subtracting the same value restores the original value.

2.3.2. Masked Gadgets

To mask an algorithm, it is decomposed into sub-functions that operate on either insensitive or sensitive data. The sub-functions dealing with insensitive data are implemented as usual, while the sub-functions handling sensitive data are implemented using masked gadgets. As stated in [1] a gadget G with m inputs and n outputs working with d shares implements a function $f : \mathbb{F}_q^m \rightarrow \mathbb{F}_q^n : (x_0, \dots, x_{m-1}) \rightarrow (y_0, \dots, y_{n-1})$.

Should multiple gadgets be combined, the resulting structure can be represented by a gadget composition. A gadget composition can be represented as a directed acyclic graph (DAG), where each vertex corresponds to an individual composing gadget. The gadget composition consists of multiple such composing gadgets, each represented by a vertex. Incoming edges to a gadget represent its inputs, while outgoing edges represent its outputs. For each input of the gadget composition, there is a vertex with no incoming edges and any number of outgoing edges. Conversely, for each output of the composition, there is a vertex with one incoming edge and no outgoing edges. Each edge in the graph consists of d wires, where d denotes the number of shares.

Probing model

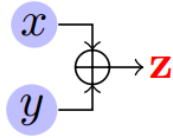
To analyze the security of gadgets and gadget compositions, [1] employs a t -probing model. In this model, an adversary can select a set of t probes, corresponding to a set P of t wires, each representing t values. A gadget is considered t -probing secure if the adversary cannot extract any information about sensitive variables by probing the selected set of t wires. A gadget is said to have a security order of t if it is t -probing

2. Background

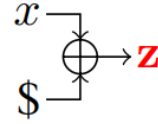
secure. In the case of d shares, the optimal scenario is when $t = d - 1$. Probes taken at the output of a gadget are referred to as output probes, while probes taken on any wire within the gadget are called internal probes. Notably, output probes are a subset of internal probes.

Simultability

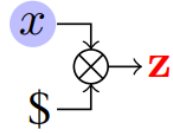
Another concept introduced by [1] is simultability. A set of probes is considered simultaneous if their statistical distribution can be simulated using a subset of the input shares. In 2.1, the inputs marked in blue are required to simulate the probes marked in red. The $\$$ signs represent random values used in the simulation. The concept of simultability can be used to describe the security guarantees provided by different gadgets, as well as how the knowledge gained from a probe propagates upstream or downstream within the DAG.



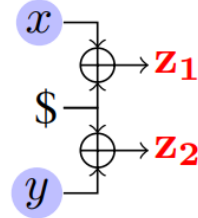
(a) If all inputs are deterministic, they are needed to simulate the output.



(b) When a fresh random is XORed, the output is independent of inputs.



(c) Multiplication with an input is not bijective, thus input is needed.



(d) With two probes, the random is not fresh.

Figure 2.1.: Simultability examples. All outputs (bold, red) are probed. Inputs needed for simulation are circled in blue [1].

Non-Interference (NI)

A gadget is considered to be t -NI if and only if for every set $t' \leq t$ internal probes can be simulated using at most t' shares. For a gadget with dt shares NI is strictly stronger than t -probing security.

2. Background

Strong Non-Interference (SNI)

A gadget with one output sharing is considered to be t -SNI if and only if for every set I of t_1 internal probes and every set O of t_2 output probes s.t. $t_1 + t_2 \geq t$, the set O can be simulated using at most t_1 shares of the input.

A composite gadget is t -NI if all it's composing gadgets are t -NI and all composing gadgets (and input vertices) are connected to an SNI refresh gadget. In 2.2, a) does not fulfill the SNI properties while b) does. For a) the edge for the input $x = (x_1, x_2)$ is connected to two edges without refresh gadgets.

For a gadget with dt shares NI is strictly stronger than t -NI security.

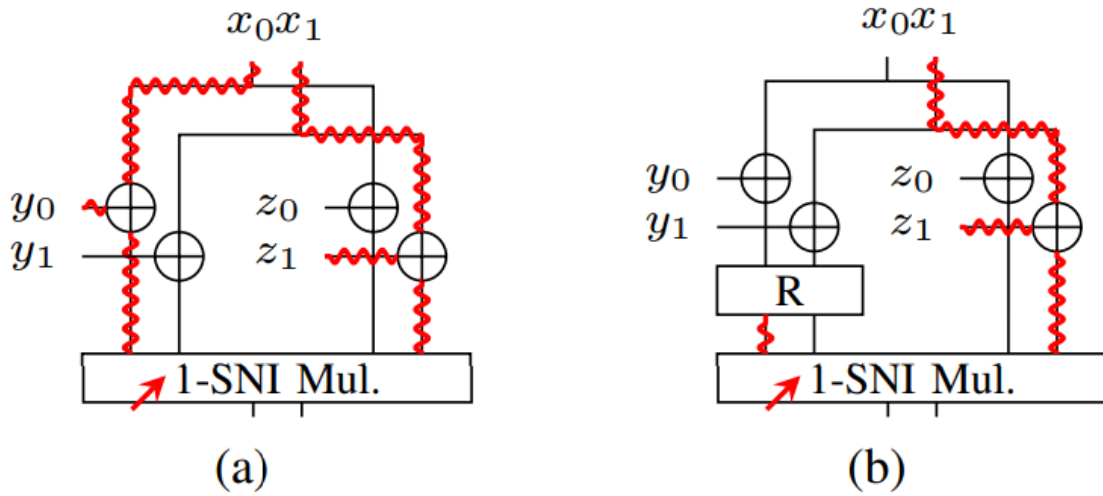


Figure 2.2.: Implementation of $(x + y)(x + z)$ masked with $d = 2$ shares, with input sharings (x_0, x_1) , (y_0, y_1) and (z_0, z_1) . The circuit is made of a 1-SNI multiplication with linear circuits at its inputs (two trivial implementations of the addition). The circuits illustrate (a) the limitation of SNI input composability and (b) a simple fix. The arrows indicate the adversarial probes (there is thus one internal probe in the multiplication) and the red snake wires are the propagated probes. The R box is a 1-SNI refresh gadget [1].

Probe Isolating Non-Interference (PINI)

A gadget is considered PINI-secure if it does not mix different shares of a single sharing. For PINI-secure gadgets, the position of the share indices, rather than the number of probes, is analyzed. Different sharings may be mixed, provided that shares with different indices are not combined. PINI-security is trivially satisfied by any linear function with respect to the masking. Furthermore, any composition of PINI-secure gadgets remains PINI-secure and does not require share refreshing. In such compositions, output probes

2. Background

propagate to all input shares with the same index. A t -PINI secure gadget, when using dt shares, is also t -probing secure.

2.3.3. Test Vector Leakage Assessment (TVLA)

Once we implement a masked algorithm, we need to test for potential leakage. In other words, we must determine whether the power traces recorded during execution can be correlated with secret intermediate data. The purpose of TVLA is to check whether two different inputs produce distinguishable side-channel information. If they do, it indicates that the device is leaking information.

In [13], TVLA is performed by recording a large number of power traces for two different input sets. For each sample in these recordings, a histogram is constructed. From these histograms, statistical properties such as moments, the t-statistic, and other relevant metrics can be computed. Equation 2.17 calculates the mean, where the summation runs from $i = 0$ to $i = 2^Q - 1$. The value i represents all possible quantized power measurement values, while $H_0[n][i]$ denotes the count of the i -th quantized value at sample n from the histogram of input value 0.

$$m_0[n] = \frac{1}{\sum_{i=0}^{2^Q-1} H_0[n][i]} \sum_{i=0}^{2^Q-1} i \cdot H_0[n][i]. \quad (2.17)$$

Similarly, the variance can be computed. Using the calculated means and variances, the t-statistic is constructed as shown in 2.18 and 2.19. Here, $\bar{X}_i$ represents the mean of the i -th sample, while $s_{\bar{X}_i}$ denotes its standard error. The term s_i corresponds to the corrected sample standard deviation, and N_i represents the sample size.

$$t = \frac{\Delta \bar{X}}{s_{\Delta \bar{X}}} = \frac{\bar{X}_1 - \bar{X}_2}{\sqrt{s_{\bar{X}_1}^2 + s_{\bar{X}_2}^2}} \quad (2.18)$$

$$s_{\bar{X}_i} = \frac{s_i}{\sqrt{N_i}} \quad (2.19)$$

Fixed Versus Random

In this work, a specific variant of TVLA is employed. The two sets used for histogram generation are referred to as the fixed and random sets. In the fixed set, the input remains constant for all traces, while in the random set, the input varies for each trace. From the recorded traces, we construct histograms for both sets and compute the relevant statistical properties as described earlier. This approach allows us to determine whether the mean power trace of a fixed value deviates significantly from the overall average

2. Background

power trace. If no significant difference is detected, the gadget is considered free from first-order leakage.

2.4. OpenTitan

OpenTitan is the first open-source project dedicated to developing a transparent, high-quality reference design and integration guidelines for silicon root of trust (RoT) chips. This work focuses on extending the OpenTitan repository by implementing secure ML-DSA gadgets.

2.4.1. OpenTitan Big Number Accelerator (OTBN)

OTBN is a specialized processor designed for executing security-sensitive asymmetric cryptographic algorithms, such as RSA and ECC. These algorithms rely heavily on wide integer arithmetic, which OTBN efficiently handles through its 256-bit data path, registers, and wide-word instructions.

To minimize data leakage, OTBN strictly separates control flow from data and keeps it minimal. Given the sensitive nature of the data it processes, the processor’s design prioritizes simplicity to reduce the attack surface and facilitate verification and testing. For instance, it excludes interrupts and exceptions, ensuring that all instructions execute within a single cycle.

OTBN functions as a standalone co-processor with dedicated instruction and data memory, accessible as a bus-connected device. All essential ML-DSA sub-functions are implemented using the custom OTBN ISA and are optimized for execution on the OTBN co-processor.

Chapter 3

Related Work

This work builds upon a broad body of existing research. This chapter provides the necessary context by reviewing related work in key areas. It first explores research on Post-Quantum Cryptography (PQC) implementations, followed by studies on the sensitivity analysis of PQC schemes. Finally, it examines related work on Boolean-to-Arithmetic mask conversion, highlighting advancements and methodologies relevant to this study.

3.1. Post Quantum Cryptography Implementations

Extensive research has been conducted on post-quantum cryptography (PQC) implementations in both hardware and software. Several studies have explored software implementations of CRYSTALS-Kyber and CRYSTALS-Dilithium for the Arm Cortex-M4 [14, 15, 16, 17, 18, 19, 20], while an optimized implementation on Arm Neon was presented in [21].

In the realm of the hardware acceleration, research has focused on optimizing PQC algorithms, including Dilithium, Kyber, and Falcon, for the OpenTitan Big Number (OTBN) core [22, 23]. Various accelerators targeting different performance-resource trade-offs have been proposed [24, 25, 26, 27, 28], including a processor specifically designed for module lattices [29]. Additionally, research on masked accelerators, which enhance security against side-channel attacks, has been conducted in [30].

3.2. Sensitivity Analysis of Post Quantum Cryptography Algorithms

In recent years, significant research has been conducted on the side-channel analysis of newly proposed Post-Quantum Cryptography (PQC) algorithms. Studies such as [31, 32, 33] demonstrate powerful side-channel attacks on various key encapsulation mechanisms (KEMs) that participated in the NIST PQC standardization process. Further research has identified vulnerabilities in specific components, including leakage in the Number Theoretic Transform (NTT) and Key Derivation Function (KDF) [34, 35, 36, 37].

Additionally, [38] highlights the significant performance overheads introduced by hardening PQC KEMs against side-channel attacks. Side-channel analysis has also been applied to Dilithium, as shown in [39]. Furthermore, research in [2, 40] presents sensitivity analyses of Dilithium, proposing countermeasures to enhance its resistance to side-channel attacks.

3.3. Boolean to Arithmetic Mask Conversion

This work includes a hardware design for Boolean to Arithmetic (B2A) mask conversion that is based on [41]. The first efficient first-order secure B2A implementation over k bits ($B2A_{2^k}$) was introduced in [42]. Further work introducing improvements are done in [43, 44, 45, 46].

Furthermore, work on mask conversions with an arbitrary modulus q was first initiated by [47]. A single bit Boolean to Arithmetic conversion modulo q ($B2A_q$) was introduced by [48]. Further improvements were introduced in [49, 50, 51, 52].

Among the first work introducing B2A mask conversions in hardware is [53], which presents results using a Ripple Carry Adder or a Kogge-Stone Adder. Similar implementations are introduced in [54, 55, 56, 57, 58].

Chapter 4

Hardening ML-DSA Sign on OTBN

This chapter analyzes the ML-DSA Sign algorithm and identifies a set of sensitive sub-functions that require hardening. It provides a detailed explanation of how these selected functions operate and delves deeper into the methodologies used to harden them. Additionally, this chapter justifies the security of the hardened sub-functions, ensuring their resilience against potential attacks.

4.1. ML-DSA Sign Sensitivity Analysis

This section first explains the ML-DSA signature algorithm sensitivity analysis presented in [2]. The objective is to identify which operations and variables are sensitive and require careful handling. Sensitive variables must be masked, and for sensitive operations, secure gadgets are needed to ensure computations on masked variables prevent information leakage.

This work focuses specifically on hardening the ML-DSA signing algorithm. The techniques and concepts applied to secure the signing process could also be adapted to enhance the security of the key generation. However, no additional protections are necessary for the verification algorithm, as it exclusively operates on publicly known values, such as the public key and the signature σ .

In Figure 4.1, variables and operations highlighted in red are classified as sensitive, while those in green are considered non-sensitive. The curved lines indicate rejection sampling operations that occur towards the end of the main loop in the ML-DSA signing algorithm, determining whether the loop continues or terminates. Additionally, the NTT and INTT operations on lines 2, 4, 13, 18, and 19 are also sensitive.

By definition, the public key is not sensitive since it is already expected to be known by adversaries. Similarly, the public matrix A , derived from ρ , does not require protection.

4. Hardening ML-DSA Sign on OTBN

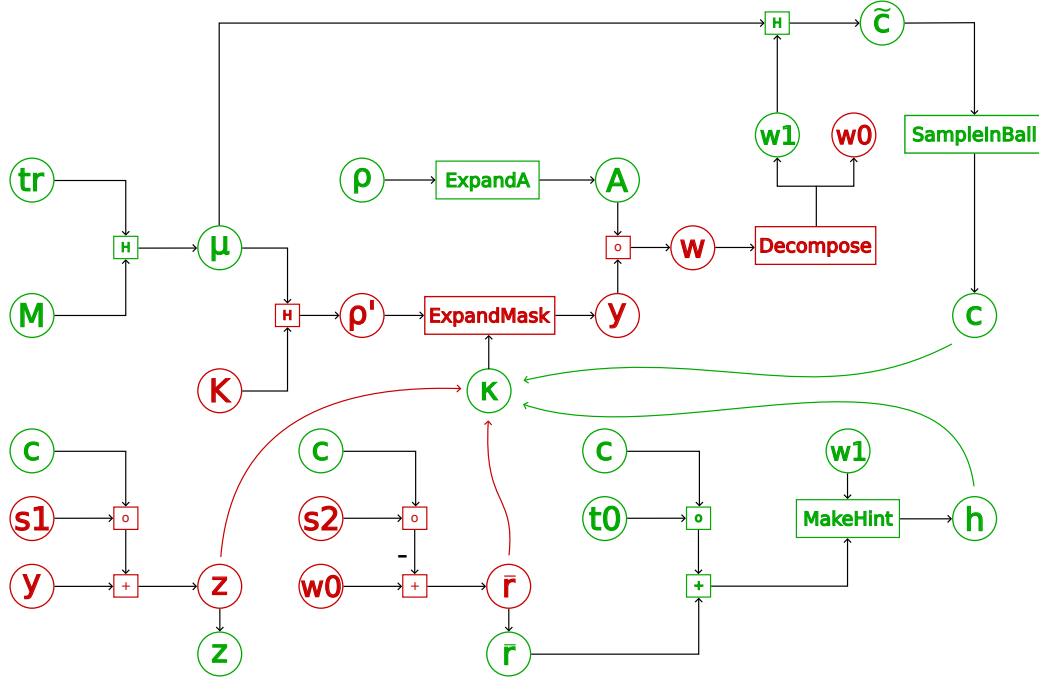


Figure 4.1.: A graphical representation of the signature generation process is shown. The input consists of the secret key (sk) and message (M), while the output is the signature $\sigma = (\tilde{c}, z, h)$. Curved arrows indicate rejection checks. Red elements represent sensitive operations and variables and blue elements represent non-sensitive operations and variables [2].

Some components of the secret key can be inferred from their public counterparts, making them non-sensitive as well. The value tr is also public, as it is simply the hash of the public key. Furthermore, the vector t_0 can be considered public, as Dilithium security proofs assume t (and thus t_0 and t_1) to be publicly known [59]. The message M does not require protection either.

However, the secret vectors s_1 and s_2 , along with the private random seed K , must be securely protected against side-channel leakage.

In Figure 4.1, the vector of polynomials y is classified as sensitive because, given a valid signature $\sigma = (\tilde{c}, z, h)$, the secret vector s_1 can be recovered from $z = y + cs_1$ if y is known or partially known [60]. Similarly, w_0 must be protected, as its exposure could enable the recovery of s_2 [61].

Moreover, w requires protection since w_0 is directly derived from it, and if both A and w are known, the equation $Ay = w$ can be solved to reveal y . Additionally, ρ' , which

4. Hardening ML-DSA Sign on OTBN

serves as an input for the *ExpandMask* function used to derive y , must be protected. The same holds for K and the hash function responsible for generating ρ' .

The sensitivity of the vector of polynomials w_1 is more nuanced, as it depends on the outcome of the rejection sampling checks on z and $\bar{r}$ (represented by the red curved lines in Figure 4.1). If the boundary checks during rejection sampling pass, then the zero-knowledge security proof of Dilithium guarantees that w_1 does not leak sensitive information. Furthermore, w_1 can be reconstructed from a valid signature. This means that the computations of z and $\bar{r}$ must remain secure until the boundary checks are successfully passed.

For the hint calculation, no additional security gadgets are required, as any information contained in the hint can be reconstructed directly from the signature itself.

The implementation of secure hash functions is beyond the scope of this work. OpenTitan already includes an HMAC module, which is a SHA-2-based authentication code generator that can be leveraged to accelerate hash computations in ML-DSA. Consequently, this work does not focus on hardening hash-based computations in 2, such as *ExpandA* on line 5, the operations on lines 6 and 7, *ExpandMask* on line 12, the computation on line 15, and *SampleInBall* on line 16.

Finally, this work focuses on hardening the following functions in OTBN assembly: Number Theoretic Transform (NTT), Inverse Number Theoretic Transform (INTT), vector addition, vector subtraction, vector coefficient-wise multiplication, vector coefficient-wise Multiply and Accumulate (MAC), decomposition, and bound checking. Hardening these operations is critical to securing ML-DSA against Power Side-Channel Analysis (PSA), as they operate on sensitive data.

4.2. Secure NTT

Similar to [22], this work employs a Cooley-Tukey (CT) butterfly construction to implement the NTT in software. As in the original reference implementation [62], the CT butterfly is used for the NTT. As shown in Listing 4.1, a CT butterfly consists of a Plantard multiplication between a coefficient and a twiddle factor, followed by a modular subtraction and a modular addition. The Plantard multiplication technique is further detailed in Section 3.1 of [22]. To optimize performance, the twiddle factors are stored in Plantard space, reducing the number of required multiplications.

As explained in Section 2.2.1, the NTT is performed in $\log_2 n$ rounds (layers), where n is the number of coefficients in the polynomial being transformed ($n = 256$ in our case). The work in [22] also introduces a layer merging technique to optimize memory access.

For a t -layer NTT, a t_0 -layer merge, where $t_0 \leq t$, allows loading and storing 2^{t_0} coefficients from/to memory only once after processing t_0 layers, instead of performing memory operations t_0 times in a traditional layer-by-layer approach. A t_0 – t_1 layer merge, where

4. Hardening ML-DSA Sign on OTBN

$t_0 + t_1 = t$, means that the first t_0 and last t_1 layers are merged separately, ensuring that each coefficient is loaded and stored only twice throughout one of the transformations.

In this work, we adopt a 4-4 layer merge strategy that allows for the maximum efficiency using all 32 wide data registers (WDRs) of OTBN.

```

1  /* Shift out coefficients from buffer */
2  bn.and coeffa, coeffsa, consts >> 192
3  bn.and coeffb, coeffsb, consts >> 192
4
5  /* Plantard multiplication: z * coeffb */
6  bn.mulqacc.w0.z coeffb, coeffb.0, twiddle.0, 192 /* (z*coeffb) % 2^2d */
7  bn.add coeffb, consts, coeffb >> 96 /* +1 */
8  bn.mulqacc.w0.z coeffb, coeffb.2, consts.1, 0 /* *q */
9  bn.rshi wtmp, consts, coeffb >> 32 /* >> d */
10
11 /* Butterfly */
12 bn.subm coeffb, coeffa, wtmp
13 bn.addm coeffa, coeffa, wtmp
14
15 /* Shift results back to buffer and shift out used coefficients */
16 bn.rshi coeffsa, coeffa, coeffsa >> 32
17 bn.rshi coeffsb, coeffb, coeffsb >> 32

```

Listing 4.1: CT butterfly on OTBN [22].

4.2.1. The Security of *secNTT*

The security of the implementation is ensured by the linearity of the Number Theoretic Transform (NTT) with respect to arithmetic masking. This property allows each share of a masked value to be processed independently, without the need to combine with other shares during computation. Consequently, performing the NTT on a masked value simply involves applying the NTT separately to each share.

As noted in [1], PINI-security is inherently satisfied by any trivially implemented linear function. Since the NTT satisfies the linearity condition:

$$NTT(a + b \bmod q) = NTT(a) + NTT(b) \bmod q \quad (4.1)$$

it naturally preserves the security guarantees of the arithmetic masking scheme, making it resistant to PSA.

4.3. Secure INTT

Similar to [22], this work employs a Gentleman-Sande (GS) butterfly construction to implement the INTT in software. As in the original reference implementation [62], the

4. Hardening ML-DSA Sign on OTBN

GS butterfly is used for the INTT. As shown in Listing 4.2, a GS butterfly consists of a Plantard multiplication between a coefficient and a twiddle factor, preceded by a modular subtraction and a modular addition. To optimize performance, the twiddle factors are stored in Plantard space, reducing the number of required multiplications, just as in the NTT.

Like the NTT, the INTT is computed in $\log_2 n$ rounds (layers), where n is the number of coefficients in the polynomial being transformed ($n = 256$ in this case). The same layer merging technique applied to the NTT is also used for the INTT to minimize memory access overhead and improve efficiency.

Finally, the coefficients must be multiplied by the inverse of n , n^{-1} , to complete the transformation. A potential optimization for future work is to integrate this scaling step into the last round of the INTT, reducing computational overhead.

```

1  /* Shift out coefficients from buffer */
2  bn.and coeffa, coeffsa, consts >> 192
3  bn.and coeffb, coeffsb, consts >> 192
4
5  /* Butterfly */
6  bn.subm coeffb, coeffa, coeffb
7  bn.addm coeffa, coeffa, coeffb
8
9  /* Plantard multiplication: z * coeffb */
10 bn.mulqacc.wo.z coeffb, coeffb.0, twiddle.0, 192 /* (z*coeffb) % 2^2d */
11 bn.add coeffb, consts, coeffb >> 96 /* +1 */
12 bn.mulqacc.wo.z coeffb, coeffb.2, consts.1, 0 /* *q */
13 bn.rshi wtmp, consts, coeffb >> 32 /* >> d */
14
15 /* Shift results back to buffer and shift out used coefficients */
16 bn.rshi coeffsa, coeffa, coeffsa >> 32
17 bn.rshi coeffsb, coeffb, coeffsb >> 32

```

Listing 4.2: GS butterfly on OTBN [22].

4.3.1. The Security of *secINTT*

Just like the NTT, the security of the implementation is granted by the linearity of the Inverse Number Theoretic Transform (INTT) with respect to arithmetic masking. This property ensures that each share of a masked value can be processed independently without requiring share mixing during computation. As a result, applying the INTT to a masked sharing is as simple as performing the INTT separately on each share.

Since the INTT follows the linearity condition:

$$INTT(a + b \bmod q) = INTT(a) + INTT(b) \bmod q \quad (4.2)$$

it inherently maintains the security guarantees of arithmetic masking, making it resistant to PSA.

4.4. Secure Vector Addition

Secure vector addition is performed by iterating over all n coefficients of two masked sharings, applying coefficient-wise addition, as shown in Algorithm 4. Here, c_i^j denotes the i -th coefficient of the j -th share in the sharing c .

The addition is strictly share-wise, meaning that the j -th share of one masked vector is only combined with the j -th share of the other masked vector. If one of the input vectors represents a non-sensitive variable, it is added only to the first share, while the second share of the sensitive variable remains unchanged. This ensures that no leakage occurs when handling masked data.

Algorithm 4: Secure vector addition modulo q .

input : Vectors $a^0, a^1, b^0, b^1 \in \mathbb{Z}_q^n$, where $a = a^0 + a^1 \pmod q$ and $b = b^0 + b^1 \pmod q$

output: Vectors $c = c^1 + c^2 \pmod q$, where $c = a + b \pmod q$

```

1  $i = 0$ 
2 while  $i < n$  do
3    $c_i^0 = a_i^0 + b_i^0 \pmod q$ 
4    $c_i^1 = a_i^1 + b_i^1 \pmod q$ 
5    $i = i + 1$ 
6 end
7 return  $(c^0, c^1)$ 

```

4.4.1. The Security of *secVecAdd*

Since the vector addition follows the linearity condition:

$$(a^0 + a^1) + (b^0 + b^1) \pmod q = (a^0 + b^0) + (a^1 + b^1) \pmod q \quad (4.3)$$

it is PINI and inherently maintains the security guarantees of arithmetic masking, making it resistant to PSA.

4.5. Secure Vector Subtraction

Similar to *SecVecAdd*, secure vector subtraction is performed by iterating over all n coefficients of two masked sharings and applying coefficient-wise subtraction, as shown in Algorithm 5.

As with addition, the subtraction is strictly share-wise, meaning that the j -th share of one masked vector is only combined with the j -th share of the other masked vector. If one of the input vectors represents a non-sensitive variable, it is subtracted only from the first share, while the second share of the sensitive variable remains unchanged. This approach ensures that no unintended leakage occurs when handling masked data.

Algorithm 5: Secure vector subtraction modulo q .

input : Vectors $a^0, a^1, b^0, b^1 \in \mathbb{Z}_q^n$, where $a = a^0 + a^1 \pmod{q}$ and $b = b^0 + b^1 \pmod{q}$

output: Vectors $c = c^0 + c^1 \pmod{q}$, where $c = a - b \pmod{q}$

```

1  $i = 0$ 
2 while  $i < n$  do
3    $c_i^0 = a_i^0 - b_i^0 \pmod{q}$ 
4    $c_i^1 = a_i^1 - b_i^1 \pmod{q}$ 
5    $i = i + 1$ 
6 end
7 return  $(c^0, c^1)$ 
```

4.5.1. The Security of *secVecSub*

Since the vector subtraction follows the linearity condition:

$$(a^0 + a^1) - (b^0 + b^1) \pmod{q} = (a^0 - b^0) + (a^1 - b^1) \pmod{q} \quad (4.4)$$

it is PINI and inherently maintains the security guarantees of arithmetic masking, making it resistant to PSA.

4.6. Secure Coefficient-Wise Vector Multiplication

Secure vector multiplication is performed by iterating over all n coefficients of a masked sharing and a non-sensitive vector, applying coefficient-wise multiplication, as shown in Algorithm 6. Each coefficient of the non-sensitive vector is multiplied individually with both shares of the masked vector. For the multiplication, we employ Barrett multiplication, as described in [63]. This method efficiently computes the product modulo q .

4. Hardening ML-DSA Sign on OTBN

Throughout this process, the individual shares remain separate and are never mixed. The multiplication is strictly coefficient-wise. This operation is particularly useful for matrix-vector multiplications where the matrix is non-sensitive or for coefficient-wise multiplications between two vectors, where only one of the vectors contains sensitive data.

Algorithm 6: Secure coefficient-wise vector multiplication modulo q .

input : Vectors $a, b^0, b^1 \in \mathbb{Z}_q^n$, where $b = b^0 + b^1 \pmod{q}$

output: Vectors $c = c^1 + c^2 \pmod{q}$, where $c = a * b \pmod{q}$

```

1  $i = 0$ 
2 while  $i < n$  do
3    $c_i^0 = a * b_i^0 \pmod{q}$ 
4    $c_i^1 = a * b_i^1 \pmod{q}$ 
5    $i = i + 1$ 
6 end
7 return  $(c^0, c^1)$ 

```

4.6.1. The Security of *secVecMul*

Since the vector multiplication follows the linearity condition:

$$a * (b^0 + b^1) \pmod{q} = (a * b^0) + (a * b^1) \pmod{q} \quad (4.5)$$

it is PINI and inherently maintains the security guarantees of arithmetic masking, making it resistant to PSA.

4.7. Secure Vector Coefficient-Wise Multiply and Accumulate

Secure vector multiply-and-accumulate (MAC) is performed by iterating over all n coefficients of a masked sharing, a non-sensitive vector, and a masked accumulation vector. The computation applies coefficient-wise multiplication, followed by a share-wise addition of the product to the accumulation vector, as shown in Algorithm 7. Each coefficient of the non-sensitive vector is multiplied separately with both shares of the masked vector. The resulting shares of the product vector are then added share-wise to the corresponding shares of the accumulation vector. For the multiplication, we employ Barrett multiplication, as described in [63]. This method efficiently computes the product modulo q .

4. Hardening ML-DSA Sign on OTBN

Throughout this process, the individual shares remain separate and are never mixed. Both the multiplication and vector addition operations are strictly coefficient-wise, ensuring secure computation. This operation is particularly useful for matrix-vector multiplications where the matrix is non-sensitive.

Algorithm 7: Secure coefficient-wise vector multiply and accumulate modulo q .

input : Vectors $a, b^0, b^1, c^0, c^1 \in \mathbb{Z}_q^n$, where $b = b^0 + b^1 \pmod{q}$ and $c = c^0 + c^1 \pmod{q}$

output: Vectors $d = d^1 + d^2 \pmod{q}$, where $d = (a * b) + c \pmod{q}$

```

1  $i = 0$ 
2 while  $i < n$  do
3    $d_i^0 = a * b_i^0 + c_i^0 \pmod{q}$ 
4    $d_i^1 = a * b_i^1 + c_i^1 \pmod{q}$ 
5    $i = i + 1$ 
6 end
7 return  $(d^0, d^1)$ 

```

4.7.1. The Security of *secVecMac*

Since the vector multiply and accumulate follows the linearity condition:

$$(a * (b^0 + b^1)) + (c^0 + c^1) \pmod{q} = (a * b^0 + c^0) + (a * b^1 + c^1) \pmod{q} \quad (4.6)$$

it is PINI and inherently maintains the security guarantees of arithmetic masking, making it resistant to PSA.

4.8. Secure And

The *SecAnd* gadget in this work is based on the PINI multiplication gadget in [1]. It is fundamental to the *SecAdd* gadget, the *SecAddm* gadget, the *SecA2B* and *SecB2A* gadgets, which are all used for the *SecBoundCheck* and *SecDecompose* gadgets. It allows for secure multiplication of bits of multiple sharings. The implementation used in this work can be seen in Algorithm 8. Essentially the algorithm computes a one bit sharing equal to $((a_0 \wedge b_0) \oplus (a_0 \wedge b_1) \oplus r, (a_1 \wedge b_1) \oplus (a_1 \wedge b_0) \oplus r)$, where r is a random bit, (a_0, a_1) is a one bit sharing and (b_0, b_1) is a one bit sharing.

The *SecAnd* gadget in this work is based on the PINI multiplication gadget from [1]. It plays a fundamental role in the *SecAdd*, *SecAddm*, *SecA2B*, and *SecB2A* gadgets, all of

4. Hardening ML-DSA Sign on OTBN

which are essential for the *SecBoundCheck* and *SecDecompose* gadgets. This gadget enables secure multiplication of bits across multiple sharings. The specific implementation used in this work is detailed in Algorithm 8.

At its core, the algorithm computes a one-bit sharing as follows:

$$(d_0, d_1) = ((a_0 \wedge b_0) \oplus (a_0 \wedge b_1) \oplus r, (a_1 \wedge b_1) \oplus (a_1 \wedge b_0) \oplus r) \quad (4.7)$$

where r is a random bit, and (a_0, a_1) and (b_0, b_1) are one-bit sharings.

Algorithm 8: SecAnd gadget.

input : Masked bits a_0, a_1, b_0, b_1 , where $a = a_0 \oplus a_1$ and $b = b_0 \oplus b_1$

output: Masked bit $d = d_0 \oplus d_1$

```

1  $r \leftarrow \$ \mathbb{F}_q$ 
2  $t_0 \leftarrow (a_0 \wedge r) \oplus r$ 
3  $t_1 \leftarrow (a_1 \wedge r) \oplus r$ 
4  $s_0 \leftarrow b_1 \oplus r$ 
5  $s_1 \leftarrow b_0 \oplus r$ 
6  $p_0 \leftarrow s_0 \wedge a_0$ 
7  $p_1 \leftarrow s_1 \wedge a_1$ 
8  $c_0 \leftarrow p_0 \oplus t_0$ 
9  $c_1 \leftarrow p_1 \oplus t_1$ 
10  $d_0 \leftarrow c_0 \oplus (a_0 \wedge b_0)$ 
11  $d_1 \leftarrow c_1 \oplus (a_1 \wedge b_1)$ 
12 return  $(d_0, d_1)$ 

```

4.8.1. The Security of *secAnd*

As shown in [1], the *SecAnd* gadget is 1-PINI secure. This security stems from the fact that the only intermediate values where shares are mixed occur in the AND operations $a_0 \wedge b_1$ and $a_1 \wedge b_0$. However, these terms are masked by the random bit r , ensuring that no intermediate value requires knowledge of both shares for simulation. As a result, the operation remains 1-PINI secure.

4.9. Secure Add

The secure adder used in the assembly implementation of this work is based on the bitsliced approach from [51]. It leverages a secure full adder, as shown in Algorithm 9. This secure full adder follows the structure of a standard full adder but replaces the

4. Hardening ML-DSA Sign on OTBN

conventional AND gate with the *SecAnd* operation from the previous section and the standard XOR gate with the $\oplus^B$ operation.

The $\oplus^B$ operation performs a share-wise XOR between two Boolean sharings by applying XOR to the i -th share of the first sharing and the i -th share of the second sharing. Since this operation is linear with respect to Boolean masking, it preserves the PINI security property.

In Algorithm 9, $x^{B/A,k}$ denotes a Boolean or Arithmetic sharing, where each share consists of k bits.

Algorithm 9: *SecFullAdder*^d

input : Boolean sharings $x^{B,1}$, $y^{B,1}$, and $z^{B,1}$

output: Boolean sharing $w^{B,2}$ such that $w = x + y + z$

```

1  $a^{B,1} \leftarrow x^{B,1} \oplus^B y^{B,1}$ 
2  $w^{B,2}[0] \leftarrow z^{B,1} \oplus^B a^{B,1}$ 
3  $w^{B,2}[1] \leftarrow x^{B,1} \oplus^B \text{SecAnd}_1^d(a^{B,1}, x^{B,1} \oplus^B z^{B,1})$ 
4 return  $w^{B,2}$ 

```

As shown in Algorithm 10, the implementation of *SecAdd* follows a Ripple Carry Adder structure, utilizing both *SecFullAdder* and the $\oplus^B$ operation. According to [51], the overall complexity of *SecAdd* is $\mathcal{O}(kd^2)$, where k (23 in this work) represents the number of bits per share, and d (2 in this work) denotes the masking order.

Algorithm 10: *SecAdd*_k^d

input : Boolean sharings $x^{B,k}$ and $y^{B,k}$, such that $x, y \in [0, 2^k[$

output: Boolean sharing $z^{B,k}$ such that $z = x + y \bmod 2^k$

```

1  $c^{B,1} \leftarrow (0, 0)$ 
2  $i = 0$ 
3 while  $i < k - 1$  do
4    $t^{B,2} \leftarrow \text{SecFullAdder}^d(x^{B,k}[i], y^{B,k}[i], c^{B,1})$ 
5    $(z^{B,k}[i], c^{B,1}) \leftarrow (t^{B,2}[0], t^{B,2}[1])$ 
6    $i = i + 1$ 
7 end
8  $z^{B,k}[k - 1] \leftarrow x^{B,k}[k - 1] \oplus^B y^{B,k}[k - 1] \oplus^B c^{B,1}$ 
9 return  $z^{B,k}$ 

```

4.9.1. The Security of *secAdd*

As mentioned in [51], the two gadgets discussed in this section, *SecAdd* and *SecFullAdder*, are composed of PINI gadgets. Consequently, they inherit the PINI property them-

selves.

4.10. Secure Add Modulo q

To implement *SecAddm* as described in [51], we first require the *BitCopyMask* gadget. The *BitCopyMask* algorithm, detailed in Algorithm 11, takes an integer p and a Boolean-masked bit $x^{B,1}$ as inputs. If the unmasked bit x equals 0, the output will be $y^{B,k} = (0, 0)$. Otherwise, if x equals 1, the output will be a Boolean-masked sharing of the integer p .

Algorithm 11: *BitCopyMask_k^d*

input : Boolean sharing $x^{B,1}$ and integer $p < 2^k$

output: Boolean sharing $y^{B,k}$

```

1  $i = 0$ 
2 while  $i < k$  do
3   if  $\left\lfloor \frac{p \bmod 2^i}{2^i} \right\rfloor = 1$  then
4      $y^{B,k}[i] \leftarrow x^{B,1}$  ▷ If the  $i$ -th bit of  $p$  is set, copy  $x^{B,1}$  to the  $i$ -th bit of  $y^{B,k}$ 
5   else
6      $y^{B,k}[i] \leftarrow (0, 0)$ 
7   end
8    $i = i + 1$ 
9 end
10 return  $y^{B,k}$ 

```

By leveraging *BitCopyMask* and *SecAdd*, we can now construct *SecAddm* as described in [51]. This results in Algorithm 12. The core idea of the algorithm is to first compute the sum of the input sharings $x^{B,k}$ and $y^{B,k}$ using *SecAdd*. Next, we add $2^{k+1} - p$ to this sum and inspect the most significant bit (MSB) to detect an overflow.

If the unmasked MSB is 1, the sum lies within the range $[p, 2^k[$, indicating that we need to subtract p to bring the value back into the range $[0, p[$. Otherwise, the sum is already within the correct range. In both scenarios, the final result $z^{B,k}$ is computed modulo p , achieving the desired outcome for *SecAddm*.

4.10.1. The Security of *secAddm*

Similar to *SecAdd*, all sub-gadgets used in this construction are PINI. Since any composition consisting solely of PINI gadgets remains PINI, *SecAddm* inherits this property.

Algorithm 12: $SecAddMod_{p,k}^d$

input : Boolean sharings $x^{B,k}$ and $y^{B,k}$, integer p such that $p < 2^k$ and $x, y \in [0, p[$

output: Boolean sharing $z^{B,k}$ such that $z = x + y \mod p$

```

1  $p^{B,k+1} \leftarrow (2^{k+1} - p, 0, \dots, 0)$ 
2  $s^{B,k+1} \leftarrow SecAdd_{k+1}^d(x^{B,k}, y^{B,k})$ 
3  $s'^{B,k+1} \leftarrow SecAdd_{k+1}^d(s^{B,k+1}, p^{B,k+1})$ 
4  $b^{B,1} \leftarrow s'^{B,k+1}[k]$ 
5  $a^{B,k} \leftarrow BitCopyMask_k^d(b^{B,1}, p)$ 
6  $z^{B,k} \leftarrow SecAdd_k^d(a^{B,k}, s^{B,k+1})$ 
7 return  $z^{B,k}$ 

```

Additionally, *BitCopyMask* simply replicates a sharing without introducing dependencies between shares. This ensures that it is share-isolating, further confirming its PINI security.

4.11. Secure Arithmetic to Boolean Mask Conversion

The $SecA2BMod_k^d$ assembly implementation in this work follows the approach described in [51]. The algorithm is outlined in Algorithm 13 and converts an arithmetic sharing x^{A_p} (modulo p) into a k -bit Boolean sharing $z^{B,k}$.

The core idea behind the algorithm is to first decompose the arithmetic input sharing into two Boolean sharings. The first, $s^{B,k+1}$, consists of the first share of x^{A_p} with an offset of $2^k - p$ in its first share, while the second share is set to zero. The second Boolean sharing, $s'^{B,k}$, has zero as its first share and the second share of x^{A_p} as its second share.

Next, the algorithm computes the $k + 1$ -bit sum of $s^{B,k+1}$ and $s'^{B,k}$, then unmask the most significant bit (MSB). The MSB is used to determine whether the unmasked sum falls within the range $[0, p[$ or $[p, 2^k[$. If the sum is in the latter range, p must be subtracted to bring it back into $[0, p[$. This final step ensures that the resulting Boolean sharing is correctly reduced modulo p .

4.11.1. The Security of $SecA2BMod_k^d$

Once again, since $SecA2BMod_k^d$ is composed entirely of PINI gadgets, its security follows directly from the PINI composition theorem. This ensures that the overall construction retains the PINI security property.

Algorithm 13: $SecA2BMod_k^d$

input : d shares arithmetic sharing x^{A_p} , integer p such that $p < 2^k$ and $x \in [0, p[$
output: d shares Boolean sharing $z^{B,k}$ such that $z = x$

```

1  $y^{B,k} \leftarrow x^{A_p}[0]$ 
2  $y'^{B,k} \leftarrow x^{A_p}[1]$ 
3  $p^{B,k+1} \leftarrow (2^k - p)$ 
4  $t^{B,k+1} \leftarrow SecAdd_{k+1}^1(p^{B,k+1}, y^{B,k})$ 
5  $s^{B,k+1} \leftarrow (t^{B,k+1}, 0)$ 
6  $s'^{B,k} \leftarrow (0, y'^{B,k})$ 
7  $u^{B,k+1} \leftarrow SecAdd_{k+1}^d(s^{B,k+1}, s'^{B,k})$ 
8  $b^{B,1} \leftarrow u^{B,k+1}[k]$ 
9  $a^{B,k} \leftarrow BitCopyMask_k^d(b^{B,1}, p)$ 
10  $z^{B,k} \leftarrow SecAdd_k^d(a^{B,k}, u^{B,k+1})$ 
11 return  $z^{B,k}$ 

```

4.12. Secure Boolean to Arithmetic Mask Conversion

The assembly implementation of $SecB2AMod_k^d$ in this work is based on [51]. The algorithm is detailed in Algorithm 16. It leverages the $SecRefresh_k^d$ gadget, described in Algorithm 14, and the $SecUnMask_k^d$ gadget, outlined in Algorithm 15.

Algorithm 14: $SecRefresh_k^d$

input : Boolean sharing $x^{B,k}$ with $0 \leq x < 2^k$
output: Refreshed Boolean sharing $z^{B,k}$ with $0 \leq x < 2^k$

```

1  $r \leftarrow \mathbb{Z}_{2^k}$ 
2  $z_0^{B,k} \leftarrow x_0^{B,k} \oplus r$ 
3  $z_1^{B,k} \leftarrow x_1^{B,k} \oplus r$ 
4 return  $z^{B,k}$ 

```

The core idea behind the $SecB2AMod_k^d$ gadget is to first generate a random mask, $z_0^{A_p}$. This mask is then subtracted from the input sharing $x^{B,k}$ giving us a new sharing $b^{B,k}$. Since the unmasked value of the $b^{B,k}$ is now independent of the original unmasked value x , we can safely unmask $b^{B,k}$ after applying a refresh operation. Adding the unmasked value b back to the random mask reconstructs the original unmasked value of x . As a result, we obtain a valid arithmetic output sharing by taking the mask as the first share and the unmasked value b as the second share.

4. Hardening ML-DSA Sign on OTBN

Algorithm 15: $SecUnMask_k^d$

input : Boolean sharing $x^{B,k}$ with $0 \leq x < 2^k$

output: Output the k -bit unmasked value x

1 $y^{B,k} \leftarrow SecRefresh_k^d(x^{B,k})$

2 $x \leftarrow \bigoplus_{i=0}^{d-1} x_i^{B,k}$

3 **return** x

Algorithm 16: $SecB2AMod_k^d$

input : d shares Boolean sharing $x^{B,k}$, integer p such that $p < 2^k$ and $x \in [0, p[$

output: d shares arithmetic sharing z^{A_p} such that $z = x$

1 $z_0^{A_p} \leftarrow \mathbb{Z}_p$

2 $z_0^{A_p} \leftarrow p - z_i^{A_p}$

3 $z_1^{A_p} \leftarrow 0$

4 $a^{B,k} \leftarrow SecA2BMod_k^d(z_1^{A_p})$

5 $b^{B,k} \leftarrow SecAddMod_k^d(a^{B,k}, x^{B,k})$

6 $c^{B,k} \leftarrow SecRefresh_k^d(b^{B,k})$

7 $z_{d-1}^{A_p} \leftarrow UnMask_k^d(c^{B,k})$

8 **return** $z_d^{A_p}$

4.12.1. The Security of SecB2AMod_k^d

A proof demonstrating that SecB2AMod_k^d is PINI can be found in [51]. The key idea behind the proof is that the refresh gadget ensures that the output shares cannot be traced back to more than one input share. This property guarantees that no intermediate value requires more than a single input share to be simulatable, thereby preserving PINI security.

4.13. Secure Bound Check

The secure gadget $\text{SecBoundCheck}_{q,\lambda_0,\lambda_1}^d$ is employed in ML-DSA for rejection sampling. The assembly implementation in this work builds upon the approach presented in [2].

At the core of the secure bound check gadget lies the SecLeq_ψ^d gadget, whose functionality is detailed in Algorithm 17. This gadget takes a Boolean sharing as input and outputs a bit b , where $b = 1$ if the unmasked value satisfies $x \leq \psi$, and $b = 0$ otherwise.

The underlying concept of the algorithm is straightforward. First, it adds $2^{k+1} - \psi - 1$ to the input sharing. Next, the overflow bit is unmasked and returned. If the overflow bit b equals 1, this indicates that the input sharing exceeded ψ . Otherwise, it confirms that the input was within the desired range.

Algorithm 17: SecLeq_ψ^d

input : Boolean sharing $x^{B,k}$ with $0 \leq x < 2^k$ and $\psi \geq 0$

output: For $0 \leq \psi < 2^k - 1$, public bit b with $b = 1$ if $x \leq \psi$ and $b = 0$ otherwise. If $\psi \geq 2^k - 1$, trivially returns $b = 1$

- 1 $x'^{B,k+1} \leftarrow \text{SecAdd}_{k+1}^d(x^{B,k}, 2^{k+1} - \psi - 1)$
 - 2 $b \leftarrow \text{SecUnMask}_1^d(x'^{B,k+1}[k])$
 - 3 **return** b
-

The secure gadget $\text{SecBoundCheck}_{q,\lambda_0,\lambda_1}^d$ operates similarly to SecLeq_ψ^d , with the key distinction being that its input sharing, $x^{A,q}$, is an Arithmetic sharing rather than a Boolean one.

The process begins by offsetting the input sharing by λ_0 . In the next step, the Arithmetic sharing is converted into a Boolean sharing. From this point, Algorithm 18 leverages the SecLeq_ψ^d gadget to determine whether the input sharing falls within the desired range.

4. Hardening ML-DSA Sign on OTBN

Algorithm 18: $SecBoundCheck_{q,\lambda_0,\lambda_1}^d$

input : Arithmetic sharing x^{A_q} , $q < 2^k$ and $\lambda_0 + \lambda_1 < q$ with $\lambda_0 \geq 0$ and $\lambda_1 \geq 0$

output: Bit b with $b = 1$ if $-\lambda_0 \leq x \leq \lambda_1 \pmod q$, $b = 0$ otherwise

```

1  $x_0'^{A_q} \leftarrow x_0^{A_q} + \lambda_0 \pmod q$ 
2  $x'^{B,k} \leftarrow SecA2BMod_q^d(x'^{A_q})$ 
3  $b \leftarrow SecLeq_{\lambda_0+\lambda_1}^d(x'^{B,k})$ 
4 return  $b$ 

```

4.13.1. The Security of $SecBoundCheck_{q,\lambda_0,\lambda_1}^d$

As discussed in [2], both the $SecBoundCheck_{q,\lambda_0,\lambda_1}^d$ and $SecLeq_\psi^d$ gadgets maintain PINI security. This follows directly from the PINI composition theorem, as each of these gadgets is constructed exclusively from PINI-compliant sub-gadgets, ensuring their overall PINI security.

4.14. Secure Decompose

The final gadget, $SecDecompose$, is designed to decompose an input sharing of $r \in \mathbb{Z}_q$ into two values, (r_1, r_0) , such that:

$$r = r_1 \cdot \alpha + r_0 \tag{4.8}$$

where $-\alpha/2 < r_0 \leq \alpha/2$ and $0 \leq r_1 \leq \delta$. In this work, we use $\delta = 44$, $\alpha = (q - 1)/\delta$, and q as the modulus in ML-DSA.

The assembly implementation is based on [64]. While many implementations rely on modulus switching, this approach is impractical for OTBN assembly because modulus switching requires a division by q , which OTBN lacks native support for. Instead, the method used here leverages mask conversion modulo q , avoiding the need for direct division.

Algorithm 19 is based on the property that $\alpha = (q - 1)/\delta$. Given Equation 4.11 and the constraints $-\alpha/2 < r_0 \leq \alpha/2$ and $0 \leq r_1 \leq \delta$, we derive the bound:

$$0 \leq -\delta \cdot r_0 + r_1 + (q - 1)/2 \leq q - 1. \tag{4.9}$$

From this, we obtain:

$$s \bmod {}^+\delta = r_1. \tag{4.10}$$

4. Hardening ML-DSA Sign on OTBN

$$s = -\delta \cdot r + (q-1)/2 = -\delta \cdot r_0 + r_1 + (q-1)/2 \pmod{q} \quad (4.11)$$

The masked implementation of this decomposition is outlined in Algorithm 19. For ML-DSA security levels 3 and 5, where $\delta = 16$ (a power of 2), the B2A conversion is unnecessary, as extracting the lowest 4 bits directly yields r_1 . However, in this work, we use $\delta = 44$, which is not a power of 2, necessitating the B2A conversion step.

Algorithm 19: *SecDecompose*

input : $x_1, x_2 \in \mathbb{Z}_q$, with $r = x_1 + x_2 \pmod{q}$

output: r_1 and $y_1, y_2 \in \mathbb{Z}_q$, such that $(r_1, r_0) = \text{Decompose}_q(r, \alpha)$, with
 $r_0 = y_1 + y_2 \pmod{q}$

```

1  $(s_1, s_2) \leftarrow (-\delta \cdot x_1 \pmod{q}, -\delta \cdot x_2 \pmod{q})$ 
2  $s'_1, s'_2 \leftarrow A2B_q(s_1, s_2)$ 
3  $s''_1, s''_2 \leftarrow B2A_\delta(s'_1, s'_2)$ 
4  $r_1 \leftarrow s''_1 + s''_2 \pmod{\delta}$ 
5  $(y_1, y_2) \leftarrow (x_1, x_2)$ 
6  $y_1 \leftarrow y_1 - \alpha \cdot r_1 \pmod{q}$ 
7 return  $r_1, (y_1, y_2)$ 
```

4.14.1. The Security of *SecDecompose*

As demonstrated in [64], the *SecDecompose* gadget satisfies PINI security. This follows from the fact that it is constructed entirely from PINI-secure sub-gadgets. By the PINI composition theorem, any composition of PINI-secure gadgets remains PINI-secure, ensuring that *SecDecompose* inherits this property.

Chapter 5

Analysis

In this chapter, we comprehensively analyze the software implementation of the ML-DSA sub-functions introduced in the previous chapter. First, we present a security analysis by executing the sub-functions on OTBN running on an FPGA board and examining the power traces for potential leakage. Next, we perform a performance evaluation, assessing the sub-functions in terms of execution time and memory utilization.

5.1. Security Analysis

The security analysis is conducted on the ChipWhisperer CW310 board, featuring a Kintex FPGA as the target. This advanced evaluation platform is specifically designed for FPGA-based security assessments, providing a robust environment for analyzing potential power side-channel leakage.

5.1.1. Importance of FPGA Side-Channel Evaluation

Even though cryptographic algorithms are theoretically secure, their practical implementation introduces numerous potential pitfalls that must be carefully addressed. When writing assembly code for OTBN, it is crucial to ensure that a share is not inadvertently overwritten by its counterpart. Using multiple shares within a single instruction can lead to leakage, as can accessing multiple shares in successive instructions. Additionally, failing to write constant-time code can expose secret values through timing side channels.

These concerns extend beyond software implementations to secure gadgets implemented in hardware. For instance, sensitive intermediate values sometimes need to be stored in

5. Analysis

registers to prevent glitch propagation—a phenomenon where bits flip unpredictably in combinational logic between active clock edges.

Due to these and many other factors, conducting thorough security analysis—such as testing on an FPGA board—is of utmost importance after implementation and functional verification. FPGA-based evaluation not only helps detect vulnerabilities but also allows for rapid iteration over different implementations, enabling incremental improvements. This flexibility is a significant advantage compared to analysis on production silicon, where modifications are costly and time-consuming.

5.1.2. The Measurement Setup

To capture and analyze power traces for the respective sub-functions, this work utilizes a dedicated penetration testing and measurement setup, illustrated in Figure 5.1. The setup integrates the ot-sca side-channel evaluation framework, which operates on a host computer, with the CW310 FPGA target board, which is connected to the host. The host is responsible for flashing the FPGA board with a binary that includes the OTBN core, the Ibex 32-bit RISC-V CPU core and the firmware containing the penetration testing framework.

Once running on the FPGA board, the penetration testing framework awaits commands from the host. Using the UART protocol, the host initiates the recording of power traces by sending commands based on a predefined configuration file. This file specifies which sub-function should be executed, how many times it should run, the number of clock cycles to record, and parameters such as the seed for the Pseudo Random Number Generator (PRNG) or the Linear-Feedback Shift Register (LFSR). After seeding the PRNG and LFSR accordingly, the penetration testing framework executes the sub-function as configured. The penetration testing framework runs on Ibex. Within this framework, the OTBN core is first properly configured for execution on Ibex before initiating the execution of the targeted sub-function on OTBN.

The Husky scope, seen in the top right of Figure 5.1, is connected to the power supply of the FPGA target board to record power traces. Each time a sub-function starts executing on OTBN, the FPGA sends a trigger signal to the Husky scope, ensuring trace alignment. The configuration file in the ot-sca framework also determines how the Husky scope is set up before running the sub-functions.

While the sub-functions are executed, the Husky scope continuously sends batches of recorded power traces to the ot-sca framework. Once the required number of traces have been collected, the ot-sca framework can proceed with power leakage analysis.

For this work, we conduct a Test Vector Leakage Assessment (TVLA) using fixed versus random testing, as described in the preliminaries chapter.

5. Analysis

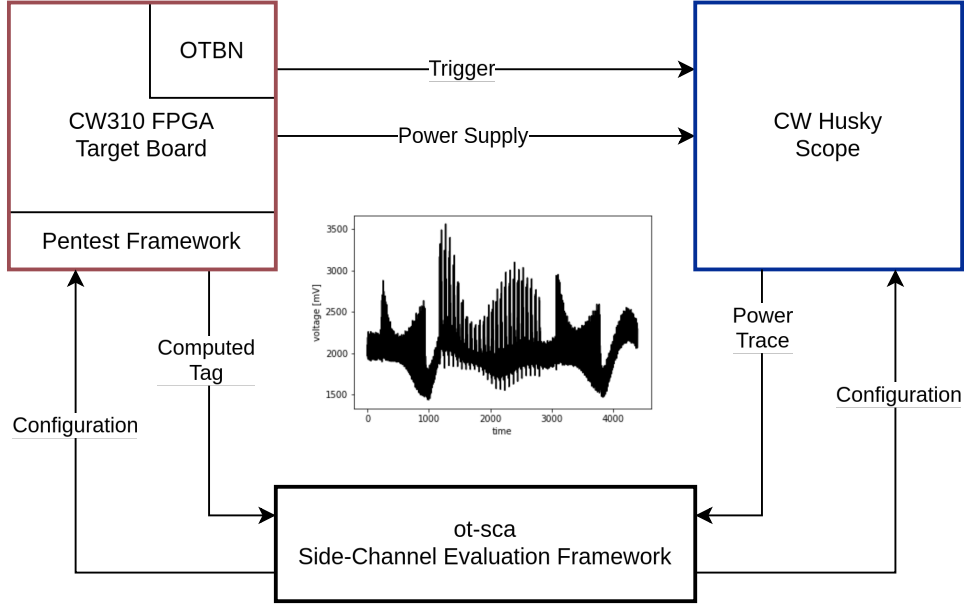


Figure 5.1.: The measurement setup including the target FPGA board, the CW Husky scope and the ot-sca side-channel evaluation framework.

5.1.3. Challenges While Hardening the Sub-Functions

While iterating through different versions of the hardened sub-function code, several issues were encountered that led to unintended leakage. Initially, masking of the sharings was performed on OTBN, which resulted in leakage in multiple sub-functions. To mitigate this, the masking process was moved to the Ibex core, which runs the penetration testing framework. Since, in a practical implementation, inputs are already stored in multiple shares, the specific location where masking is applied does not matter in this case.

Additionally, to prevent interference between consecutive runs, the OTBN core had to be wiped between executions of the sub-functions. However, since the masking was now done on Ibex, leakage from the Ibex core itself introduced vulnerabilities during OTBN operations. To mitigate this, Ibex was placed in sleep mode during OTBN execution. It was crucial to ensure that Ibex entered sleep mode well before OTBN started executing the sub-functions and remained in sleep mode for some time afterward. As a result, 5'000 stall cycles had to be added at the beginning of each OTBN assembly sub-function.

Another significant challenge arose with the decompose and bound check sub-functions,

5. Analysis

which require fresh randomness from OTBN to refresh the sharings. Initially, true randomness was used, but this posed a problem—it could take upwards of 10'000 cycles to generate, leading to execution times exceeding hundreds of thousands of cycles. The Husky Scope, however, can only store 11'802 cycles worth of trace data at once, meaning that capturing a sufficient number of traces for analysis would have taken months per sub-function. This issue was resolved by switching to pseudo-randomness, which is generated much faster, ensuring that traces fit within the Husky Scope's memory in a single capture.

Furthermore, an issue arose with the decompose function, which splits an input sharing of r into a non-sensitive variable r_1 and a sensitive sharing of r_0 . Since only r_0 is sensitive, r_1 needs to be randomized to prevent false leakage from appearing in the analysis.

5.1.4. Power Side-Channel Leakage Analysis

In this section, the power traces for each sub-function are presented and analyzed. The unmasked implementation is compared to the hardened implementation in terms of power side-channel leakage. While the unmasked sub-functions in this work typically start leaking within 10-100 traces, this work records 150'000 traces for the masked sub-functions to ensure a thorough leakage analysis.

Each figure in this section contains three plots:

1. Raw power traces – Visualizing the recorded power consumption.
2. First-order t-test values – Checking for first-order leakage.
3. Second-order t-test values – Checking for second-order leakage.

For this work, only first-order leakage is of interest, as the attack model assumes a first-order attack, and the countermeasures employ first-order masking, which is specifically designed to mitigate first-order attacks.

In this work, a sub-function is considered leakage-free if the t-test values remain below an absolute value of 4.5, indicated by a red line. This threshold suggests a 99.999% confidence level that there is no statistically significant difference between the mean of the fixed set power traces and the mean of the random set power traces, indicating the absence of detectable leakage.

NTT Results

Figure 5.2 presents the power trace of the unmasked NTT along with the corresponding t-test values for each sample. After around 5'000 samples, OTBN begins processing the unmasked coefficients of the vector. From this point onward, significant leakage is

5. Analysis

observed throughout the remainder of the power trace. Notably, even with only 100 recorded traces, the unmasked NTT already exhibits detectable leakage.

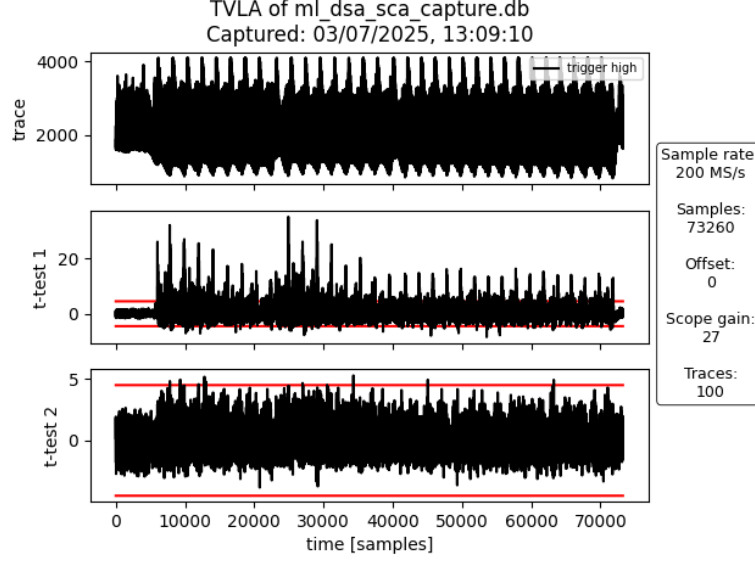


Figure 5.2.: The unmasked TVLA results for the NTT sub-function.

Figure 5.3 displays the power trace of the NTT applied to a single share vector, along with the corresponding t-test values. Due to the linearity of the NTT with respect to Arithmetic masking, the computation on the second share would appear identical. The core principle of Arithmetic masking is that the second share remains uniformly distributed and uncorrelated with the first share. Unlike the unmasked NTT, the masked NTT exhibits no detectable leakage, even after recording 150'000 traces.

5. Analysis

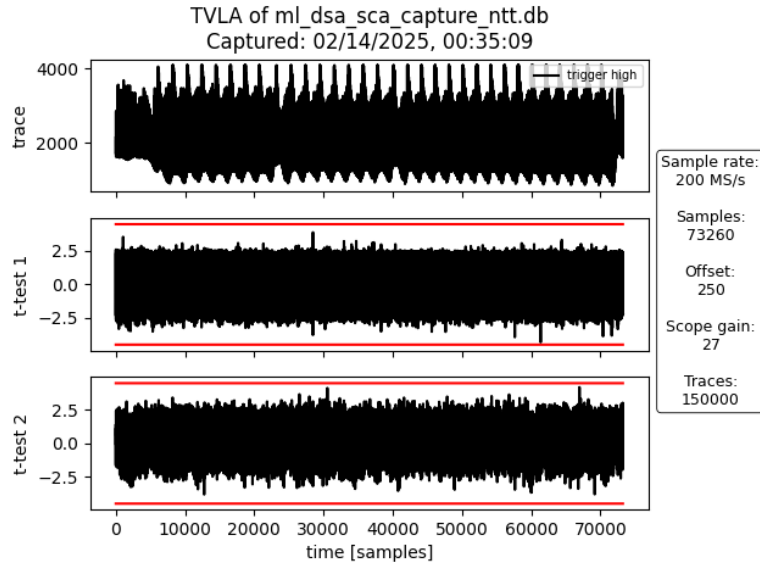


Figure 5.3.: The masked TVLA results for the NTT sub-function.

INTT Results

Figure 5.2 illustrates the power trace of the unmasked INTT alongside its corresponding t-test values for each sample. Similar to the NTT, after approximately 5'000 samples, OTBN begins processing the unmasked coefficients of the vector, leading to significant leakage throughout the remainder of the power trace. The unmasked INTT already exhibits leakage after 100 traces.

5. Analysis

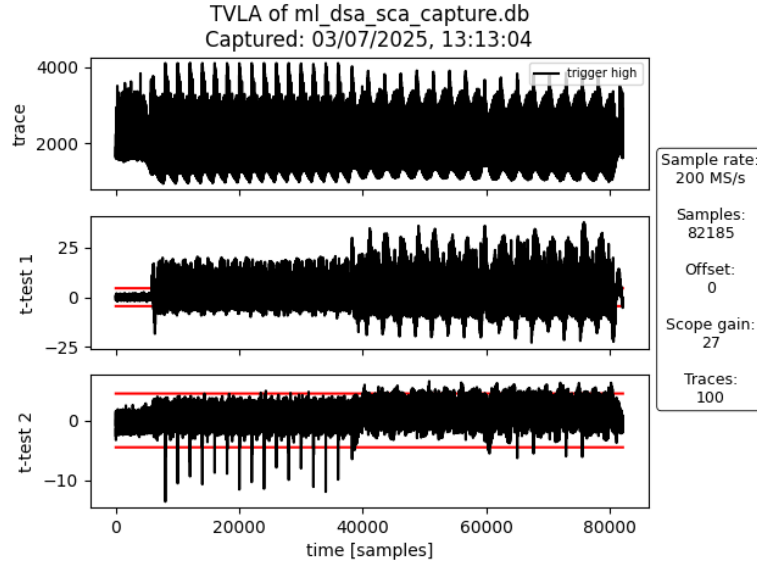


Figure 5.4.: The unmasked TVLA results for the INTT sub-function.

Figure 5.5 illustrates the power trace of the INTT applied to a single share vector, accompanied by its corresponding t-test values. Given the linearity of the INTT under Arithmetic masking, the computation on the second share follows the same structure. Unlike the unmasked INTT, the masked INTT demonstrates no observable leakage, even after collecting 150'000 traces.

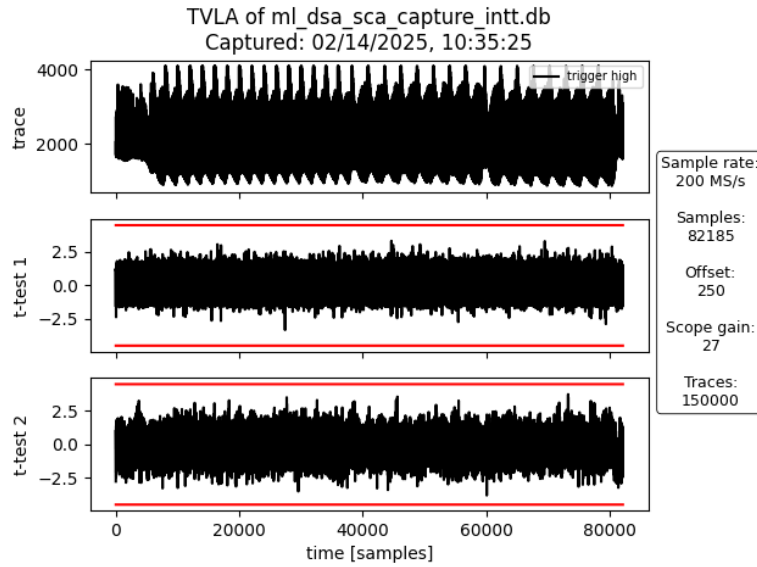


Figure 5.5.: The masked TVLA results for the INTT sub-function.

5. Analysis

Vector Add Results

Figure 5.6 presents the power trace of the unmasked vector addition operation alongside its corresponding t-test values. Leakage can already be seen after just 100 recorded traces, starting from approximately sample 5'000 and persisting throughout the remainder of the trace.

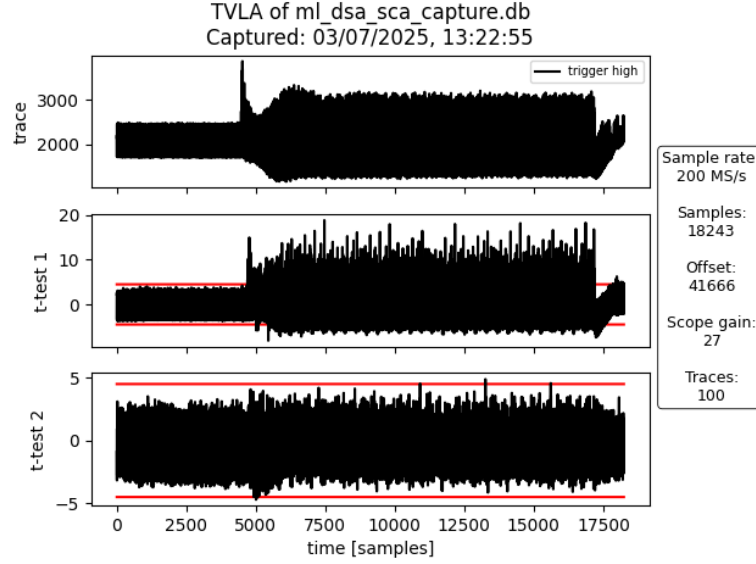


Figure 5.6.: The unmasked TVLA results for the Vector Add sub-function.

In contrast, the masked vector addition exhibits no detectable leakage, even after recording 150'000 traces, as shown in Figure 5.7. Similar to the NTT, this trace represents the vector addition of the first share. Since vector addition, similar to the NTT and INTT, maintains linearity under Arithmetic masking, the analysis of the second share vector appears identical to that of the first share.

5. Analysis

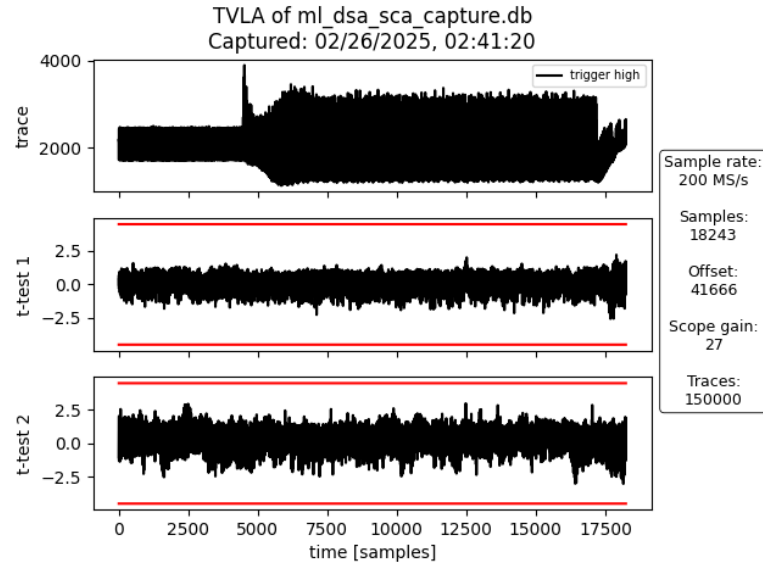


Figure 5.7.: The masked TVLA results for the Vector Add sub-function.

Vector Subtract Results

The analysis results for the unmasked vector subtraction are presented in Figure 5.8. Even with just 100 recorded traces, significant leakage is already observable.

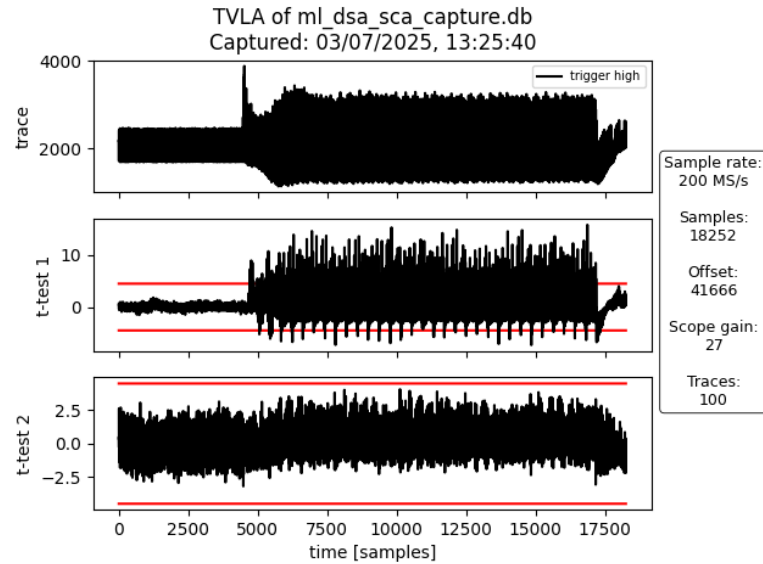


Figure 5.8.: The unmasked TVLA results for the Vector Subtract sub-function.

5. Analysis

Similar to the masked vector addition, the masked vector subtraction exhibits no detectable leakage as can be seen in Figure 5.9, even after 150'000 recorded traces. The traces correspond to the operation on the first share vector, and as with addition, the same linearity principle applies.

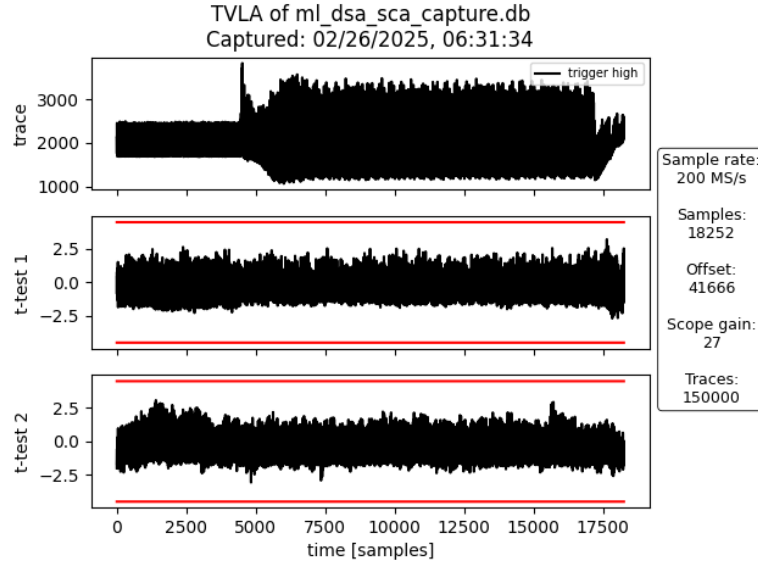


Figure 5.9.: The masked TVLA results for the Vector Subtract sub-function.

Vector Multiply Results

The power trace for the unmasked vector multiplication, shown in Figure 5.10, is approximately twice as long as those for vector addition and subtraction due to the use of Barrett multiplication. However, the results remain similar, with detectable leakage appearing after just 100 recorded traces.

5. Analysis

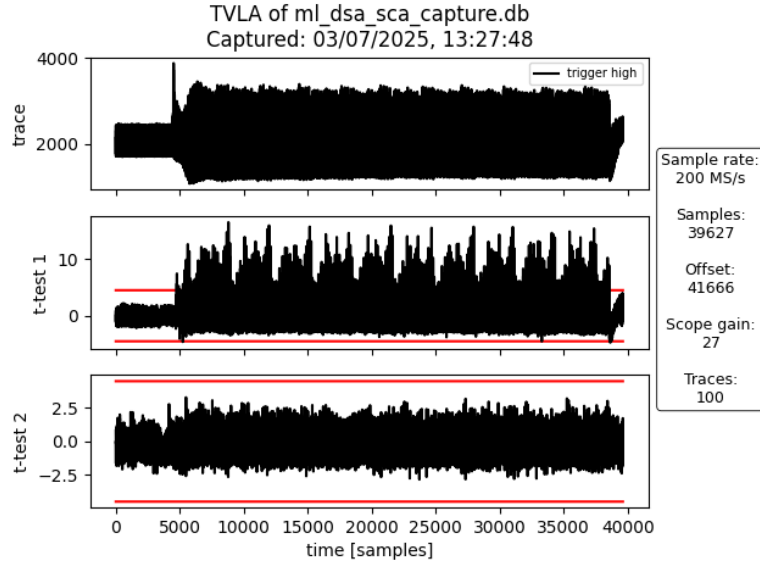


Figure 5.10.: The unmasked TVLA results for the Vector Multiply sub-function.

Figure 5.11 presents the results for the masked vector multiplication, following the same pattern observed for masked vector addition and subtraction. Even after analyzing 150'000 traces, no leakage is detected. As with the other vector operations, the plots display computations on the first share vector, and the linearity principle remains applicable.

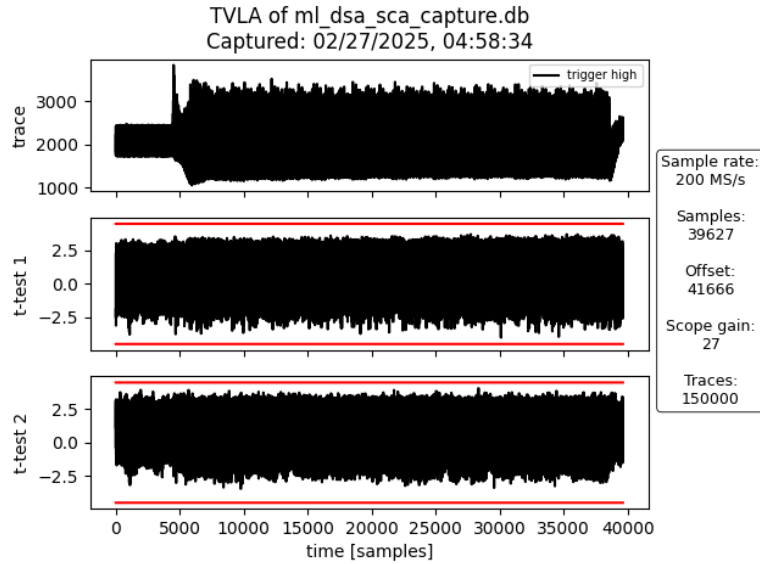


Figure 5.11.: The masked TVLA results for the Vector Multiply sub-function.

Vector Multiply and Accumulate Results

Figure 5.12 presents the power trace and t-test plots for the unmasked vector multiply-accumulate (MAC) sub-function. As anticipated, leakage is evident with just 100 recorded traces. This aligns with previous observations, as both the unmasked vector addition and multiplication exhibit leakage within the same trace count, and the MAC operation inherently combines these two functions.

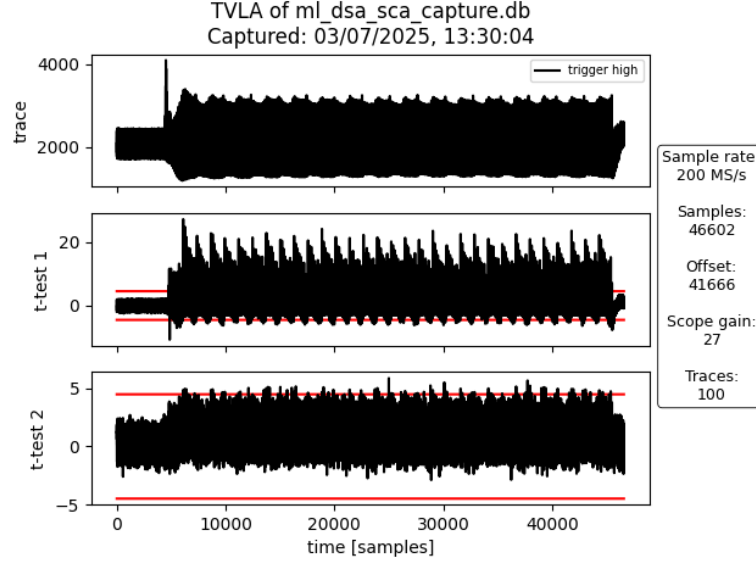


Figure 5.12.: The unmasked TVLA results for the Vector Multiply and Accumulate sub-function.

Figure 5.13 displays the power trace and t-test plots for the masked vector MAC. Similar to the other vector operations, even after analyzing 150'000 traces, no leakage is detected. The plot specifically illustrates the operation on the first share vector, while the second share vector is expected to exhibit identical behavior due to the linearity property of Arithmetic masking.

5. Analysis

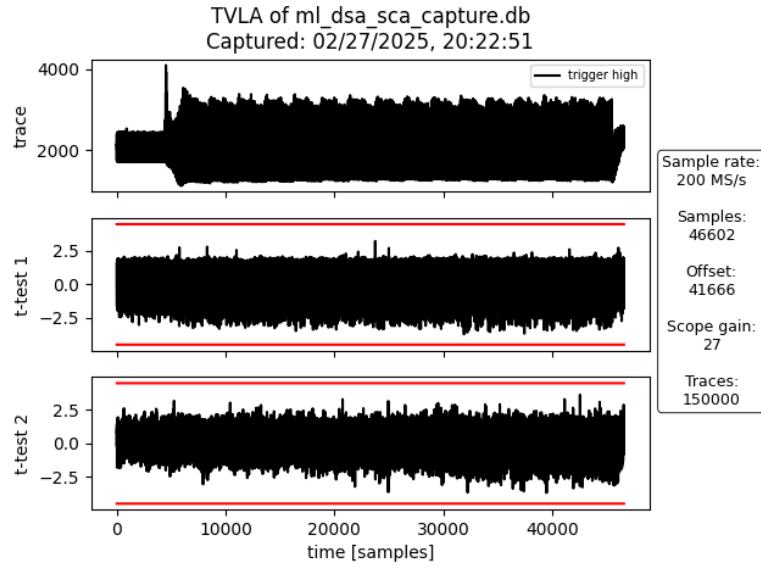


Figure 5.13.: The masked TVLA results for the Vector Multiply and Accumulate sub-function.

Bound Check Results

Figure 5.14 presents the power trace and t-test plots for the unmasked bound check sub-function, where leakage is evident with just 1’500 recorded traces. The initial segment of the trace exhibits significant leakage, corresponding to the Arithmetic-to-Boolean mask conversion performed at the beginning of the bound check algorithm. This step is where the arithmetic sharing is converted into a Boolean sharing.

5. Analysis

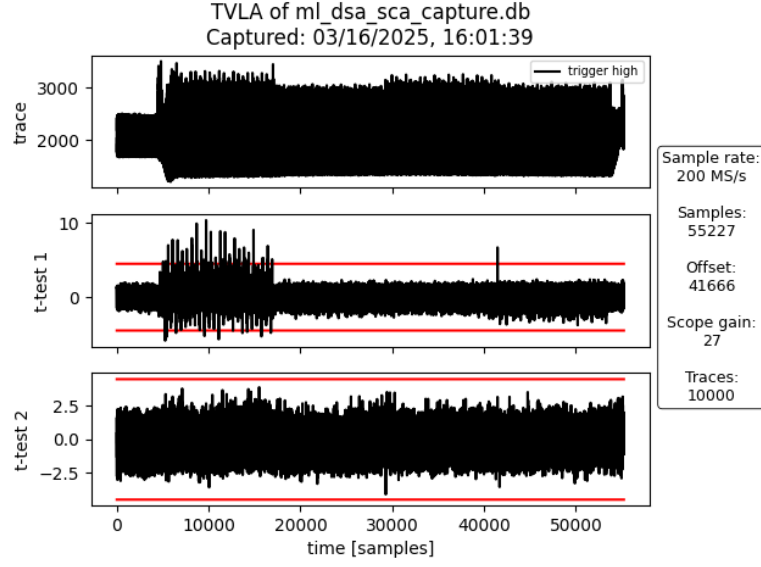


Figure 5.14.: The unmasked TVLA results for the Bound Check sub-function.

Figure 5.15 presents the power trace and t-test plots for the masked bound check sub-function. After analyzing 150'000 traces, only minor leakage is detected. The arithmetic-to-Boolean mask conversion is now leakage-free. However, towards the end of the trace, some leakage appears in the section where the secure Boolean addition is performed.

Initially, the execution time of the sub-function was too long to be analyzed within the scope of this thesis. To address this, the sub-function was optimized to fit within a single trace by replacing true randomness with pseudo-randomness. However, this modification was introduced too late to fully harden the sub-function. The remaining leakage is attributed to transient effects between instructions, which require further investigation.

5. Analysis

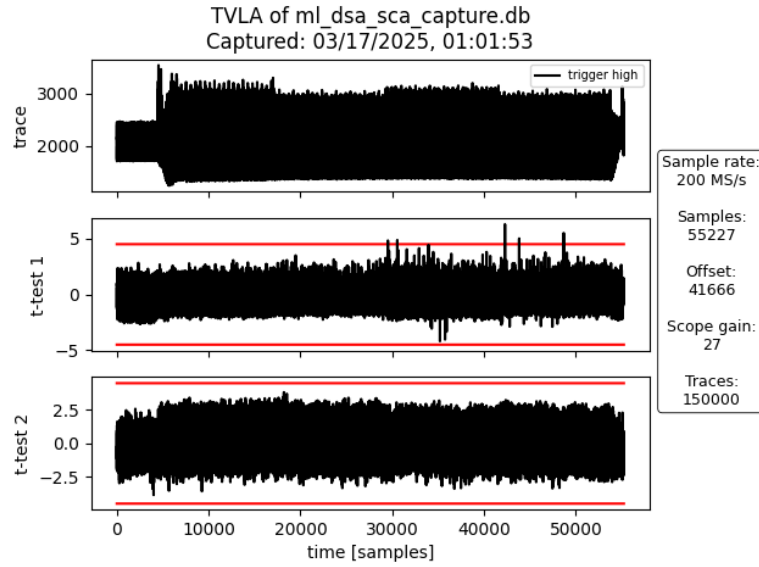


Figure 5.15.: The masked TVLA results for the Bound Check sub-function.

Decompose Results

Due to memory limitations in power trace recording, the traces for the decompose function had to be captured in two separate parts. Figures 5.16 and 5.17 present the power traces and t-test plots for the unmasked decompose sub-function, where leakage is evident with just 1'500 recorded traces. Leakage is observed consistently throughout both traces.

5. Analysis

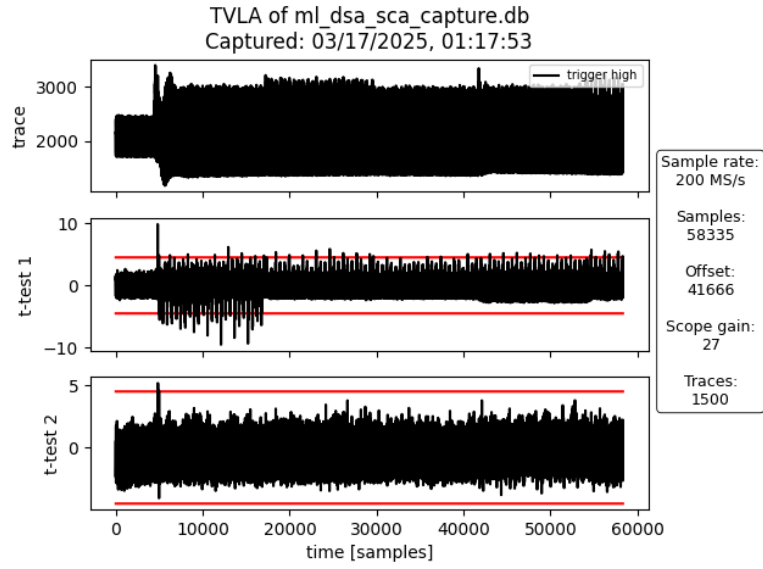


Figure 5.16.: The first part of the unmasked TVLA results for the Decompose sub-function.

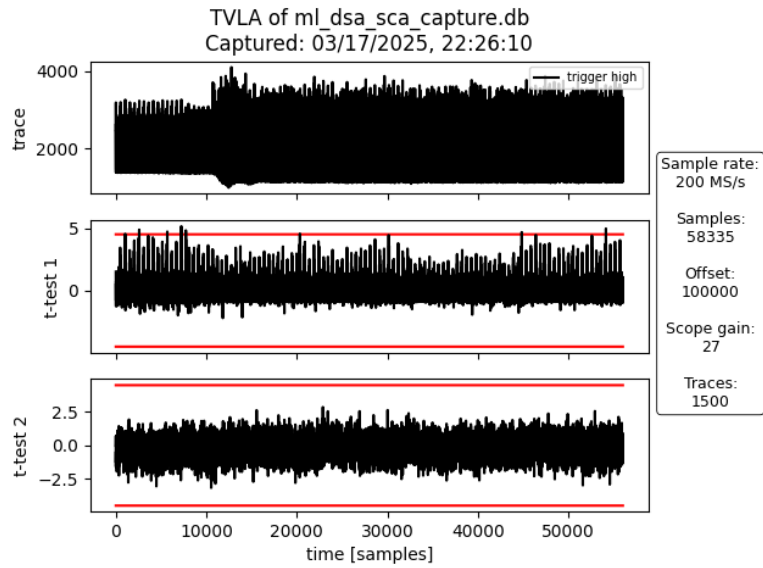


Figure 5.17.: The second part of the unmasked TVLA results for the Decompose sub-function.

Figures 5.18 and 5.19 present the power traces and t-test plots for the masked decompose sub-function. After analyzing 150'000 traces, only minor leakage is detected, primarily

5. Analysis

in the secure Boolean addition and the secure Boolean addition modulo q .

Similar to the bound check sub-function, the execution time of the decompose sub-function was initially too long to be analyzed within the scope of this thesis. To work around this, the sub-function was optimized to fit within a single trace by replacing true randomness with pseudo-randomness. However, this modification was introduced too late to fully harden the sub-function. The remaining leakage is attributed to transient effects between instructions, which require further analysis.

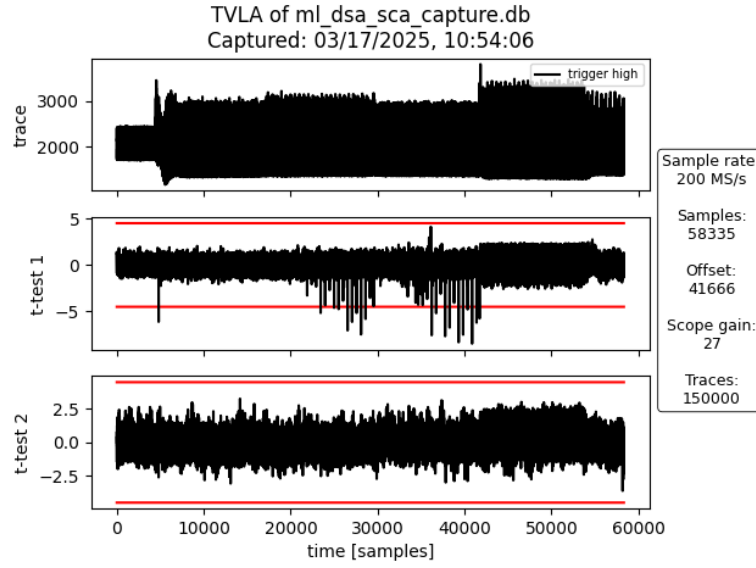


Figure 5.18.: The first part of the masked TVLA results for the Decompose sub-function.

5. Analysis

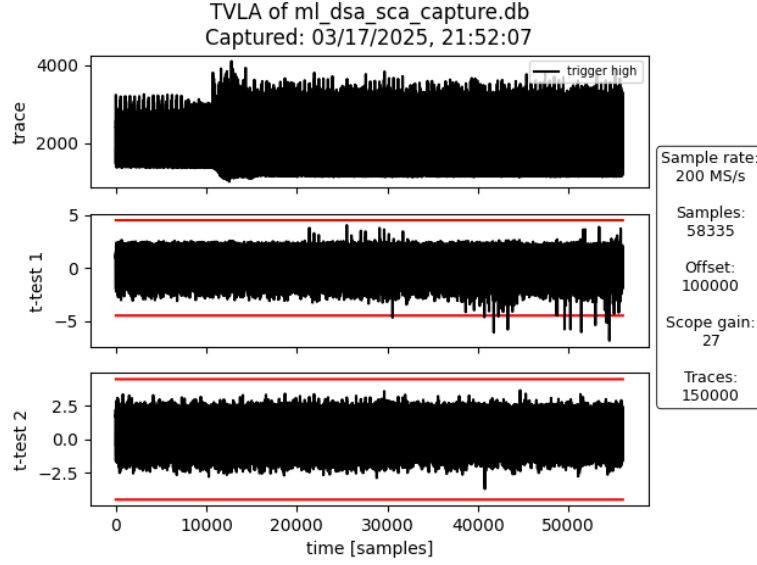


Figure 5.19.: The second part of the masked TVLA results for the Decompose sub-function.

5.2. Performance Analysis

This section presents a performance analysis of the implemented ML-DSA sub-functions. It begins by examining the instruction and data memory utilization of the sub-functions, comparing the unmasked and masked implementations to assess the overhead introduced by masking. Following this, the total memory utilization of the ML-DSA signing algorithm is estimated by combining the memory analysis from this work with existing data on unmasked ML-DSA implementations. This estimation accounts for the additional overhead caused by masking sensitive sub-functions and intermediate values.

Additionally, this section provides a timing analysis, evaluating the execution cycles of the masked sub-functions in comparison to their unmasked counterparts. Finally, an estimate of the total cycle count for the entire ML-DSA signing algorithm is derived using the same methodology as the memory utilization analysis.

5.2.1. Memory Utilization Analysis

This section analyzes memory utilization in terms of both data memory and instruction memory, with all values expressed in bytes. According to [22], the ML-DSA signing algorithm requires 56'028 bytes of data memory and 31'024 bytes of instruction memory.

5. Analysis

Table 5.1 provides a breakdown of data memory usage by individual variables. The first column lists the variables, the second column presents the memory required in the unprotected implementation, and the last column shows the corresponding values for the hardened version. Sensitive variables require twice the memory due to masking, while non-sensitive variables maintain the same memory footprint. Variables not explicitly listed in Table 5.1 reuse allocated memory for other variables throughout the algorithm, replacing existing variables without requiring additional memory allocation.

The last row of Table 5.1 presents the total data memory usage. In the unprotected implementation, the total data memory consumption is 50,432 bytes, whereas in the hardened implementation, it increases to 71,012 bytes. This represents a 40.8% increase in data memory usage due to the overhead introduced by masking.

Table 5.1.: Data memory usage broken down into the individual variables.

Function	Reference [22] [bytes]	This Work [bytes]
Signature σ	4	4
Message M	4	4
Message length M_{len}	4	4
Secret key sk	4	8
Public random seed ρ	48	48
Private random seed K	32	64
Public key hash tr	32	32
Public key polynomial vector t_0	4'096	4'096
Secret key polynomial vector s_0	4'096	8'192
Secret key polynomial vector s_1	4'096	8'192
Message hash μ	64	64
Private random seed ρ'	64	128
Public key polynomial matrix A	16'384	16'384
Polynomial vector y	4'096	8'192
Response polynomial vector z	4'096	8'192
Commitment polynomial vector w_0	4'096	8'192
Commitment polynomial vector w_1	4'096	4'096
Commitment polynomial vector c	1'024	1'024
Hint polynomial vector h	4'096	4'096
Total data memory used	50'432	71'012

Table 5.2 details the instruction memory usage, categorizing it by the sub-functions that were hardened in this work. The first column lists the function names, while the second and third columns show the instruction memory size for the unmasked and masked implementations, respectively.

For sub-functions that are linear with respect to the masking method, the instruction memory size remains unchanged, as the same sub-routine is simply executed twice—once

5. Analysis

per share. However, for more complex sub-functions requiring operations like mask conversions, entirely new algorithms replace the unmasked versions, leading to increased instruction memory usage.

The two functions that are entirely reimplemented in the hardened version are the *decompose* and *bound check* functions. Since both functions rely on the same sub-routines, these sub-routines are listed separately in the table. Notably, these sub-routines are exclusive to the hardened implementation and are not present in the unmasked case, which is why no instruction memory usage is reported for the unmasked version.

The last row of the table compares the total instruction memory used by the ML-DSA sign algorithm in both cases. The instruction memory footprint increases from 6,412 bytes to 6,943 bytes, marking an 8.28% increase due to the added masking operations.

Table 5.2.: Instruction memory usage broken down into the individual secure sub-functions and their unmasked counterparts.

Function	Unmasked [bytes]	Masked [bytes]
Vector NTT	2'576	2'576
Vector INTT	2'892	2'892
Vector add	128	128
Vector subtract	128	128
Vector coefficient-wise multiplication	188	188
Vector coefficient-wise multiply and accumulate	212	212
Decompose (single coefficient)	232	116
Bound check (single coefficient)	56	92
Secure decompose	-	116
Secure bound check	-	7
Secure leq	-	68
B2A	-	76
A2B	-	80
Add modulo	-	64
Add	-	200
Total instruction memory used	6'412	6'943

5.2.2. Cycle Analysis

This section analyzes the execution time of the implemented sub-functions, with all values presented in clock cycles. Table 5.3 provides a breakdown, where the first column lists the function names, the second column shows execution times for the unmasked case, the third column presents execution times for the masked case, and the final column details the difference between the two.

5. Analysis

For functions that are linear with respect to the masking method, execution time increases proportionally to the number of shares. However, the *decompose* and *bound check* functions require entirely different implementations in the masked case, incorporating mask conversions and Boolean adders. As a result, their execution times increase significantly.

The execution times are listed for vector-based operations and are measured per vector of 256 coefficients.

The NTT and INTT implementations, derived from the work in [22], require 8'097 and 9'167 cycles, respectively. These execution times align closely with the results of the OTBN implementation reported in [22], where the corresponding cycle counts are 8'206 and 8'701 cycles.

Table 5.3.: Cycle Analysis for the ML-DSA Subfunctions.

Function	Unmasked	Masked	Difference
Vector NTT	8'097	16'194	8'097
Vector INTT	9'167	18'334	9'167
Vector add	1'526	3'052	1'526
Vector subtract	1'526	3'052	1'526
Vector coefficient-wise multiplication	4'092	8'186	4'092
Vector coefficient-wise multiply and add	5'126	10'252	5'126
Decompose	13'824	1'834'752	1'795'328
Bound check	11'520	818'944	806'144
A2B	-	603'136	603'136
B2A	-	1'210'624	1'210'624
Add	-	199'168	199'168
Add modulo	-	602'112	602'112

Table 5.4 presents an estimation of the total execution time for a single run of the ML-DSA sign algorithm, broken down by the sub-functions implemented in this work. The first column lists the function names, the second column provides their execution time in cycles, and the third column shows their percentage contribution to the estimated total execution time of the masked ML-DSA sign algorithm. Both unmasked and masked implementations are included, as only operations involving sensitive variables require secure implementations.

The execution time estimation is based on the average number of iterations in the main loop of the ML-DSA-44 sign algorithm, which is approximately 4.25. A worst-case assumption is made, where every operation in the main loop executes in each iteration without early termination. Additionally, it is assumed that the secure bound check is always performed on the entire vector. In practice, the bound check and the loop iteration can terminate early when a coefficient fails the check. Using these assumptions, the final

5. Analysis

execution count for each sub-function is determined and multiplied by the corresponding values in Table 5.3.

The second-to-last row shows the median execution time of 10,000 runs from the implementation in [22], which is approximately 4.35% of the estimated total execution time for the masked implementation in this work.

The total execution time in the last row is estimated by taking the median execution time from [22] and adding all additional cycles from the last column of Table 5.4. The masking applied in this work introduces an estimated 2'197% overhead compared to the unmasked reference implementation.

The *decompose* and *bound check* functions dominate execution time, jointly accounting for 99.86% of the total runtime.

Table 5.4.: Total execution time in cycles split into the sub-functions mentioned in this work. The third column is the percentage compared to the estimated total execution of the hardened ML-DSA sign algorithm. The last column contains the number of additional cycles due to hardening.

Function	Exec. time	Exec. time in %	Cycles added
Vector NTT	170'037	0.29	-
Secure vector NTT	404'850	0.68	202'425
Vector INTT	155'839	0.26	-
Secure vector INTT	935'034	1.58	467'517
Vector add	25'942	0.044	-
Secure vector add	51'884	0.088	25'942
Vector subtract	25'942	0.044	-
Secure vector subtract	51'884	0.088	25'942
Vector mul.	69'564	0.12	-
Secure vector mul.	417'486	0.71	208'743
Vector MAC	0	0	-
Secure vector MAC	522'852	0.88	261'426
Decompose	0	0	-
Secure decompose	31'190'784	52.76	30'520'576
Bound check	195'840	0.33	-
Secure bound check	27'844'096	47.10	27'408'896
Total unmasked exec. time	2'574'222	4.35	-
Total masked exec. time	59'121'467	100	56'547'245

Chapter 6

Hardware Accelerator for Boolean to Arithmetic Mask Conversion

Both Boolean-to-Arithmetic (B2A) and Arithmetic-to-Boolean (A2B) conversions are an essential part of the hardening approach discussed in chapter 4. The A2B function is invoked once per call to the `secBoundsCheck` function, while both A2B and B2A are called once per execution of the `secDecompose` function. Additionally, every B2A call internally triggers an A2B call. B2A approximately makes up 34.8% of the entire signing algorithm execution time. Hence, the overall execution time could be decreased by providing hardware support for the B2A mask conversion. In this section, we present our approach of accelerating B2A.

Each iteration of the main loop in the ML-DSA-44 signing algorithm results in 1'024 calls to `secDecompose` (processing four vectors with 256 coefficients each) and 2'048 calls to `secBoundsCheck` (processing four vectors with 256 coefficients each). Consequently, A2B is executed 4'096 times, while B2A is executed 1'024 times per iteration. Based on the cycle counts from Table 5.3, this amounts to 9'650'176 cycles for A2B and 4'842'496 cycles for B2A per main loop execution of ML-DSA-44. For an average of 4.25 iterations of the main loop per execution of the ML-DSA-44 sign algorithm, this amounts to 41'013'248 and 20'580'608 cycles.

While B2A implementations often rely on modulus switching, which involves a division by the modulus q , such as $y = \lfloor \frac{x \cdot 2^k}{q} \rfloor$, A2B implementations are typically more straightforward.

6.1. Design

The design of the efficient B2A mask conversion is based on [41] and incorporates an adapted version of Goubin’s method for securely performing mask conversion modulo 2^k . It also employs a secure Kogge-Stone Adder implementation to compute a masked borrow bit.

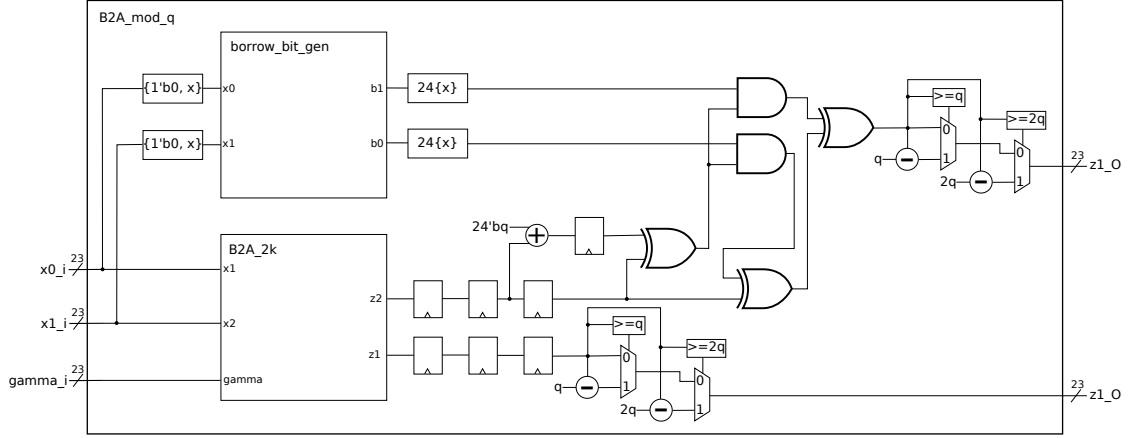


Figure 6.1.: The top level block for the Boolean to Arithmetic mask conversion modulo q $B2A_{modq}$.

Figure 6.1 illustrates the overall architecture, which consists of two main components: the B2A conversion block modulo 2^k and the borrow bit generator block. The following subsection provides a detailed discussion of these components.

6.1.1. Boolean to Arithmetic Mask Conversion Modulo q

The top-level module is the Boolean-to-Arithmetic mask conversion block, $B2A_q$. As shown in 6.1, the B2A implementation in this work consists of two primary sub-modules: the Boolean-to-Arithmetic conversion module ($B2A_{2k}$) and a module responsible for computing a masked borrow bit for the difference $x - q$, where x represents the unmasked share and q is the modulus used in ML-DSA.

The circuit in 6.1 implements **Algorithm 20**, which efficiently converts a Boolean-masked value into an Arithmetic-masked value modulo q . The core idea of the algorithm is to first transform the Boolean sharing of x into an Arithmetic sharing modulo 2^k using the **B2A2k** module. This step produces an Arithmetic sharing $t = t_1 - t_2 \bmod 2^k$.

Once the Boolean mask is converted to its Arithmetic counterpart, the unmasked value t falls into one of two possible ranges:

- If $0 \leq t < q$, the result is already correctly represented.

6. Hardware Accelerator for Boolean to Arithmetic Mask Conversion

- If $q \leq t < 2^k$, an adjustment is required by securely subtracting q from the shared value t .

The **BorrowBitGen** submodule is responsible for securely determining into which range t falls.

If the unmasked Boolean borrow bit sharing b is equal to 0, then we are in the range $0 \leq t < q$. Conversely, if $b = 1$, we are in the range $q \leq t < 2^k$.

In the first case, we can take t_2 as is. In the latter case, we add q to t_2 , which effectively subtracts q from the unmasked value, ensuring that $t = t_1 - t_2 \pmod q$. This adjustment is performed in lines 4 and 5 of Algorithm 20.

The values $\tilde{b}_1$ and $\tilde{b}_2$ are extended versions of the single-bit borrow values b_1 and b_2 , respectively, expanded to 25 bits. Finally, line 7 ensures the correct computation of z_2 based on the borrow bit's value.

The primary distinction between this work and 6.1 lies in the representation of the final result. In this work, the result is expressed as $z = z_1 + z_2 \pmod q$, whereas in 6.1, it is represented as $z = z_1 - z_2 \pmod q$.

Additionally, in 6.1, the values of t_1 and t_2 can exceed q . To address this, a final reduction step is introduced in this work on lines 6 and 7, ensuring that the computed values remain within the desired range.

Algorithm 20: SecB2Aq: 1-PINI Secure $B2A_q$ Mask Conversion

input : Shares $x = x_1 \oplus x_2$, where $x_1, x_2 \in \mathbb{Z}_{2^k}$, modulus $q < 2^k$.

output: $z = (z_1, z_2)$ where $x = z_1 + z_2$ with $x \in [0, q - 1]$

```

1  $(t_1, t_2) \leftarrow \text{B2A2k}(x_1, x_2)$ 
2  $u \leftarrow t_2 + q$ 
3  $(b_1, b_2) \leftarrow \text{BorrowBitGen}(x_1, x_2, q)$ 
4  $w_1 \leftarrow (\tilde{b}_1 \wedge (t_2 \oplus u)) \oplus t_2$ 
5  $w_2 \leftarrow (\tilde{b}_2 \wedge (t_2 \oplus u))$ 
6  $z_1 \leftarrow t_1 \pmod q$ 
7  $z_2 \leftarrow (q - (w_1 \oplus w_2)) \pmod q$ 
8 return( $z = (z_1, z_2)$ )
```

6.1.2. Boolean to Arithmetic Conversion Modulo 2^k

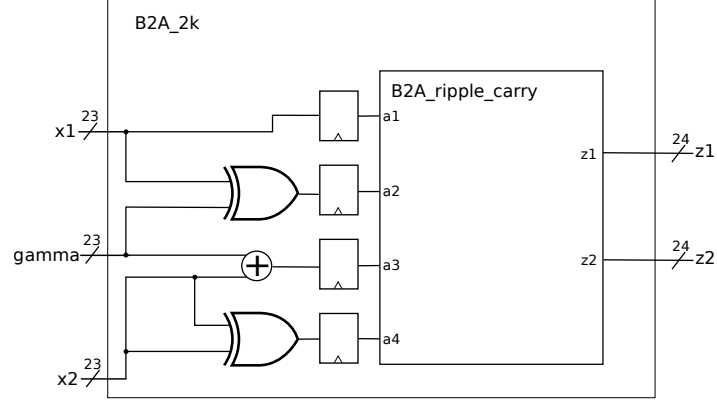


Figure 6.2.: Boolean-to-Arithmetic conversion block (B2A_2k).

The Boolean-to-Arithmetic conversion block modulo 2^k in 6.2 is based on Goubin's method. The process begins by computing a set of intermediate values using the Boolean sharing $x = x_1 \oplus x_2$ and a 23-bit random value γ . These intermediate values are defined as follows:

$$a_1 = x_1 \tag{6.1}$$

$$a_2 = x_1 \oplus \gamma \tag{6.2}$$

$$a_3 = x_2 + \gamma \tag{6.3}$$

$$a_4 = x_2 \oplus \gamma \tag{6.4}$$

6. Hardware Accelerator for Boolean to Arithmetic Mask Conversion

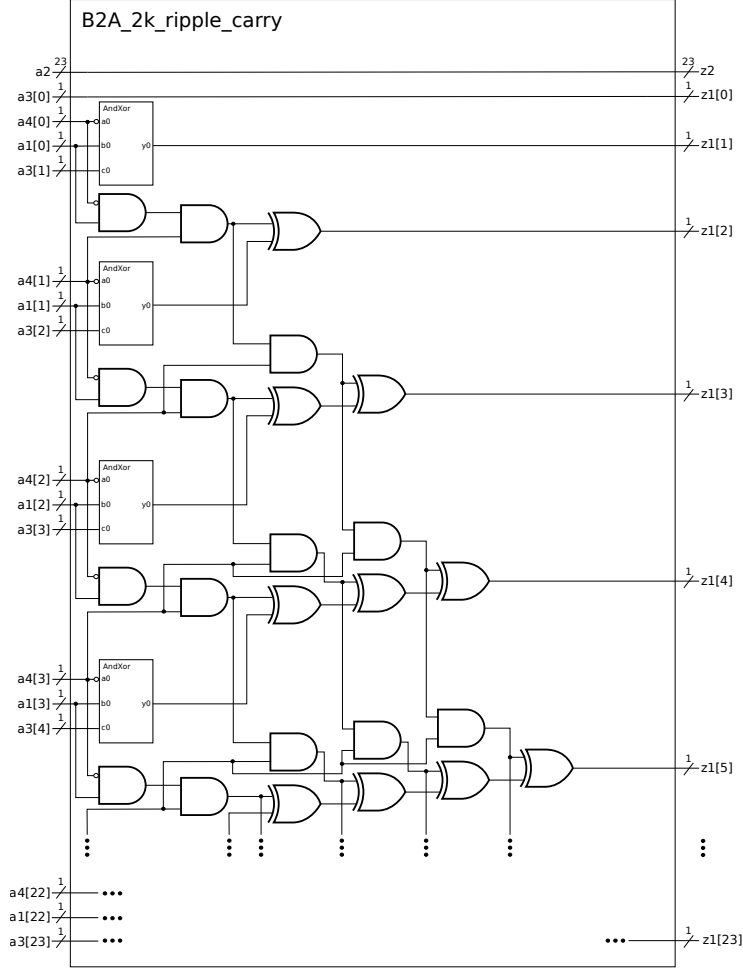


Figure 6.3.: The Ripple Carry Adder used in the Boolean to Arithmetic mask conversion modulo 2^k B2A2kRippleCarry.

Using these intermediate values, the output sharing is computed through the Ripple Carry Adder construction, as shown in 6.3. The final output is an arithmetic sharing modulo 2^k , calculated as follows:

$$z_1 = ((a_4 \oplus a_1) + a_1) \oplus a_3 \oplus a_4 \quad (6.5)$$

$$z_2 = a_2 \quad (6.6)$$

6. Hardware Accelerator for Boolean to Arithmetic Mask Conversion

where $x = z_1 - z_2$, with the relationships:

$$z_1 = x + (x_1 \oplus \gamma) \quad (6.7)$$

$$z_2 = x_1 \oplus \gamma \quad (6.8)$$

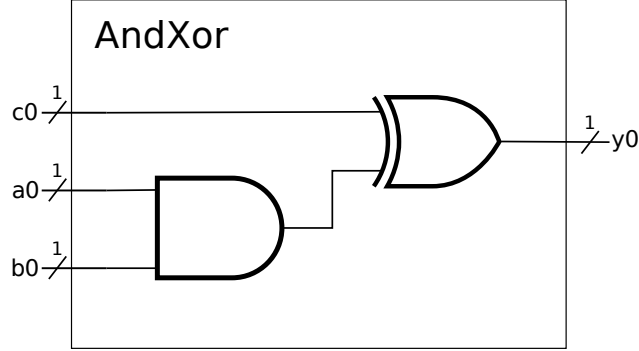


Figure 6.4.: This block only contains an *AND* and an *XOR* gate and is there to simplify the Ripple Carry Adder block diagram.

The Ripple Carry Adder in 6.3 makes use of the *AndXor* block in 6.4 to simplify the design.

6.1.3. Borrow Bit Generator

To compute the masked borrow bit, the top-level block in Algorithm 20 utilizes a secure Kogge-Stone Adder construction consisting of five stages. The objective is to determine the overflow bit resulting from the addition of the two shares of x , where $x = x_1 \oplus x_2$, with $2^k - q$. As previously explained, this step is crucial for distinguishing whether the unmasked value falls within the range $0 \leq x < q$ or $q \leq x < 2^k$.

6. Hardware Accelerator for Boolean to Arithmetic Mask Conversion

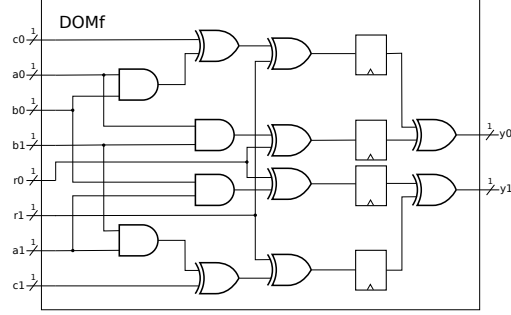


Figure 6.5.: The $DOMf$ block, which can be used as $secAnd$, $secAndRefresh$, $secAndXor$ or $secAndXorRefresh$.

Each of the five stages leverages the $DOMf$ block, as illustrated in Figure 6.5. This block efficiently implements a masked Boolean function, which takes the inputs $a_0, a_1, b_0, b_1, c_0, c_1, r_0, r_1$ and produces the masked outputs y_0, y_1 . These outputs are computed as follows:

$$f_0(a_0, b_0, c_0, r^1) = a_0b_0 \oplus c_0 \oplus r^1 \rightarrow y'_0 \quad (6.9)$$

$$f_1(a_0, b_1, r^0) = a_0b_1 \oplus r^0 \rightarrow y'_1 \quad (6.10)$$

$$f_2(a_1, b_0, r^0) = a_1b_0 \oplus r^0 \rightarrow y'_2 \quad (6.11)$$

$$f_3(a_1, b_1, c_1, r^1) = a_1b_1 \oplus c_1 \oplus r^1 \rightarrow y'_3 \quad (6.12)$$

$$y'_0 \oplus y'_1 = y_0, \quad (6.13)$$

$$y'_2 \oplus y'_3 = y_1. \quad (6.14)$$

The variables a, b , and c represent masked bits, while r_0 and r_1 are random bits used to refresh the outputs. The $DOMf$ function can be used to implement four different secure operations, depending on the selection of input variables:

- Using only the black-colored variables results in the $secAnd$ function.
- Incorporating both black and red variables produces the $secAndRefresh$ function.
- Utilizing black and blue variables yields the $secAndXor$ function.
- When all three—black, red, and blue variables—are included, the function becomes the $secAndXorRefresh$ variant.

6. Hardware Accelerator for Boolean to Arithmetic Mask Conversion

The five stages are best understood by first explaining the Kogge-Stone Adder. In this algorithm, the carry bits for two k -bit inputs, a and b , are computed through a pre-processing step followed by a prefix-computation step.

In the initial pre-processing step, the propagate and generate bits are calculated as follows:

$$g^i = a^i \wedge b^i, \quad (6.15)$$

$$p^i = a^i \oplus b^i. \quad (6.16)$$

Here, a^i and b^i represent the i -th bits of the inputs a and b .

In the prefix-computation step, the carry bits are then computed over $\log_2 k$ stages. For $k = 23$, this results in five stages. In each stage, the group carry propagate $p^{i:j}$ and the group carry generate $g^{i:j}$ values are computed for $i > j$ as follows:

$$g^{i:j} = g^i \oplus (g^j \wedge p^i), \quad (6.17)$$

$$p^{i:j} = p^i \wedge p^j. \quad (6.18)$$

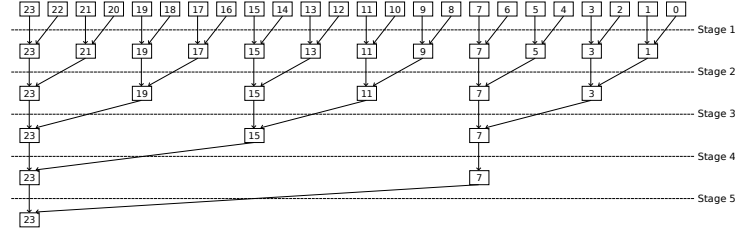


Figure 6.6.: This figure explains how the indices should be combined in each stage.

The specific indices i and j that are combined at each stage can be observed in Figure 6.6. In stage 4, the 7th bit is simply forwarded to the next stage.

6. Hardware Accelerator for Boolean to Arithmetic Mask Conversion

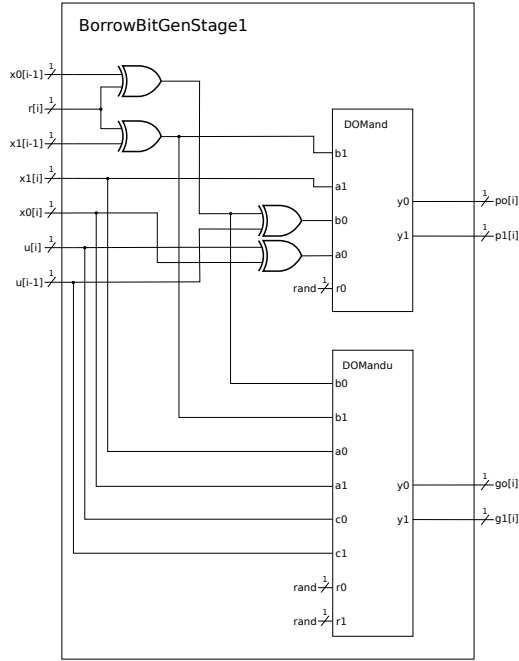


Figure 6.7.: Stage one of the secure Kogge-Stone Adder implementation *borrowBitGenStage1*.

To make the implementation power side-channel resistant, the $DOMf$ function is utilized in place of standard XOR and AND operations. As a result, stage 1 is structured as shown in Figure 6.7.

6. Hardware Accelerator for Boolean to Arithmetic Mask Conversion

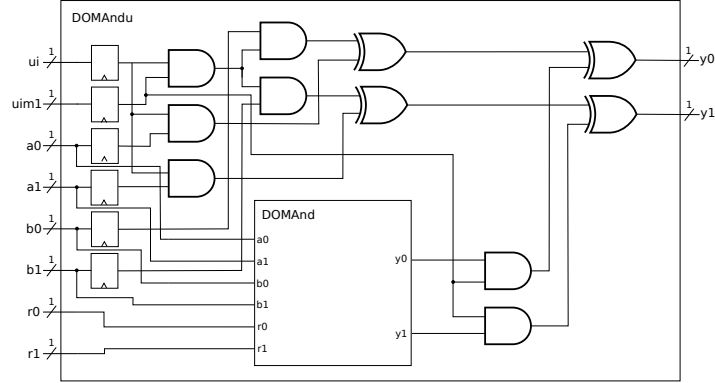


Figure 6.8.: This block implements a protected version of the carry generate bit calculation while also adding $u = 2^k - q$.

To compute the carry generate bits while adding $2^k - q$ to the sharing of x , we employ the secure block illustrated in Figure 6.8. This block implements a protected version of the carry generate bit calculation:

$$g = (u_j \wedge x_j \wedge x_i) \oplus (u_i \wedge x_i) \oplus (x_j \wedge u_j \wedge x_i), \quad (6.19)$$

where:

- u_i and u_j are two neighboring bits in $2^k - q$,
- x_i and x_j are two neighboring bits in the masked representation of x .

Additionally, before the inputs of stage 1 enter the $DOMf$ blocks, the even bits are refreshed.

6. Hardware Accelerator for Boolean to Arithmetic Mask Conversion

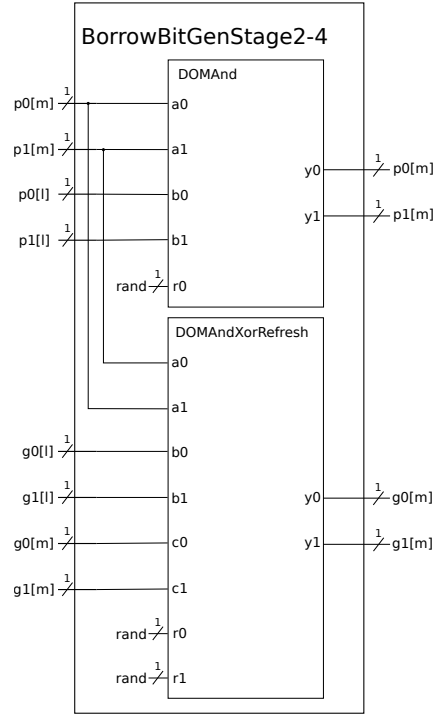


Figure 6.9.: Stages two to four of the secure Kogge-Stone Adder implementation *borrowBitGenStage2 – 4*.

6. Hardware Accelerator for Boolean to Arithmetic Mask Conversion

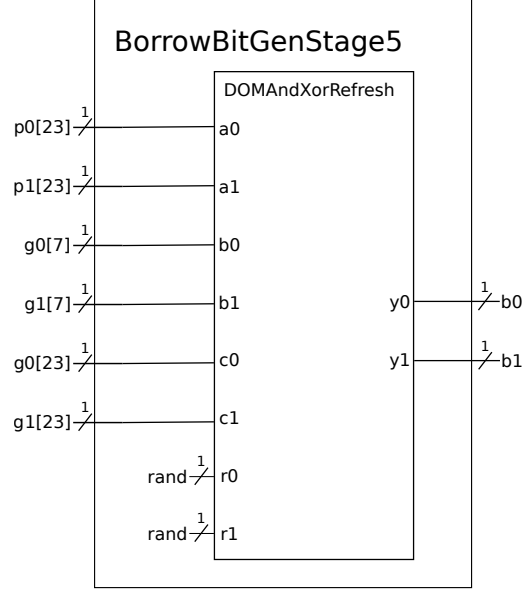


Figure 6.10.: Stage five of the secure Kogge-Stone Adder implementation *borrowBitGenStage5*.

Stages 2 to 4 consist of secure implementations of the Kogge-Stone Adder stages, as illustrated in Figure 6.9. In the final stage, the borrow bit is determined by computing the generate bits of the most significant bit (MSB) of the inputs. This final computation is depicted in Figure 6.10.

A key distinction between the Kogge-Stone Adder implementation in this work and the one presented in [41] lies in the handling of the 7th carry bits. In this work, the 7th carry bits are forwarded from stage 4 to stage 5, where they are then combined with the 23rd carry bits to compute the final masked borrow bits. This modification is essential for ensuring the correctness of the algorithm.

6.2. Functional Verification

To ensure the correctness of the design, functional verification was performed. A dedicated testbench was developed for each circuit block discussed in this work.

6. Hardware Accelerator for Boolean to Arithmetic Mask Conversion

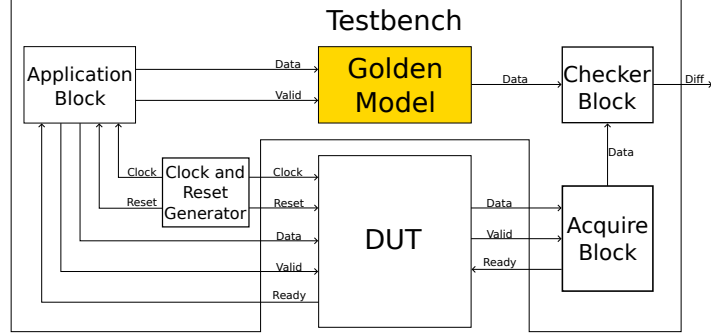


Figure 6.11.: Functional verification setup.

For every module, a SystemVerilog testbench was implemented, with the golden model embedded directly within the testbench. Figure 6.11 illustrates the testbench setup.

For simpler blocks, such as the *DOM_f* blocks (Figure 6.5), the golden model computes the exact expected result. However, for more complex blocks, including the borrow bit generator, the B2A_2k block (Figure 6.2), and the top-level module *B2Amodq* (Algorithm 20), the acquired results from the DUT are first unmasked and then compared against the expected unmasked values.

Each module has undergone verification, with at least 10,000 random test vectors used per verification run.

6.3. Results

The final design of the B2A mask conversion accelerator achieves a latency of 5 cycles and is fully pipelined, enabling a throughput of one B2A mask conversion per cycle.

As shown in Table 6.1, utilizing the hardware accelerator for B2A mask conversion dramatically reduces execution time. The B2A conversion for a vector with 256 coefficients takes only 1'280 cycles, compared to 1'210'624 cycles in the base software implementation—just 0.11% of the original execution time. Consequently, the *decompose* function, which relies on B2A mask conversion, experiences a substantial performance boost. With hardware acceleration, it requires only 34.39% of the cycles compared to the base implementation.

6. Hardware Accelerator for Boolean to Arithmetic Mask Conversion

Table 6.1.: Cycle Analysis for the accelerated ML-DSA Subfunctions.

Function	Cycles base impl.	Cycles accel. impl.	Comparison in %
Decompose	1'834'752	625'408	34.09
B2A	1'210'624	1'280	0.11

Table 6.2 presents the same execution time estimation as Table 5.4, but with the proposed hardware accelerator incorporated. The accelerator significantly reduces the estimated execution time from 59,121,467 cycles to 41,136,841 cycles. This optimization results in the accelerated implementation requiring only 69.58% of the cycles compared to the base implementation, demonstrating a substantial performance improvement.

Table 6.2.: Total execution time in cycles split into the sub-functions mentioned in this work using the proposed hardware accelerator. The third column is the percentage compared to the estimated total execution of the hardened ML-DSA sign algorithm. The last column contains the number of additional cycles due to hardening.

Function	Exec. time in cycles	Exec. time in %	Cycles added
Vector NTT	170'037	0.41	-
Secure vector NTT	404'850	0.98	202'425
Vector INTT	155'839	0.38	-
Secure vector INTT	935'034	2.27	467'517
Vector add	25'942	0.063	-
Secure vector add	51'884	0.13	25'942
Vector subtract	25'942	0.063	-
Secure vector subtract	51'884	0.13	25'942
Vector mul.	69'564	0.17	-
Secure vector mul.	417'486	1.01	208'743
Vector MAC	0	0	-
Secure vector MAC	522'852	1.27	261'426
Decompose	0	0	-
Secure decompose	10'631'936	25.85	9'961'728
Bound check	195'840	0.48	-
Secure bound check	27'844'096	67.69	27'408'896
Total unmasked exec. time	2'574'222	6.26	-
Total masked exec. time	41'136'841	100	38'562'619

The B2A mask conversion accelerator modulo q utilizes 103 bits of randomness per conversion for refreshing the intermediate values during computation.

As shown in Tables 6.3 and 6.4, after synthesis and routing, the total size of the accel-

6. Hardware Accelerator for Boolean to Arithmetic Mask Conversion

erator consists of approximately 392 Look-Up Tables (LUTs) and 473 Flip-Flops (FFs). The design was synthesized for a clock frequency of 24 MHz, aligning with the operating frequency of OTBN.

In terms of resource utilization on the Arty A7-35t FPGA, the accelerator utilizes approximately 1.5% of OTBN’s LUTs and 10.7% of its Flip-Flops.

The original implementation presented in [41] reports 76 (188) LUTs and 94 (126) FFs for the small (fast) implementation. However, a direct comparison to the design in this work remains challenging, as the authors do not provide a public implementation for further analysis.

Table 6.3.: Resource utilization of the B2A mask conversion after synthesis and implementation, split up into sub-modules and compared to OTBN.

Module	Tot. LUTs	Logic LUTs	LUTRAMs	SRLs
OTBN	25417	24841	576	0
<i>B2Amoduloq</i>	392	378	0	14
<i>BorrowBitGen</i>	192	192	0	0
<i>B2Amodulo2^k</i>	104	104	0	0
<i>secAnd</i>	6	6	0	0
<i>SecAndU</i>	5	5	0	0
<i>SecAndXorRefr</i>	6	6	0	0

Table 6.4.: Resource utilization of the B2A mask conversion after synthesis and implementation, split up into sub-modules and compared to OTBN.

Module	FFs	RAMB36	RAMB18	DSP Blocks
OTBN	4404	10	1	16
<i>B2Amoduloq</i>	473	0	0	0
<i>BorrowBitGen</i>	163	0	0	0
<i>B2Amodulo2^k</i>	115	0	0	0
<i>secAnd</i>	4	0	0	0
<i>SecAndU</i>	10	0	0	0
<i>SecAndXorRefr</i>	4	0	0	0

Conclusion and Future Work

To conclude, this chapter provides a summary of the research conducted, highlighting key findings and contributions. It also discusses the limitations of the study and explores potential directions for future research.

7.1. Summary

This thesis investigated the security and performance optimization of an ML-DSA signing algorithm implementation, which is focused on resistance to power side-channel analysis. The study involved identifying security-sensitive sub-functions, implementing hardened versions, and designing a hardware accelerator to improve the execution time.

7.2. Key Findings

The results demonstrate that the power side-channel-hardened implementations presented in this work significantly reduce leakage in sub-functions operating on security-sensitive data. The applied hardening techniques increased data memory usage by 40.8%, instruction memory by 8.28%, and execution time from 2,574,222 to an estimated 59,121,467 cycles. Notably, two of the eight hardened sub-functions account for the majority of this execution overhead, highlighting opportunities for further optimization.

To address this, a hardware accelerator for Boolean-to-Arithmetic mask conversion was proposed. The accelerator is estimated to achieve a 30% reduction in execution time while increasing OTBN area by only 1.5%, demonstrating a practical balance between performance and FPGA resource utilization.

7.3. Limitations

This work primarily focuses on the implementation and analysis of security-sensitive sub-functions. While these sub-functions are the most relevant in the context of power side-channel analysis, a broader examination of the entire ML-DSA signing algorithm would provide additional valuable insights. Furthermore, this work does not implement hardened versions of sub-functions that rely on secure hash functions.

Moreover, ML-DSA comprises a triple of algorithms. This study exclusively analyzes the ML-DSA signing algorithm, while the ML-DSA verification algorithm, though not relevant to power side-channel attacks as it does not process sensitive data, remains unexamined. However, the ML-DSA key generation algorithm, which does operate on sensitive data, warrants a hardened implementation and thorough side-channel analysis.

Additionally, while this work enhances resistance to power side-channel attacks, it does not address other potential threats, such as fault injection attacks. Furthermore, the proposed hardware accelerator was evaluated on an FPGA, and further validation on an ASIC could provide deeper insights into its real-world feasibility and performance trade-offs.

7.4. Future Work

Future research could focus on implementing and conducting a side-channel analysis of a hardened ML-DSA key generation algorithm, similar to this work. Additionally, future efforts could extend to implementing the complete ML-DSA signing algorithm, including its security-sensitive sub-functions that rely on hash functions.

Moreover, the current implementation of the ML-DSA signing algorithm presents significant opportunities for optimization and hardware acceleration. For instance, the Arithmetic-to-Boolean mask conversion and the secure adder could be implemented in hardware, potentially achieving performance gains similar to those observed with the Boolean-to-Arithmetic mask conversion accelerator proposed in this work.

Further research could also explore countermeasures against other side-channel attacks, such as fault injection, and analyze the implementation on an ASIC to gain deeper insights.

7.5. Final Words

As quantum computing continues to advance, securing cryptographic implementations against emerging threats is crucial. This work contributes to the ongoing efforts in post-

7. Conclusion and Future Work

quantum cryptography by enhancing the security of ML-DNA, paving the way for more robust digital signatures in the quantum era.

Appendix	A
----------	----------

Task Description



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Institut für Integrierte Systeme
Integrated Systems Laboratory

TASK ASSIGNMENT FOR A
MASTER'S THESIS AT THE DEPARTMENT OF
INFORMATION TECHNOLOGY AND ELECTRICAL ENGINEERING

FALL SEMESTER 2024

Hakim Filali

**Power Side-Channel Evaluation and Hardening of
PQC Algorithms on OpenTitan**

September 16, 2024

Advisors: Pirmin Vogel, vogelpi@lowrisc.org
Pascal Nasahl, nasahlpa@lowrisc.org
Christopher Reinwardt, ETZ J76.2, creinwar@iis.ee.ethz.ch

Professor: Prof. Dr. L. Benini

Handout: September 16, 2024

Due: March 17, 2025

The final report is to be submitted electronically. All copies remain property of the Integrated Systems Laboratory.

1 Introduction

With the expected uprise of quantum computers, the security of well-established cryptographic schemes could be threatened in the future. Especially the resilience of public-key cryptography, e.g., RSA (Rivest–Shamir–Adleman), DH (Diffie–Hellman), or ECC (Elliptic-Curve Cryptography), could be reduced by Shor’s algorithm [1, 2]. Although currently available quantum computers are not advanced enough to pose a threat to cryptography, i.e., they cannot offer enough high-quality, error-corrected qubits, the National Institute of Standards and Technology (NIST) urges companies to prepare information security systems to be able to resist quantum computing [3].

Post quantum cryptography (PQC) is believed to offer protection against upcoming quantum computers as well as cryptanalysis approaches performed on traditional computers. To accelerate the development of such PQC schemes, NIST opened a call for proposals for selecting post quantum secure public-key cryptographic algorithms. In June 2022, they announced the first batch of post-quantum cryptographic algorithms slated for standardization which came to fruition in August of 2024 with the release of the FIPS-[203,204,205] standards [4]. FIPS-204 [5] is the Module-Lattice-Based Digital Signature Standard (ML-DSA), which is based on Dilithium [6], that is intended to sign and verify messages. The security of ML-DSA is based on lattices, which is believed to be a hard problem for quantum computers.

However, post quantum cryptography is still a rather young research field [7], especially in comparison to well-established ciphers, such as AES. This is particularly noticeable when analyzing the state-of-the-art of current side-channel analysis (SCA) attacks and corresponding countermeasures aiming to mitigate this threat. Currently, there is a lack of scientific articles that provide an in-depth analysis and comparison of unprotected and SCA protected ML-DSA implementations on real-world hardware.

Nevertheless, post quantum cryptography schemes still need to be adapted by cutting-edge platforms to provide a secure foundation for the future. One example of such a platform is OpenTitan, which is an open-source design that is intended to be used as a secure root of trust in complex systems, i.e., laptops and servers. Hence, it is from uttermost importance to fully understand the SCA security properties when enabling ML-DSA on OpenTitan.

2 Project Description

The core goal of the project is to work towards a side-channel analysis hardened ML-DSA implementation running on OpenTitan’s Big Number Accelerator (OTBN). To achieve this goal, the student first needs to identify sub-functions of the ML-DSA algorithm that are especially vulnerable against side-channel analysis attacks by studying related work. Afterwards, these sub-functions should be ported to OTBN using assembly

code and the functionality of the code base needs to be checked using a reference model. Here, the goal is to run these sub-functions on OTBN on the CW310 FPGA board that will be provided by lowRISC. To enable power side-channel measurements of ML-DSA, the student needs to extend the existing side-channel infrastructure with support for measuring the targeted algorithms on the FPGA. With the captured power side-channel traces, a security analysis should be performed, revealing the vulnerable spots of the implementation. Using this knowledge and related work, the student needs to enhance the resilience of the implemented sub-functions by applying suitable protection mechanisms, i.e., masking. Finally, a re-evaluation should demonstrate whether the embedded countermeasures are capable of protecting the execution of ML-DSA against SCA. Hereby, a performance comparison between the protected and unprotected implementation needs to be conducted to show the performance penalty.

3 Milestones

The following are the milestones that we expect to achieve throughout the project:

3.1 Milestone 1 - Getting Started

The goal of this task is that the student familiarizes himself with the ML-DSA signature algorithm, side-channel analysis attacks, and the big number accelerator provided in the OpenTitan project. Hereby, the student will perform a literature review that summarizes the functionality of ML-DSA, side-channel attacks and countermeasures in general as well as in the case for the targeted signature scheme. Using this information, ML-DSA sub-functions that are particularly vulnerable against side-channel attacks are extracted. By deploying and analyzing existing programs on OTBN in the system model (e.g., the OTBN Python simulator or Verilator), the student will learn about the inner working of the accelerator. The **outcome** of this milestone is:

- A draft of the background section of the thesis containing information about ML-DSA, Side-Channel Attacks, and OpenTitan.
- A list of ML-DSA sub-functions that are especially sensitive to side-channel analysis attacks.
- A working environment allowing the student to experiment with OTBN.

3.2 Milestone 2 - Unprotected OTBN Implementation

After having familiarizing with the fundamental corner stones of the project, the student then will implement the extracted ML-DSA sub-functions for OTBN. This implementation will be done in OTBN assembly and can be based on related work [8, 9]. Each

implemented OTBN sub-function is tested against a reference model to ensure the functional correctness. Finally, the student will map OpenTitan to a CW310 FPGA board and test the algorithms on real hardware. The **outcome** of this milestone is:

- Unprotected sub-functions of ML-DSA implemented in OTBN assembly.
- Tests to ensure the functionalities of these sub-functions.
- A working FPGA platform capable of running the OTBN programs.

3.3 Milestone 3 - Side-Channel Infrastructure

To prepare for the side-channel measurements, the student needs to setup the side-channel evaluation gear and gets familiar with it. This gear consists of a ChipWhisperer Husky scope that is capable of capturing power traces on the CW310 target board. In order to verify the correct setup, existing demo SCA measurements available in lowRISC's side-channel framework [10], e.g., targeting AES, can be performed. For measuring the ML-DSA sub-functions on OTBN, the student will need to extend this framework and also provide code on OpenTitan that establishes a communication interface with the framework. The **outcome** of this milestone is:

- A functional SCA setup that is verified using existing SCA targets, e.g., AES.
- A modified SCA framework that is capable of triggering measurements of the ML-DSA sub-functions on the device.

3.4 Milestone 4 - Side-Channel Analysis

Finally, after implementing the ML-DSA sub-functions and preparing the side-channel infrastructure, the student will capture power traces and analyzes them for leakage. Hereby, the student will need to determine which number of power traces and testing strategy, e.g., Test Vector Leakage Assessment (TVLA), is suitable. The **outcome** of this milestone is:

- Strategy for analyzing leakage of the ML-DSA sub-functions.
- Captured power traces and performed analysis.

3.5 Milestone 5 - Hardened OTBN Implementation

Based on the previous leakage assessment and related work, the student selects suitable protection mechanisms, e.g., masking, to enhance the resilience of the sub-functions against side-channel analysis attacks. Then, the baseline implementation will be extended by applying the selected countermeasures. The **outcome** of this milestone is:

- Selected mitigation strategies.

- Implemented the countermeasure for the ML-DSA sub-functions.

3.6 Milestone 6 - Re-Evaluation

Finally, the student will perform a re-evaluation of the side-channel resilience of the protected implementation. This analysis will be based on the outcomes of Milestone 3, 4, and 5. Moreover, for a comparison with the unprotected implementation, an analysis of the performance and code size will be conducted. The **outcome** of this milestone is:

- Side-channel assessment of the protected implementation and a comparison with the unprotected version.
- Performance and code size comparison between the protected and baseline implementation.

3.7 Stretch Goals

Should the above milestones be reached earlier than expected and the student is motivated to do further work, we propose the following stretch goal to aim for. Protecting the sub-functions of ML-DSA will induce a larger performance overhead. For the stretch goal, the student will identify spots in the protected implementation that are responsible for a large portion of the overhead, e.g., the arithmetic to boolean and boolean to arithmetic conversion functions, that are frequently used in masking. By studying related work, analyze which of these performance critical functions can be realized in hardware. Extend the OTBN instruction set architecture (ISA) in hardware with these functions. Re-evaluate the implementation by conducting a side-channel assessment, measure the performance and code size overhead, and extract the area overhead.

4 Project Realization

4.1 Time Schedule

The time schedule presented in Table 1 is merely a proposition; it is primarily intended as a reference and an estimation of the time required for each required step.

Project phase	Time estimate
Milestone 1	2 weeks
Milestone 2	2 weeks
Milestone 3	3 weeks
Milestone 4	3 weeks
Milestone 5	3 weeks
Milestone 5	2 weeks
<i>Stretch goals</i>	<i>remaining time</i>
Write report	2 weeks
Prepare presentation	1 week

Table 1: Proposed time schedule and investment

4.2 Meetings

Weekly meetings will be held between the student and the assistants. The exact time and location of these meetings will be determined within the first week of the project in order to fit the student's and the assistants' schedule. These meetings will be used to evaluate the status and progress of the project. Beside these regular meetings, additional meetings can be organized to address urgent issues as well.

4.3 Weekly Reports

The student is required to write a weekly report at the end of each week and to send it to his advisors by email. The idea of the weekly report is to briefly summarize the work, progress and any findings made during the week, to plan the actions for the next week, and to discuss open questions and points. The weekly report is also an essential means for the student to get a goal-oriented attitude to work.

4.4 Coding Guidelines

HDL Code Style Adapting a consistent code style is one of the most important steps in order to make your code easy to understand. If signals, processes, and modules are always named consistently, any inconsistency can be detected more easily. Moreover, if a design group shares the same naming and formatting conventions, all members immediately *feel at home* with each other's code. At IIS, we use lowRISC's style guide for SystemVerilog HDL: <https://github.com/lowRISC/style-guides/>.

Software Code Style We generally suggest that you use style guides or code formatters provided by the language’s developers or community. For C/C++¹, Python², and OTBN³, we recommend OpenTitan’s style guides.

Version Control Even in the context of a student project, keeping a precise history of changes is *essential* to a maintainable codebase. You may also need to collaborate with others, adopt their changes to existing code, or work on different versions of your code concurrently. For all of these purposes, we heavily use *Git* as a version control system at IIS. If you have no previous experience with Git, we *strongly* advise you to familiarize yourself with the basic Git workflow before you start your project.

4.5 Report

Documentation is an important and often overlooked aspect of engineering. A final report has to be completed within this project.

The common language of engineering is de facto English. Therefore, the final report of the work is preferred to be written in English.

Any form of word processing software is allowed for writing the reports, nevertheless the use of L^AT_EX with Inkscape or any other vector drawing software (for block diagrams) is strongly encouraged by the IIS staff.

If you write the report in L^AT_EX, we offer an instructive, ready-to-use template, which can be forked from the Git repository at <https://iis-git.ee.ethz.ch/akurth/iisreport>.

Final Report The final report has to be presented at the end of the project and a digital copy needs to be handed in and remain property of the IIS. Note that this task description is part of your report and has to be attached to your final report.

4.6 Presentation

There will be a presentation (20 min presentation and 5 min Q&A) at the end of this project in order to present your results to a wider audience. The exact date will be determined towards the end of the work.

¹https://opentitan.org/book/doc/contributing/style_guides/c_cpp_coding_style.html

²https://opentitan.org/book/doc/contributing/style_guides/python_coding_style.html

³https://opentitan.org/book/doc/contributing/style_guides/otbn_style_guide.html

5 Deliverables

In order to complete the project successfully, the following deliverables have to be submitted at the end of the work:

- Final report incl. presentation slides
- Source code and documentation for all developed software and hardware
- Testsuites (software) and testbenches (hardware)
- Synthesis and implementation scripts, results, and reports

References

- [1] P. W. Shor, “Algorithms for quantum computation: Discrete logarithms and factoring,” in *35th Annual Symposium on Foundations of Computer Science, Santa Fe, New Mexico, USA, 20-22 November 1994*. IEEE Computer Society, 1994, pp. 124–134. [Online]. Available: <https://doi.org/10.1109/SFCS.1994.365700>
- [2] —, “Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer,” *SIAM J. Comput.*, vol. 26, no. 5, pp. 1484–1509, 1997. [Online]. Available: <https://doi.org/10.1137/S0097539795293172>
- [3] “Post-Quantum Cryptography (PQC),” <https://csrc.nist.gov/Projects/post-quantum-cryptography/post-quantum-cryptography-standardization/Call-for-Proposals>, accessed: 2024-09-12.
- [4] “NIST Releases First 3 Finalized Post-Quantum Encryption Standards,” <https://www.nist.gov/news-events/news/2024/08/nist-releases-first-3-finalized-post-quantum-encryption-standards>, accessed: 2024-09-12.
- [5] “FIPS-204 Module-Lattice-Based Digital Signature Standard,” <https://csrc.nist.gov/pubs/fips/204/final>, accessed: 2024-09-12.
- [6] L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, and D. Stehlé, “Crystals-dilithium: A lattice-based digital signature scheme,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 238–268, 2018.
- [7] E. Krahmer, P. Pessl, G. Land, and T. Güneysu, “Correction fault attacks on randomized crystals-dilithium,” *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2024, no. 3, pp. 174–199, 2024. [Online]. Available: <https://doi.org/10.46586/tches.v2024.i3.174-199>

- [8] A. Abdulrahman, F. Oberhansl, H. N. H. Pham, J. Philipoom, P. Schwabe, T. Stelzer, and A. Zankl, "Towards ML-KEM & ML-DSA on opentitan," *IACR Cryptol. ePrint Arch.*, p. 1192, 2024. [Online]. Available: <https://eprint.iacr.org/2024/1192>
- [9] E. Urquhart, "Acceleration of post-quantum cryptography on opentitan big number accelerator using instruction set extensions," *Trinity College*, 2024.
- [10] "ot-sca - Side-Channel Analysis & Fault Injection Setup for OpenTitan," <https://github.com/lowRISC/ot-sca>, accessed: 2024-09-13.

Zurich, September 16, 2024

Prof. Dr. L. Benini

Appendix	B
----------	----------

Declaration of Originality



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Declaration of originality

The signed declaration of originality is a component of every written paper or thesis authored during the course of studies. In consultation with the supervisor, one of the following three options must be selected:

- ☐ I confirm that I authored the work in question independently and in my own words, i.e. that no one helped me to author it. Suggestions from the supervisor regarding language and content are excepted. I used no generative artificial intelligence technologies¹.
- ☐ I confirm that I authored the work in question independently and in my own words, i.e. that no one helped me to author it. Suggestions from the supervisor regarding language and content are excepted. I used and cited generative artificial intelligence technologies².
- ☒ I confirm that I authored the work in question independently and in my own words, i.e. that no one helped me to author it. Suggestions from the supervisor regarding language and content are excepted. I used generative artificial intelligence technologies³. In consultation with the supervisor, I did not cite them.

Title of paper or thesis:

Power Side-Channel Evaluation and Hardening of PQC Algorithms on OpenTitan

Authored by:

If the work was compiled in a group, the names of all authors are required.

Last name(s):

Filali

First name(s):

Hakim

With my signature I confirm the following:

- I have adhered to the rules set out in the Citation Guide.
- I have documented all methods, data and processes truthfully and fully.
- I have mentioned all persons who were significant facilitators of the work.

I am aware that the work may be screened electronically for originality.

Place, date

Zürich, 17.03.2025

Signature(s)

If the work was compiled in a group, the names of all authors are required. Through their signatures they vouch jointly for the entire content of the written work.

¹ E.g. ChatGPT, DALL E 2, Google Bard

² E.g. ChatGPT, DALL E 2, Google Bard

³ E.g. ChatGPT, DALL E 2, Google Bard

Appendix C

File Structure

This chapter in the appendix provides an overview and description of the file structure for the most important files involved in this work.

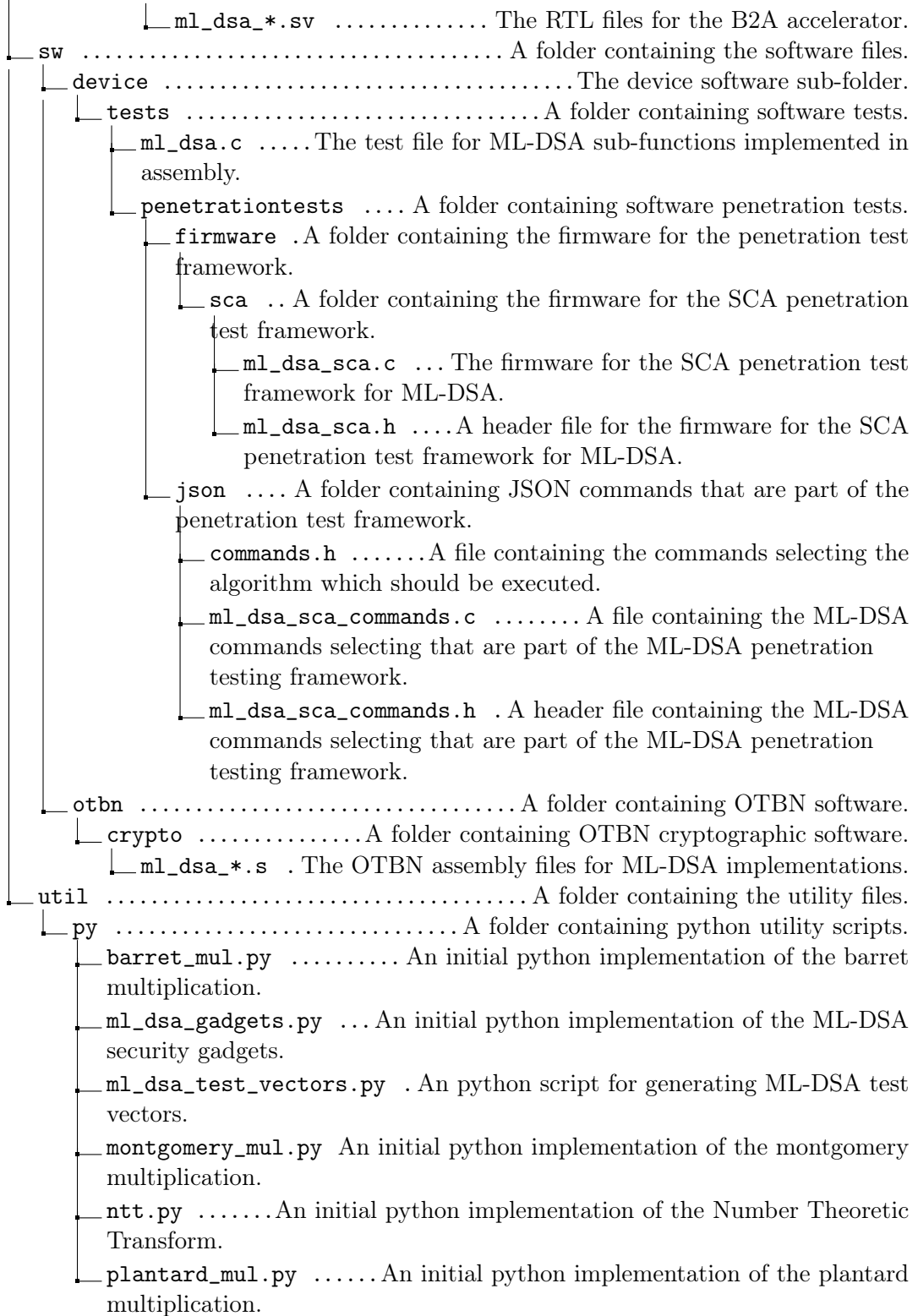
Below is the file structure of the ot-sca repository.

```
/ot-sca
├── analysis ..... A folder containing the relevant files for TVLA.
│   ├── configs ..... A folder TVLA configuration files.
│   │   └── tvla_cfg_ml_dsa.yaml ..... The TVLA configuration file for ML-DSA.
│   └── tvla.py ..... The TVLA script for ML-DSA.
├── capture ..... A folder containing the relevant files for capturing power traces.
│   ├── capture_ml_dsa.py ..... The capture script for ML-DSA.
│   ├── configs ..... A folder capture configuration files.
│   │   └── ml_dsa_sca_cw310.yaml .. The capture configuration file for ML-DSA on
│   │       cw310.
├── target ..... A folder containing files relating to different targets.
│   ├── communication A folder containing files used to communicate with the target.
│   └── sca_ml_dsa_commands.py ..... A python file containing functions used to
│       communicate with the ML-DSA capture framework on the target.
```

Below is the file structure of the opentitan repository.

```
/opentitan
├── hw ..... A folder containing the hardware files.
│   ├── ip ..... A folder containing the hardware IP blocks.
│   │   ├── otbn ..... A folder containing the OTBN IP block.
│   │   │   ├── pre_dv ..... A folder containing the DV files for the B2A accelerator.
│   │   └── rtl ..... A folder containing the RTL files for OTBN.
```


C. File Structure



C. File Structure

Bibliography

- [1] G. Cassiers and F.-X. Standaert, “Trivially and efficiently composing masked gadgets with probe isolating non-interference,” *IEEE Transactions on Information Forensics and Security*, vol. 15, pp. 2542–2555, 2020.
- [2] M. Azouaoui, O. Bronchain, G. Cassiers, C. Hoffmann, Y. Kuzovkova, J. Renes, M. Schönauer, T. Schneider, F.-X. Standaert, and C. van Vredendaal, “Protecting dilithium against leakage: Revisited sensitivity analysis and improved implementations,” *Cryptology ePrint Archive*, 2022.
- [3] N. I. of Standards and Technology. (2024) Module-lattice-based digital signature standard. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.204.pdf>
- [4] E. Kiltz, V. Lyubashevsky, and C. Schaffner, “A concrete treatment of fiat-shamir signatures in the quantum random-oracle model,” in *Advances in Cryptology—EUROCRYPT 2018: 37th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tel Aviv, Israel, April 29-May 3, 2018 Proceedings, Part III 37*. Springer, 2018, pp. 552–586.
- [5] A. Langlois and D. Stehlé, “Worst-case to average-case reductions for module lattices,” *Designs, Codes and Cryptography*, vol. 75, no. 3, pp. 565–599, 2015.
- [6] N. Sha and N. DRAFT, “standard: Permutation-based hash and extendable-output functions,” *FIPS PUB*, vol. 202, p. 2015, 3.
- [7] A. Satriawan, R. Mareta, and H. Lee. (2024) A complete beginner guide to the number theoretic transform (ntt). [Online]. Available: <https://eprint.iacr.org/2024/585.pdf>
- [8] J. W. Cooley and J. W. Tukey, “An algorithm for the machine calculation of complex fourier series,” *Mathematics of computation*, vol. 19, no. 90, pp. 297–301, 1965.

Bibliography

- [9] W. M. Gentleman and G. Sande, “Fast fourier transforms: for fun and profit,” in *Proceedings of the November 7-10, 1966, fall joint computer conference*, 1966, pp. 563–578.
- [10] S. Mangard, E. Oswald, and T. Popp, *Power analysis attacks: Revealing the secrets of smart cards*. Springer Science & Business Media, 2008, vol. 31.
- [11] S. Mangard, “A simple power-analysis (spa) attack on implementations of the aes key expansion,” in *International Conference on Information Security and Cryptology*. Springer, 2002, pp. 343–358.
- [12] C. Suresh, “Towards sound approaches to counteract power-analysis attacks,” in *Advances in Cryptology-CRYPTO’99*. Springer-Verlag, 1999, pp. 398–412.
- [13] O. Reparaz, B. Gierlichs, and I. Verbauwhede, “Fast leakage assessment,” in *Cryptographic Hardware and Embedded Systems—CHES 2017: 19th International Conference, Taipei, Taiwan, September 25-28, 2017, Proceedings*. Springer, 2017, pp. 387–399.
- [14] J. Huang, J. Zhang, H. Zhao, Z. Liu, R. C. Cheung, Ç. K. Koç, and D. Chen, “Improved plantard arithmetic for lattice-based cryptography,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2022, no. 4, pp. 614–636, 2022.
- [15] J. Huang, A. Adomnicăi, J. Zhang, W. Dai, Y. Liu, R. C. Cheung, Ç. K. Koç, and D. Chen, “Revisiting keccak and dilithium implementations on armv7-m,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2024, no. 2, pp. 1–24, 2024.
- [16] J. W. Bos, J. Renes, and A. Sprenkels, “Dilithium for memory constrained devices,” in *International Conference on Cryptology in Africa*. Springer, 2022, pp. 217–235.
- [17] A. Abdulrahman, V. Hwang, M. J. Kannwischer, and A. Sprenkels, “Faster kyber and dilithium on the cortex-m4,” in *International Conference on Applied Cryptography and Network Security*. Springer, 2022, pp. 853–871.
- [18] D. O. Greconici, M. J. Kannwischer, and A. Sprenkels, “Compact dilithium implementations on cortex-m3 and cortex-m4,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 1–24, 2021.
- [19] E. Alkim, Y. A. Bilgin, M. Cenk, and F. Gérard, “Cortex-m4 optimizations for {R, M} lwe schemes,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 336–357, 2020.
- [20] L. Botros, M. J. Kannwischer, and P. Schwabe, “Memory-efficient high-speed implementation of kyber on cortex-m4,” in *Progress in Cryptology—AFRICACRYPT 2019: 11th International Conference on Cryptology in Africa, Rabat, Morocco, July 9–11, 2019, Proceedings 11*. Springer, 2019, pp. 209–228.

Bibliography

- [21] H. Becker, V. Hwang, M. J. Kannwischer, B.-Y. Yang, and S.-Y. Yang, “Neon ntt: Faster dilithium, kyber, and saber on cortex-a72 and apple m1,” *Cryptology ePrint Archive*, 2021.
- [22] A. Abdulrahman, F. Oberhansl, H. N. H. Pham, J. Philipoom, P. Schwabe, T. Stelzer, and A. Zankl, “Towards ml-kem & ml-dsa on opentitan,” *Cryptology ePrint Archive*, 2024.
- [23] T. Stelzer, F. Oberhansl, J. Schupp, and P. Karl, “Enabling lattice-based post-quantum cryptography on the opentitan platform,” in *Proceedings of the 2023 Workshop on Attacks and Solutions in Hardware Security*, 2023, pp. 51–60.
- [24] P. Karl, J. Schupp, T. Fritzmann, and G. Sigl, “Post-quantum signatures on risc-v with hardware acceleration,” *ACM Transactions on Embedded Computing Systems*, vol. 23, no. 2, pp. 1–23, 2024.
- [25] P. Nannipieri, S. Di Matteo, L. Zulberti, F. Albicocchi, S. Saponara, and L. Fanucci, “A risc-v post quantum cryptography instruction set extension for number theoretic transform to speed-up crystals algorithms,” *IEEE Access*, vol. 9, pp. 150 798–150 808, 2021.
- [26] T. Fritzmann, G. Sigl, and J. Sepúlveda, “Risq-v: Tightly coupled risc-v accelerators for post-quantum cryptography,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 239–280, 2020.
- [27] L. Li, Q. Tian, G. Qin, S. Chen, and W. Wang, “Compact instruction set extensions for dilithium,” *ACM Transactions on Embedded Computing Systems*, vol. 23, no. 2, pp. 1–21, 2024.
- [28] L. Li, G. Qin, Y. Yu, and W. Wang, “Compact instruction set extensions for kyber,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 3, pp. 756–760, 2023.
- [29] Y. Zhao, R. Xie, G. Xin, and J. Han, “A high-performance domain-specific processor with matrix extension of risc-v for module-lwe applications,” *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 69, no. 7, pp. 2871–2884, 2022.
- [30] T. Fritzmann, M. Van Beirendonck, D. B. Roy, P. Karl, T. Schamberger, I. Verbauwhede, and G. Sigl, “Masked accelerators and instruction set extensions for post-quantum cryptography,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 414–460, 2022.
- [31] P. Ravi, M. F. Ezerman, S. Bhasin, A. Chattopadhyay, and S. S. Roy, “Will you cross the threshold for me? generic side-channel assisted chosen-ciphertext attacks on ntru-based kems,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 722–761, 2022.

Bibliography

- [32] P. Ravi, S. S. Roy, A. Chattopadhyay, and S. Bhasin, “Generic side-channel attacks on cca-secure lattice-based pke and kems,” *IACR transactions on cryptographic hardware and embedded systems*, pp. 307–335, 2020.
- [33] R. Ueno, K. Xagawa, Y. Tanaka, A. Ito, J. Takahashi, and N. Homma, “Curse of re-encryption: A generic power/em analysis on post-quantum kems,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 296–322, 2022.
- [34] P. Ravi, R. Poussier, S. Bhasin, and A. Chattopadhyay, “On configurable sca countermeasures against single trace attacks for the ntt: A performance evaluation study over kyber and dilithium on the arm cortex-m4,” in *Security, Privacy, and Applied Cryptography Engineering: 10th International Conference, SPACE 2020, Kolkata, India, December 17–21, 2020, Proceedings 10*. Springer, 2020, pp. 123–146.
- [35] M. Hamburg, J. Hermelink, R. Primas, S. Samardjiska, T. Schamberger, S. Streit, E. Strieder, and C. Van Vredendaal, “Chosen ciphertext k-trace attacks on masked cca2 secure kyber,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 88–113, 2021.
- [36] M. J. Kannwischer, P. Pessl, and R. Primas, “Single-trace attacks on keccak,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 243–268, 2020.
- [37] R. Primas, P. Pessl, and S. Mangard, “Single-trace side-channel attacks on masked lattice-based encryption,” in *Cryptographic Hardware and Embedded Systems—CHES 2017: 19th International Conference, Taipei, Taiwan, September 25–28, 2017, Proceedings*. Springer, 2017, pp. 513–533.
- [38] M. Azouaoui, O. Bronchain, C. Hoffmann, Y. Kuzovkova, T. Schneider, and F.-X. Standaert, “Systematic study of decryption and re-encryption leakage: The case of kyber,” in *International Workshop on Constructive Side-Channel Analysis and Secure Design*. Springer, 2022, pp. 236–256.
- [39] Y. Liu, Y. Zhou, S. Sun, T. Wang, R. Zhang, and J. Ming, “On the security of lattice-based fiat-shamir signatures in the presence of randomness leakage,” *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 1868–1879, 2020.
- [40] V. Migliore, B. Gérard, M. Tibouchi, and P.-A. Fouque, “Masking dilithium: Efficient implementation and side-channel evaluation,” in *Applied Cryptography and Network Security: 17th International Conference, ACNS 2019, Bogota, Colombia, June 5–7, 2019, Proceedings 17*. Springer, 2019, pp. 344–362.
- [41] A. R. Shahmirzadi and M. Hutter, “Efficient boolean-to-arithmetic mask conversion in hardware,” *Cryptology ePrint Archive*, 2024.
- [42] L. Goubin, “A sound method for switching between boolean and arithmetic masking,” in *International Workshop on Cryptographic Hardware and Embedded Systems*. Springer, 2001, pp. 3–15.

Bibliography

- [43] J.-S. Coron, J. Großschädl, M. Tibouchi, and P. K. Vadnala, “Conversion from arithmetic to boolean masking with logarithmic complexity,” in *Fast Software Encryption: 22nd International Workshop, FSE 2015, Istanbul, Turkey, March 8-11, 2015, Revised Selected Papers 22*. Springer, 2015, pp. 130–149.
- [44] A. Biryukov, D. Dinu, Y. Le Corre, and A. Udovenko, “Optimal first-order boolean masking for embedded iot devices,” in *Smart Card Research and Advanced Applications: 16th International Conference, CARDIS 2017, Lugano, Switzerland, November 13–15, 2017, Revised Selected Papers*. Springer, 2018, pp. 22–41.
- [45] Y.-S. Won and D.-G. Han, “Efficient conversion method from arithmetic to boolean masking in constrained devices,” in *International Workshop on Constructive Side-Channel Analysis and Secure Design*. Springer, 2017, pp. 120–137.
- [46] Y. Komano, H. Shimizu, and H. Miyake, “Integrative acceleration of first-order boolean masking for embedded iot devices,” *Journal of Information Processing*, vol. 27, pp. 585–592, 2019.
- [47] G. Barthe, S. Belaïd, T. Espitau, P.-A. Fouque, B. Grégoire, M. Rossi, and M. Tibouchi, “Masking the glp lattice-based signature scheme at any order,” in *Advances in Cryptology–EUROCRYPT 2018: 37th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tel Aviv, Israel, April 29–May 3, 2018 Proceedings, Part II 37*. Springer, 2018, pp. 354–384.
- [48] T. Schneider, C. Paglialonga, T. Oder, and T. Güneysu, “Efficiently masking binomial sampling at arbitrary orders for lattice-based crypto,” in *IACR International Workshop on Public Key Cryptography*. Springer, 2019, pp. 534–564.
- [49] F. Bache, C. Paglialonga, T. Oder, T. Schneider, and T. Güneysu, “High-speed masking for polynomial comparison in lattice-based kems,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 483–507, 2020.
- [50] S. Bhasin, J.-P. D’Anvers, D. Heinz, T. Pöppelmann, and M. Van Beirendonck, “Attacking and defending masked polynomial comparison for lattice-based cryptography,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 334–359, 2021.
- [51] O. Bronchain and G. Cassiers, “Bitslicing arithmetic/boolean masking conversions for fun and profit: with application to lattice-based kems,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 553–588, 2022.
- [52] J.-S. Coron, F. Gérard, S. Montoya, and R. Zeitoun, “High-order polynomial comparison and masking lattice-based encryption,” *Cryptology ePrint Archive*, 2021.
- [53] T. Schneider, A. Moradi, and T. Güneysu, “Arithmetic addition over boolean masking: Towards first-and second-order resistance in hardware,” in *Applied Cryptography and Network Security: 13th International Conference, ACNS 2015, New York, NY, USA, June 2-5, 2015, Revised Selected Papers 13*. Springer, 2015, pp. 559–578.

Bibliography

- [54] H. Groß, M. Jelinek, S. Mangard, T. Unterluggauer, and M. Werner, “Concealing secrets in embedded processors designs,” in *Smart Card Research and Advanced Applications: 15th International Conference, CARDIS 2016, Cannes, France, November 7–9, 2016, Revised Selected Papers 15*. Springer, 2017, pp. 89–104.
- [55] M. V. Beirendonck, J.-P. D’anvers, A. Karmakar, J. Balasch, and I. Verbauwhede, “A side-channel-resistant implementation of saber,” *ACM Journal on Emerging Technologies in Computing Systems (JETC)*, vol. 17, no. 2, pp. 1–26, 2021.
- [56] F. Bache and T. Güneysu, “Boolean masking for arithmetic additions at arbitrary order in hardware,” *Applied Sciences*, vol. 12, no. 5, p. 2274, 2022.
- [57] G. Cassiers, B. Gigerl, S. Mangard, C. Momin, and R. Nagpal, “Compress: Reducing area and latency of masked pipelined circuits,” *IACR Cryptol. ePrint Arch.*, vol. 2023, p. 1600, 2023.
- [58] Q. Norga, J.-P. D’Anvers, S. Kundu, and I. Verbauwhede, “Mask conversions for $d+1$ shares in hardware, with application to lattice-based pqc,” 2024.
- [59] L. Ducas, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, and D. Stehlé, “Crystals-dilithium: Digital signatures from module lattices,” 2018.
- [60] S. Marzougui, V. Ulitzsch, M. Tibouchi, and J.-P. Seifert, “Profiling side-channel attacks on dilithium: A small bit-fiddling leak breaks it all,” *Cryptology ePrint Archive*, 2022.
- [61] A. Berzati, A. C. Viera, M. Chartouni, S. Madec, D. Vergnaud, and D. Vigilant, “A practical template attack on crystals-dilithium,” *IACR Cryptol. ePrint Arch.*, vol. 2023, p. 50, 2023.
- [62] pq crystals. (2023, November) Dilithium. [Online]. Available: <https://github.com/pq-crystals/dilithium>
- [63] T. Plantard, “Efficient word size modular arithmetic,” *IEEE Transactions on Emerging Topics in Computing*, vol. 9, no. 3, pp. 1506–1518, 2021.
- [64] J.-S. Coron, F. Gérard, M. Trannoy, and R. Zeitoun, “Improved gadgets for the high-order masking of dilithium,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2023, no. 4, pp. 110–145, 2023.
- [65] J. Huang, H. Zhao, J. Zhang, W. Dai, L. Zhou, R. C. Cheung, Ç. K. Koç, and D. Chen, “Yet another improvement of plantard arithmetic for faster kyber on low-end 32-bit iot devices,” *IEEE Transactions on Information Forensics and Security*, 2024.
- [66] G. Barthe, S. Belaïd, F. Dupressoir, P.-A. Fouque, B. Grégoire, P.-Y. Strub, and R. Zucchini, “Strong non-interference and type-directed higher-order masking,” in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, 2016, pp. 116–129.

Bibliography